
Flexible Kernels for Protein Property Prediction

Martin Jankowiak¹ Yerdos Ordabayev¹ Rudraksh Tuwani¹ Henry N. Ward¹ Hunter Nisonoff¹
James M. McFarland¹ Gevorg Grigoryan¹

Abstract

Despite its importance to applications in protein design, predicting protein properties like binding affinity and thermostability from sparse experimental data remains a significant challenge. Accordingly, we introduce a class of sequence kernels that exploit evolutionary substitution matrices as well as local linearity and demonstrate that the resulting Gaussian processes provide data-efficient models of protein property landscapes, frequently outperforming alternatives that rely on foundation model embeddings. Furthermore—by learning what are in effect structure-aware substitution matrices—we show that our kernels can readily incorporate structural information from foundation models. We demonstrate that these structure-conditioned kernels are well suited to multi-task learning across multiple protein property landscapes and can decisively outperform local supervised learning methods.

1. Introduction

The ability to accurately predict protein properties in lieu of measuring them is an essential tool in protein design. While much recent attention has focused on all-atom generative models, the utility of high-fidelity discriminators based on structure predictors like AlphaFold2 (Jumper et al., 2021) is no less evident (Pacesa et al., 2025), and we contend that the entire generative-to-discriminative continuum is essential to the toolkit of the well-equipped protein designer. With that motivation in mind, we set out to develop accurate models of protein property landscapes. We are particularly interested in methods that are: i) data-efficient; ii) endowed with high-quality uncertainty estimates; and iii) well-suited to multi-task learning across multiple landscapes. Below we argue theoretically and demonstrate empirically that

¹Generate Biomedicines, Somerville, MA, USA. Correspondence to: Martin Jankowiak <jankowiak@gmail.com>, Gevorg Grigoryan <gevorg@generatebiomedicines.com>.

Gaussian processes (GPs) with suitably constructed kernels are a good fit for all three criteria. Our contributions include:

1. comprehensive benchmarking of 30+ protein property predictors on 21 datasets in three different regimes
2. a sequence-only GP that frequently outperforms more complex (structure-conditioned) predictors that rely on foundation models with millions of parameters
3. a simple but powerful recipe for training multi-task GPs that leverage foundation model embeddings to form zero-shot structure-conditioned kernels

2. Background

2.1. Problem Description

An *empirical protein property landscape* consists of N protein sequences $\{\mathbf{x}_n\}$, where each protein is associated with a scalar property t_n . Each sequence is assumed to be of a fixed length¹ L over a fixed alphabet of size A . Throughout we assume a one-hot sequence encoding in which $\mathbf{x}$ is represented as a binary matrix of shape $L \times A$ with entries $x_{\ell a}$. Additionally, each landscape is optionally associated with a reference sequence $\mathbf{x}_{\text{ref}}$ and a corresponding reference structure $\mathcal{S}$. Our aim is to learn a predictive model that can predict t for any sequence $\mathbf{x}$.²

2.2. Gaussian Processes

GPs offer powerful non-parametric function priors that often perform well in data scarce regimes. A GP on the input space $\mathcal{X}$ is specified by a covariance function or kernel $k : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ (Rasmussen, 2003). In order to be a valid kernel the $N \times N$ matrix $k(\mathbf{X}, \mathbf{X})$ must be a valid covariance matrix—i.e. positive semidefinite—for all $\mathbf{X} = \{\mathbf{x}_n\}_{n=1}^N$ with $\mathbf{x}_n \in \mathcal{X}$. For scalar regression³ $f : \mathcal{X} \rightarrow \mathbb{R}$ the joint density of a GP with zero prior mean takes the form

$$p(\mathbf{t}, \mathbf{f} | \mathbf{X}) = \mathcal{N}(\mathbf{t} | \mathbf{f}, \sigma_n^2 \mathbf{1}_N) \mathcal{N}(\mathbf{f} | \mathbf{0}, K_{\mathbf{X}\mathbf{X}}) \quad (1)$$

¹It is straightforward to handle variable length proteins, provided that sequences are aligned and we introduce gap tokens.

²In Sec. 5.4 we consider multi-task models trained on multiple related protein property landscapes.

³Apart from the experiments in Sec. 5.6 we focus on regression.

where $\mathbf{t}$ are the real-valued targets, $\mathbf{f}$ are the latent function values, σ_n^2 is the variance of the Normal likelihood $\mathcal{N}(\mathbf{t}|\cdot)$, and $K_{\mathbf{X}\mathbf{X}}$ is the $N \times N$ kernel matrix. The marginal likelihood of the observed data can be computed in closed form

$$p(\mathbf{t}|\mathbf{X}) = \int d\mathbf{f} p(\mathbf{t}, \mathbf{f}|\mathbf{X}) = \mathcal{N}(\mathbf{t}|\mathbf{0}, K_{\mathbf{X}\mathbf{X}} + \sigma_n^2 \mathbf{1}_N) \quad (2)$$

and can be maximized with gradient methods to fit kernel hyperparameters. The posterior predictive distribution of the GP at a query point $\mathbf{x}^* \in \mathcal{X}$ is the Normal distribution $\mathcal{N}(\mu_{\mathbf{f}}(\mathbf{x}^*), \sigma_{\mathbf{f}}(\mathbf{x}^*)^2)$ where $\mu_{\mathbf{f}}(\cdot)$ and $\sigma_{\mathbf{f}}(\cdot)^2$ are given by

$$\mu_{\mathbf{f}}(\mathbf{x}^*) = k_{*\mathbf{X}}^T (K_{\mathbf{X}\mathbf{X}} + \sigma_n^2 \mathbf{1}_N)^{-1} \mathbf{t} \quad (3)$$

$$\sigma_{\mathbf{f}}(\mathbf{x}^*)^2 = k_{**} - k_{*\mathbf{X}}^T (K_{\mathbf{X}\mathbf{X}} + \sigma_n^2 \mathbf{1}_N)^{-1} k_{*\mathbf{X}} \quad (4)$$

and where e.g. $k_{**} \equiv k(\mathbf{x}^*, \mathbf{x}^*)$. Computing Eqn. 2-4 is cubic in the number of data points N . Unless noted otherwise, we use the posterior mean Eqn. 3 for prediction.

2.3. Elementary Sequence Kernels

Two of the simplest kernels that can be defined on one-hot sequence space are the linear and RBF kernel. The (isotropic) linear kernel is defined as

$$k_{\text{lin}}(\mathbf{x}, \mathbf{y}) = \mathbf{x}^T \mathbf{y} = \sum_{\ell=1}^L \mathbf{x}_{\ell}^T \mathbf{y}_{\ell} = \sum_{\ell=1}^L \sum_{a=1}^A x_{\ell a} y_{\ell a} \quad (5)$$

and can be written in terms of the Hamming distance d_H as

$$k_{\text{lin}}(\mathbf{x}, \mathbf{y}) = L - d_H(\mathbf{x}, \mathbf{y}) \in \{0, 1, \dots, L\} \quad (6)$$

The kernel in Eqn. 5 is called linear, since it corresponds to Bayesian linear regression with $f(\mathbf{x}) = \beta \cdot \mathbf{x}$, where an isotropic Gaussian prior has been placed on the coefficients $\beta \in \mathbb{R}^{L \times A}$. In other words it assumes no prior structure among the coefficients $\beta_{\ell a}$. This means that, for example, observing a mutation $8A \rightarrow 8V$ does not inform the effects of other mutations like $8A \rightarrow 8I$.

In contrast to Eqn. 5, which has an additive structure, the RBF kernel has a multiplicative structure:

$$k_{\text{rbf}}(\mathbf{x}, \mathbf{y}) = \prod_{\ell=1}^L \exp\left(-\frac{1}{2} \|\mathbf{x}_{\ell} - \mathbf{y}_{\ell}\|^2 / \tau_{\ell}^2\right) \quad (7)$$

where $\tau_{\ell} > 0$ is a lengthscale. Unlike k_{lin} , the RBF kernel decays exponentially as $d_H(\mathbf{x}, \mathbf{y})$ increases. Since $\|\mathbf{x}_{\ell} - \mathbf{y}_{\ell}\|^2 \in \{0, 2\}$ for one-hot sequences, Eqn. 7 is more naturally written as

$$k_{\text{rbf}}(\mathbf{x}, \mathbf{y}) = \prod_{\ell=1}^L \exp\left(\frac{-1}{\tau_{\ell}^2}\right)^{\mathbb{1}(\mathbf{x}_{\ell} \neq \mathbf{y}_{\ell})} = \prod_{\ell=1}^L \rho_{\ell}^{\mathbb{1}(\mathbf{x}_{\ell} \neq \mathbf{y}_{\ell})} \quad (8)$$

where $\rho_{\ell} \equiv \exp(-1/\tau_{\ell}^2) < 1$ and $\mathbb{1}(\mathbf{x}_{\ell} \neq \mathbf{y}_{\ell}) \in \{0, 1\}$ encodes whether sequences $\mathbf{x}$ and $\mathbf{y}$ have the same amino

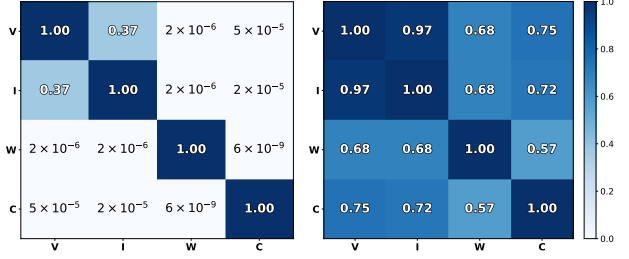


Figure 1. The BLOSUM50 substitution matrix as a correlation matrix. We zoom-in on four amino acids, two of which are bio-physically similar (valine and isoleucine) and two of which are largely dissimilar to all other amino acids (tryptophan and cysteine). The matrix on the right has been raised to the power 0.03. For details on the normalization scheme we use refer to Sec. A.3. For the full matrix see Fig. 16.

acid at position ℓ . That is to compute $k_{\text{rbf}}(\mathbf{x}, \mathbf{y})$ you take all the positions where $\mathbf{x}$ and $\mathbf{y}$ differ and multiply together the corresponding ρ_{ℓ} , with small ρ_{ℓ} corresponding to low similarity between $\mathbf{x}_{\ell}$ and $\mathbf{y}_{\ell}$. Since the RBF kernel does not distinguish between different amino acids at a given position, observing a mutation $8A \rightarrow 8V$ does not inform the effects of other mutations like $8A \rightarrow 8I$.

To connect with familiar concepts, we have introduced the RBF kernel via its canonical form in Eqn. 7, which is equally applicable to euclidean inputs $\mathbf{x}$. In the context of one-hot encoded sequences $\mathbf{x}$, however, this framing is a bit misleading, since it obscures that for such discrete $\mathbf{x}$ the RBF kernel is best viewed in terms of a product over covariance matrices:

$$k_{\text{rbf}}(\mathbf{x}, \mathbf{y}) = \prod_{\ell=1}^L S_{\ell a_{\ell}(\mathbf{x}) a_{\ell}(\mathbf{y})} = \prod_{\ell=1}^L \mathbf{x}_{\ell}^T \mathbf{S}_{\ell} \mathbf{y}_{\ell} \quad (9)$$

Here each $\mathbf{S}_{\ell}$ is a covariance matrix of size $A \times A$, and $a_{\ell}(\mathbf{x})$ is the amino acid index of sequence $\mathbf{x}$ at position ℓ . Viewed through this lens Eqn. 8 is a special case of Eqn. 9 where $\mathbf{S}_{\ell}$ is a correlation matrix with ones along the diagonal and all off-diagonal entries given by ρ_{ℓ} . For generic correlation matrices $\mathbf{S}_{\ell}$, Eqn. 9 can be understood as an *anisotropic* RBF kernel; we will make use of this straightforward generalization in Sec. 3.3.

3. Flexible Protein Sequence Kernels

Before we introduce our kernel in Sec. 3.3-3.4, we lay important foundations in Sec. 3.1-3.2. We describe the priors we place on kernel hyperparameters in Sec. 3.5. Then in Sec. 3.6 we show how to generalize our construction to *structure-conditioned* kernels.

3.1. Substitution Matrices

We begin with a short discussion of substitution matrices,⁴ a classical bioinformatics tool that quantifies how likely one amino acid is to replace another during evolution. Substitution matrices are derived empirically by counting substitutions in aligned homologous sequences and converting those frequencies into log-odds scores. In practice we find it convenient to work with the correlation matrix form of substitution matrices, i.e. for substitution matrix $\mathbf{S}$ we define the corresponding correlation matrix as

$$C_{aa'} \equiv S_{aa'} / \sqrt{S_{aa} S_{a'a'}} \quad (10)$$

In this way, an RBF-like kernel as in Eqn. 9 is equipped with a natural normalization in which $k(\mathbf{x}, \mathbf{x}) = 1$ for all $\mathbf{x}$. Moreover, the diagonal entries of BLOSUM substitution matrices—which we use throughout—encode the *absolute* level of conservation of each amino acid in evolutionary contexts, whereas we are interested in encoding *relative* similarity (Henikoff & Henikoff, 1992). See Fig. 1 for an illustration of a BLOSUM50-derived correlation matrix.

We make the interesting observation that while many substitution matrices (e.g. most PAM matrices (Dayhoff, 1972)) are positive semi-definite (PSD), some substitution matrices—in particular most of the BLOSUM family—have the further property that they are *infinitely divisible*. That is for these matrices elementwise exponentiation by a positive power (i.e. the Hadamard power S^{α} given by $(S^{\alpha})_{ij} = (S_{ij})^{\alpha}$) preserves positive semi-definiteness.⁵ The infinite divisibility of BLOSUM matrices makes them uniquely suited as ingredients in protein sequence kernels, since including learnable exponents as part of the hyperparameters of a GP kernel offers a powerful and parsimonious mechanism for modulating the covariance structure of the kernel. Powers greater than unity attenuate off-diagonal similarities $C_{aa'}$, while powers less than unity amplify them. Thus by learning landscape-specific exponents the biophysical similarity encoded by these substitution matrices can be adapted to the landscape at hand. We return to these exponents in Sec. 3.5 when we discuss hyperparameter priors.

3.2. Local Linearity

It has long been recognized that in many cases protein properties are approximately additive w.r.t. individual mutations (Wells, 1990) and that pairwise epistasis effects are often modest, at least once global epistasis is taken into account

⁴In the bioinformatics literature substitution matrices are usually reported as log-odds scores $M_{ij} = \log(\frac{q_{ij}}{p_i p_j})$ (Altschul, 1991). We work with their elementwise exponentiated form $S_{ij} \equiv e^{M_{ij}}$ (sometimes called an odds-ratio matrix), and throughout we refer to S as the substitution matrix.

⁵See Table 7 for an inventory of substitution matrices and Sec. A.1 for background on infinite divisibility.

(Otwinowski et al., 2018). We propose to incorporate this observation in our modeling approach by explicitly building local linearity into our kernel.⁶ This is straightforward to do, since any kernel of the form $k' k_{\text{lin}}$ —where k' is any kernel and k_{lin} is a linear kernel—is naturally viewed as describing a *locally linear* class of functions.

In more detail, let $f(\mathbf{x}) = \beta \cdot \mathbf{x}$ be a linear model defined on one-hot sequences $\mathbf{x}$ with coefficients β . If we promote β to $\mathbf{x}$ -dependent coefficients $\beta(\mathbf{x})$ and place a GP prior on $\beta(\mathbf{x})$ controlled by k' , then the resulting model is equivalent to a GP with product kernel $k' k_{\text{lin}}$.⁷ Indeed if k' is chosen so that $\beta(\mathbf{x})$ varies slowly across sequence space, $f(\mathbf{x}) \approx \beta(\mathbf{x}_0) \cdot \mathbf{x}$ in the vicinity of $\mathbf{x}_0$, i.e. $f(\mathbf{x})$ is locally linear. For additional details and the elementary proof see Sec. A.2.

3.3. LOCK: Locally Linear Correlation Kernels

We combine the ingredients from Sec. 3.1-3.2 to define our kernel: the Locally Linear Correlation Kernel (**LOCK**). We first define the two base kernels from which the LOCK kernel is constructed. The first is the linear kernel,

$$k_{\text{lin}}^{\text{LOCK}}(\mathbf{x}, \mathbf{y}) = \sum_{\ell=1}^L \mathbf{x}_{\ell}^T \mathbf{C}_{\ell}^{\alpha_{\ell}} \mathbf{y}_{\ell} \quad (11)$$

where $\mathbf{C}_{\ell}$ are correlation matrices as in Eqn. 10 and $\alpha_{\ell} > 0$ are (optional) learnable exponents.⁸ The kernel in Eqn. 11 corresponds to Bayesian linear regression with coefficients $\beta \in \mathbb{R}^{L \times A}$ governed by a block diagonal covariance matrix with blocks $\mathbf{C}_{\ell}^{\alpha_{\ell}}$. The second can be seen as a generalization of the RBF kernel in Eqn. 7:

$$k_{\text{nl}}^{\text{LOCK}}(\mathbf{x}, \mathbf{y}) = \prod_{\ell=1}^L \mathbf{x}_{\ell}^T \mathbf{C}_{\ell}^{\alpha_{\ell}} \mathbf{y}_{\ell} \quad (12)$$

Evidently Eqn. 12 is the multiplicative analog of Eqn. 11.⁹

With Eqn. 11-12 in hand, we can define the *locally linear correlation kernel* $k^{\text{LOCK}}(\mathbf{x}, \mathbf{y})$ as

$$\sigma_1^2 k_{\text{nl}}^{\text{LOCK}}(\mathbf{x}, \mathbf{y}) k_{\text{lin}}^{\text{LOCK}}(\mathbf{x}, \mathbf{y}) + \sigma_2^2 \tilde{k}_{\text{lin}}^{\text{LOCK}}(\mathbf{x}, \mathbf{y}) \quad (13)$$

where we have introduced learnable kernel scales $\sigma_1, \sigma_2 > 0$. Due to the inclusion of the product kernel $k_{\text{nl}}^{\text{LOCK}} k_{\text{lin}}^{\text{LOCK}}$, the kernel k^{LOCK} is *locally linear* as described in Sec. 3.2. For more flexibility we also include the second linear kernel $\tilde{k}_{\text{lin}}^{\text{LOCK}}$ (see Sec. 3.4 & Sec. A.2 for additional motivation). Besides depending on correlation matrices $\mathbf{C}_{\ell}$, we

⁶For a penalized interpolation method that utilizes a particular notion of Laplacian roughness to limit deviations from additivity see (Zhou & McCandlish, 2020).

⁷This general observation has been leveraged by Yoshikawa & Iwata (2023) in the context of explainable AI.

⁸Note that in Eqn. 11 we allow for the possibility that the correlation matrices vary from position to position.

⁹Importantly, since the entries of $\mathbf{C}_{\ell}$ are positive for infinitely divisible matrices, Eqn. 12 can be computed in log space for improved numerical stability: $\exp(\sum_{\ell=1}^L \alpha_{\ell} \mathbf{x}_{\ell}^T \log^{\circ}(\mathbf{C}_{\ell}) \mathbf{y}_{\ell})$

emphasize that each of the three base kernels in Eqn. 13 optionally depend on additional learnable exponents, a modeling choice that we discuss in more detail in Sec. 3.5.

Unless noted otherwise, we use the same **BLOSUM50**-derived correlation matrix at each position.

3.4. Kernel Properties

Above we have emphasized two properties of the LOCK kernel k^{LOCK} : i) local linearity; and ii) how it incorporates prior information from substitution matrices. Here we touch on a few additional properties of the kernel.¹⁰

Basis Functions It is helpful to view k^{LOCK} as defining a function space built from ‘rotated’ amino acid basis functions

$$f_{\ell a}(\mathbf{x}) = (\mathbf{L}_{\ell}^T \mathbf{x})_a \quad \text{with} \quad \mathbf{L}_{\ell} \equiv \text{Cholesky}(\mathbf{C}_{\ell})$$

so that $\mathbf{L}_{\ell} \mathbf{L}_{\ell}^T = \mathbf{C}_{\ell}$. For example if we refer to the BLOSUM50 correlation matrix in Fig. 1 we can infer that one basis function puts significant weight on both valine and isoleucine, since these are highly correlated. Thus the linear kernel $k_{\text{lin}}^{\text{LOCK}}$ can be seen as Bayesian linear regression w.r.t. the L A basis functions $f_{\ell a}$ with an *isotropic* prior covariance in the rotated basis. By contrast the non-linear kernel $k_{\text{nl}}^{\text{LOCK}}$ utilizes a *tensor-product basis* made up of A^L composite basis functions, each of which is a L -wise product of $f_{\ell a}$. This basis function perspective makes it manifest how observing a mutation like $8A \rightarrow 8V$ informs the effects of other mutations like $8A \rightarrow 8I$.

Kernel Decay When using a model trained on fitness data in model-based protein design, an important consideration is how that model performs in interpolatory (close to the training data) and extrapolatory (away from the training data) regimes. Consider a GP with a linear kernel like in Eqn. 11. It assumes perfect additivity, even for sequences that differ from the training data by tens of mutations. Put differently, since the kernel decays slowly (in particular linearly) as the Hamming distance to the training data increases, the linear kernel extrapolates aggressively. By contrast a multiplicative kernel like in Eqn. 12 decays exponentially as the Hamming distance to the training data increases. Such a kernel extrapolates more conservatively; in particular it is *mean-reverting*, i.e. the GP predictor asymptotes towards the prior mean far away from the training data.

Neither of these behaviors is particularly attractive in the context of model-based design, and by construction the LOCK kernel in Eqn. 13 avoids both these extremes. Near the training data ($k_{\text{nl}}^{\text{LOCK}} \gg 0$) it provides locally linear predictions that respond to the nuances of the local sequence data. Instead of reverting to the prior mean far away from

the training data ($k_{\text{nl}}^{\text{LOCK}} \approx 0$), it reverts to a linear predictor controlled by $\tilde{k}_{\text{lin}}^{\text{LOCK}}$. This is arguably the best of both worlds: nuanced non-linear predictions in the vicinity of the training data that smoothly yield to a simple and robust linear predictor further away—all informed by the biophysical knowledge encoded in substitution matrices.

3.5. Hyperparameter Priors

With two kernel scales σ_1 and σ_2 and the noise scale σ_n (see Eqn. 2) the LOCK-GP defined by Eqn. 13 has at least three hyperparameters. As for the exponents for each of the three base kernels, we can consider various levels of flexibility:

1. no exponents (i.e. all exponents are unity)
2. a global exponent α that is the same for all positions ℓ
3. local exponents α_{ℓ} that vary with ℓ

Crucially, these hyperparameters allow the LOCK kernel to adapt to the landscape at hand and are readily fit by maximizing the GP marginal likelihood in Eqn. 2 using gradient methods. Particularly in the case where the exponents α_{ℓ} are allowed to vary with ℓ , it is important to regularize the hyperparameters, both for numerical stability as well as to mitigate against possible overfitting. Unless noted otherwise, we utilize the following regularized hyperparameters:

1. $k_{\text{nl}}^{\text{LOCK}}$ is equipped with local exponents α_{ℓ}
2. $k_{\text{lin}}^{\text{LOCK}}$ and $\tilde{k}_{\text{lin}}^{\text{LOCK}}$ each receive a global exponent
3. σ_1^2 , σ_2^2 , and σ_n^2 are governed by weak Gamma priors
4. global exponents are governed by weak priors
5. local exponents are governed by relatively strong priors

We find that these choices work well across a variety of scenarios; see Sec. A.3 for the precise hyperpriors used. In experiments in Sec. 5.2 we ablate some of these choices.

3.6. CLOCK: Structure-conditioned Kernels

The LOCK kernel is a sequence kernel that depends on global protein data solely through pre-canned substitution matrices. In this section we show that k^{LOCK} yields a natural generalization to *conditional* kernels that we dub **CLOCK**. The result is a powerful and elegant mechanism for incorporating protein foundation models into LOCK-GPs.

The core idea is that we map positional structure embeddings $\mathbf{h}_{1:L}(\mathcal{S})$ obtained from a pre-trained foundation model to positional correlation matrices $\mathbf{C}_{1:L}$, each of size $A \times A$.¹¹ These correlation matrices, which are specialized to the local structural context as encoded by each $\mathbf{h}_{\ell}$, are then used to construct (zero-shot) LOCK kernels as in Sec. 3.3.¹² Each

¹¹We could instead use protein language model embeddings that do not rely on structure, but we do not explore that possibility here.

¹²The user can choose how many kernel hyperparameters to

¹⁰For a discussion of epistasis see Sec. A.4.

correlation matrix is parameterized as

$$C_{\ell aa'} = \exp(-\|\mathbf{z}_{\ell a} - \mathbf{z}_{\ell a'}\|^2) \quad (14)$$

where $\mathbf{z}_{\ell a} \in \mathbb{R}^{A-1}$ is obtained from $\mathbf{h}_{\ell}$ via a linear map:

$$\mathbf{z}_{\ell} = \mathbf{W}\mathbf{h}_{\ell} \quad \text{where} \quad \mathbf{z}_{\ell} \in \mathbb{R}^{A \times (A-1)} \quad (15)$$

Thus $\mathbf{C}_{\ell}$ can be thought of as an RBF kernel defined on an abstract amino acid embedding space with embeddings $\mathbf{z}_{\ell a}$. Importantly, this parameterization is *generic* in that each $\mathbf{C}_{\ell}$ is infinitely divisible and all infinitely divisible correlation matrices are of the form in Eqn. 14, see Sec. A.1.

To learn $\mathbf{W}$ we use a simple and robust training procedure based on the so-called concentrated form of the standard GP training objective in which the overall kernel scale is ‘profiled’ out (see Sec. A.8 for details). Since $\mathbf{W}$ has $\sim 49k$ parameters for $A = 20$ and 128-dimensional embeddings $\mathbf{h}_{\ell}$, this setup is most appropriate for settings with sufficient data. In particular it is an attractive approach for *multi-task learning* across multiple related protein property landscapes, as we demonstrate empirically in Sec. 5.4.

4. Related Work

Quantitative modeling of protein sequence–function relationships has a long history, with early studies employing partial least squares regression (PLS) and Gaussian processes to model properties such as enzyme activity and thermostability (Fox et al., 2003; Romero et al., 2013). Much recent work has focused on zero-shot methods that utilize protein language models (PLMs), typically via masked-token pseudo-likelihoods or autoregressive log-likelihood ratios aggregated over mutated sites (Meier et al., 2021; Rives et al., 2021). Alongside MSA-based generative models such as DeepSequence and EVE (Riesselman et al., 2018; Frazer et al., 2021), these likelihood-based scores achieve strong zero-shot performance on ProteinGym (Notin et al., 2023a).

A popular approach to supervised learning on protein property landscapes uses PLMs like ESM-2 (Lin et al., 2023) as feature extractors and fits a lightweight supervised head (Alley et al., 2019; Elnaggar et al., 2022). This approach can be extended to supervised fine-tuning, where the weights of the PLM are adapted to the supervised task (Rao et al., 2019; Brandes et al., 2022), and includes methods that fine-tune likelihood-based scores with ranking- or regression-based losses (Zhao et al., 2024; Hawkins-Hooker et al., 2024). Indeed several works in the literature benchmark a variety of such recipes, including for example those that employ parameter-efficient adapters (Schmirler et al., 2024; Sledzieski et al., 2024; Zhou et al., 2024; Bikias et al., 2025). While recognizing the broad utility of this line of work,

adapt to the local landscape. In most cases learning the kernel and noise scales (at a minimum) is likely prudent.

some authors have observed that improved performance on the PLM pretraining task can fail to translate to the supervised task (Li et al., 2024; Vieira et al., 2025). Likewise, despite the demonstrated value of this family of methods, a number of disadvantages must also be acknowledged, including the large number of training hyperparameters, the high computational cost, and the risk of overfitting.

As of Jan. 2026, the top performer on ProteinGym’s supervised DMS substitutions benchmark is the Kermut GP (Groth et al., 2024), which utilizes ESM-2 embeddings, an ESM-2-based zero-shot prior mean, 3d structure coordinates, and ProteinMPNN inverse folding logits to construct a kernel that sums over pairs of mutations.¹³ As shown in Groth et al. (2024), Kermut convincingly outperforms strong baselines like ProteinNPT (Notin et al., 2023b). Product kernels along the lines of Eqn. 12 have been explored by Zhou et al. (2025). Researchers have also explored other kernels, including e.g. fingerprint kernels on BLOSUM-derived embeddings (Gessner et al., 2024) and string kernels on mutation codes (Benjamins et al., 2024). For a theoretical discussion of biological sequence kernels please refer to Amin et al. (2025). For a recent analysis of BLOSUM similarity scores in the context of antibody binding see Ucar & Sormanni (2025).

5. Experiments

We provide an open-source implementation of LOCK-GP at https://github.com/generatebio/lock_gp.

5.1. Local Landscape Data

We curate 21 datasets with reference structures for benchmarking that span a range of property types (thermostability, binding affinity, fluorescence, capsid viability, etc.), including 9 from ProteinGym (Notin et al., 2023a). To enable comprehensive benchmarking we choose datasets with ≥ 1800 data points, ≥ 10 variable positions, and an abundance of higher-order mutants. See Tables 8-9 for additional information on each dataset.¹⁴ We evaluate model performance in three different regimes: i) i.i.d. cross-validation; ii) Hamming-distance-based extrapolation; and iii) an ‘unseen mutations’ regime, in which all test sequences have at least one mutation not present in the corresponding training dataset. For details see Sec. A.14.

5.2. Ablation Study

We conduct a systematic ablation study using the data from Sec. 5.1. We highlight a few conclusions: i) dependence on

¹³See Sec. A.13 for a more detailed discussion of Kermut.

¹⁴For all datasets the maximum Hamming distance to the reference sequence is ≥ 6 ; for 19/21 datasets it is ≥ 10 . Structures for 10/21 datasets are experimental; the rest are predicted.

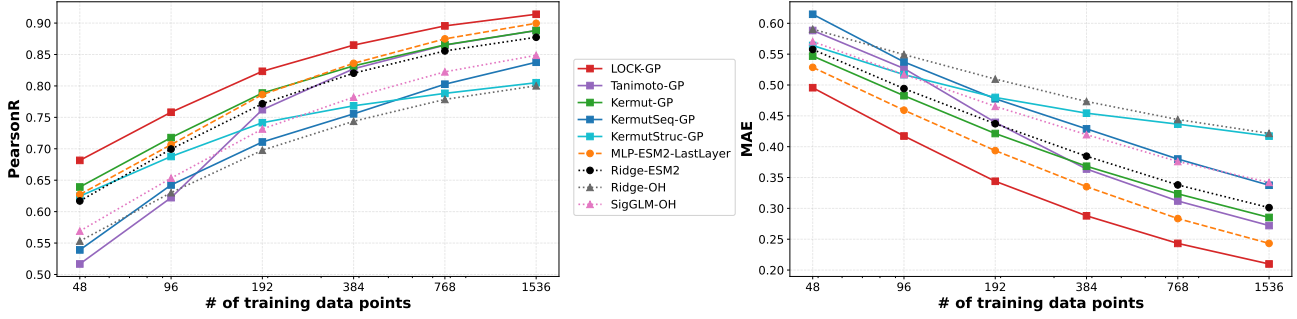


Figure 2. Predictive performance as a function of the number of training data points in the cross-validation setting. We depict Pearson R (left) and mean absolute error (right); metrics are averaged across 21 datasets. See Sec. 5.3 for discussion and Fig. 14 for Spearman R and RMSE.

Table 1. Model performance metrics for three different evaluation regimes, with the number of training data points ranging from 48 to 1536. Metrics are averaged across 21 datasets. For each column the best performing metric value is marked in bold. See also Table 23.

	Cross-validation (48 data points)			Cross-validation (1536 data points)			Unseen mutations (96 data points)			Extrapolation (128 data points)			Extrapolation (512 data points)		
	Spearman	Pearson	MAE	Spearman	Pearson	MAE	Spearman	Pearson	MAE	Spearman	Pearson	MAE	Spearman	Pearson	MAE
LOCK-GP	0.655	0.682	0.496	0.867	0.914	0.210	0.610	0.622	0.591	0.669	0.711	0.592	0.759	0.807	0.439
Tanimoto-GP	0.520	0.517	0.588	0.846	0.888	0.272	0.555	0.560	0.675	0.632	0.654	0.762	0.739	0.769	0.574
Kermut-GP	0.638	0.639	0.547	0.850	0.888	0.285	0.629	0.632	0.617	0.639	0.654	0.845	0.750	0.767	0.670
KermutSeq-GP	0.541	0.539	0.615	0.794	0.838	0.338	0.532	0.537	0.681	0.518	0.505	0.836	0.605	0.614	0.714
KermutStruc-GP	0.622	0.625	0.564	0.809	0.805	0.417	0.614	0.613	0.634	0.624	0.641	0.847	0.706	0.713	0.847
MLP-ESM2-LastLayer	0.607	0.627	0.529	0.851	0.900	0.243	0.558	0.573	0.626	0.628	0.619	0.771	0.738	0.742	0.618
Ridge-ESM2	0.606	0.617	0.558	0.837	0.877	0.301	0.476	0.490	1.628	0.629	0.653	0.781	0.719	0.740	0.685
Ridge-OH	0.547	0.553	0.590	0.802	0.800	0.422	0.514	0.519	0.682	0.589	0.599	0.939	0.694	0.698	0.877
SigGLM-OH	0.550	0.569	0.570	0.804	0.849	0.342	0.515	0.535	0.661	0.587	0.610	0.800	0.697	0.724	0.669

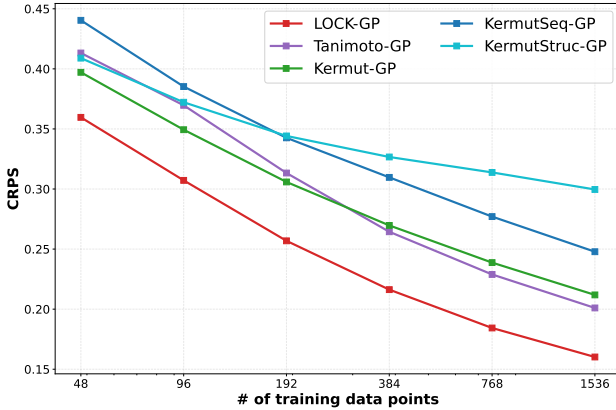


Figure 3. Predictive performance of GP models as a function of the number of training data points in the cross-validation setting. We depict the continuous ranked probability score (CRPS; Gneiting & Raftery (2007)), an MAE-like proper scoring rule that evaluates both accuracy and calibration of predictive distributions. Metrics are averaged across 21 datasets.

the particular BLOSUM substitution matrix used is weak; ii) the non-linear kernel k_{nl}^{LOCK} in Eqn. 12 consistently outperforms the RBF kernel in Eqn. 7, underscoring the value of incorporating substitution matrices; iii) the exponent priors in Sec. 3.5 are important for regularization; iv) using ‘local’ residuewise exponents α_ℓ improves fit noticeably; and v) assuming local linearity improves uncertainty quantification. Taken together our analysis shows that the thoughtful inclusion of BLOSUM matrices is the single most critical

Table 2. Overview of models benchmarked in Sec. 5.3. See Sec. A.16.1 for details on each model and Table 21 for performance metrics of additional baseline models.

Model Name	Model Class	Prior Information
LOCK-GP	Gaussian process	BLOSUM
Tanimoto-GP	Gaussian process	BLOSUM
Kermut-GP	Gaussian process	ESM-2 + ProteinMPNN
KermutSeq-GP	Gaussian process	ESM-2
KermutStruc-GP	Gaussian process	ESM-2 + ProteinMPNN
MLP-ESM2-LastLayer	Neural network	ESM-2
Ridge-ESM2	Neural network	ESM-2
Ridge-OH	Linear	—
SigGLM-OH	Linear + global non-linearity	—

ingredient in the good performance of LOCK. For detailed results and additional analysis see Sec. A.5.

5.3. Local Learning

We use the data from Sec. 5.1 to benchmark LOCK-GP against 8 different baselines; these include 4 GPs with the following kernels: Tanimoto-GP) a Tanimoto kernel that uses BLOSUM62-derived embeddings (Gessner et al., 2024); KermutSeq-GP) an isotropic RBF kernel that uses mean-pooled ESM-2 features; KermutStruc-GP) a structure-conditioned kernel that exploits ProteinMPNN inverse folding logits (Dauparas et al., 2022); and Kermut-GP) an additive combination of the previous two kernels (Groth et al.,

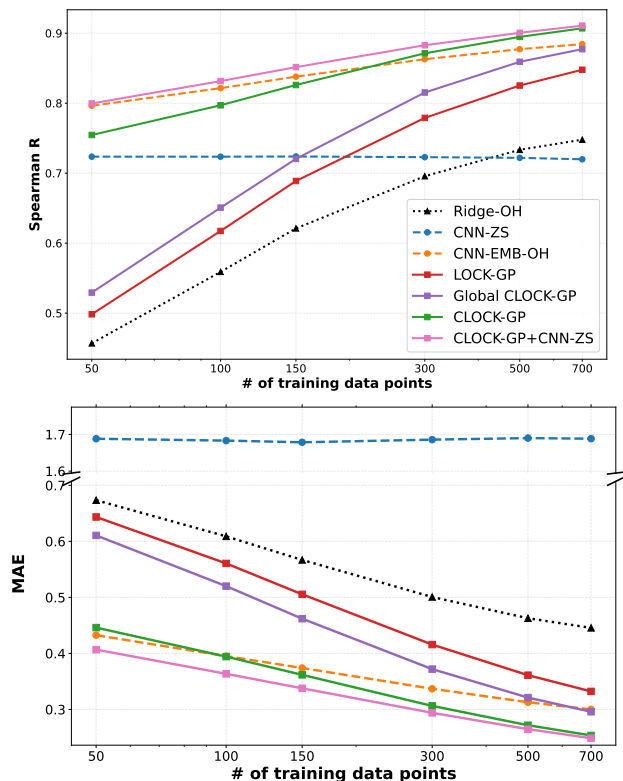


Figure 4. Predictive performance as a function of the number of (landscape-local) training data points on the multi-landscape thermostability data described in Sec. 5.4. We depict both Spearman R (top) and mean absolute error (bottom); metrics are averaged across 100 train/test splits in 50 held-out landscapes. The structure-conditioned CLOCK-GP augmented with an additional CNN-based kernel performs best across the board. Note the axis break for the MAE figure: the absolute scale of the zero-shot predictor is very poorly calibrated. See Fig. 15 for additional metrics.

2024).¹⁵ We also include 4 non-GP baselines: Ridge-OH) ridge regression on one-hot features; Ridge-ESM2) ridge regression on ESM-2 features; MLP-ESM2-LastLayer) a single-layer neural network with ESM-2 features in which the last layer of the PLM is fine-tuned; and SigGLM-OH) generalized linear regression on one-hot features with a sigmoidal link function (Ding et al., 2024). See Table 2 for a high-level model taxonomy and Sec. A.16.1 for details on each baseline model.

We find that LOCK-GP exhibits the strongest cross-regime performance, with Kermut-GP performing comparably on the unseen mutations benchmark. See Fig. 2 and Table 1 for the results and Table 21 for twelve additional baselines, including zero-shot methods and ConFit (Zhao et al., 2024). In the cross-validation setting for $N = 192$ training data points, for example, LOCK-GP has the best Pearson R (resp., MAE) for 16/21 (resp., 17/21) datasets; see Ta-

¹⁵In addition all three Kermut variants use an ESM-2-derived zero-shot prior mean.

bles 11-19 for these and other landscape-level results. This strong performance is striking, since LOCK-GP makes do with the 210 parameters of a BLOSUM50-derived correlation matrix,¹⁶ while Kermut-GP relies on ESM-2 (650M parameters), ProteinMPNN (1.7M parameters) and the reference structure. The low MAEs of LOCK-GP are matched by low CRPS—see Fig. 3—reflecting the high quality of LOCK-GP uncertainty estimates. We find that Ridge-ESM2 and MLP-ESM2-LastLayer perform rather poorly on the unseen mutations benchmark—indeed Ridge-ESM2 is outperformed by the much simpler Tanimoto-GP—highlighting the potential fragility of high-dimensional embeddings in OOD settings.¹⁷ Similarly in the extrapolation regime with $N = 128$, LOCK-GP (resp., Tanimoto-GP) outperforms Kermut-GP on 16/21 (resp., 11/21) datasets w.r.t. Pearson R, raising questions about the added value of using two foundation models. Moreover, in the extrapolation regime with $N = 512$ LOCK-GP exhibits a mean MAE that is 22% lower than the next best model (Tanimoto-GP). We also note that all three Kermut variants exhibit poor log likelihoods on some datasets, likely due to poorly regularized hyperparameters; see Table 22 and Sec. A.13 for additional discussion. By contrast in the extrapolation regime with $N = 128$ LOCK-GP exhibits the best negative log likelihood (NLL) on 12/21 datasets, whereas the corresponding numbers for Tanimoto-GP and Kermut-GP are 4/21 and 3/21, respectively; see Table 20. Finally we note that LOCK-GP can be much faster ($\sim 2 - 140\times$) at training and inference time than methods like MLP-ESM2-LastLayer and Kermut-GP that rely on foundation models; see Sec. A.15 for details.

5.4. Multi-task Learning

We demonstrate the utility of the CLOCK-GP introduced in Sec. 3.6 by evaluating a variety of multi-task models on 371 thermostability landscapes from (Tsuboyama et al., 2023). Each landscape has ~ 1000 sequences, most of which are double mutants; the typical sequence length is $L \sim 50$. Each landscape has an AlphaFold2 predicted structure (Jumper et al., 2021). We use 280 landscapes for training, while holding out 41 + 50 landscapes for validation and testing.

As our main baseline we train a multi-scale CNN with 2.53M parameters that utilizes 128-dimensional positional sequence-and-structure embeddings provided by Chroma (Ingraham et al., 2023). We use the CNN both as a zero-shot predictor (CNN-ZS) as well as a supervised learner (CNN-EMB-OH) in which we combine features from the CNN with one-hot sequences and feed the concatenated features into a ridge regression head. For CLOCK-GP we use 128-

¹⁶Note that Tanimoto-GP also leverages BLOSUM matrices, but via an eigendecomposition in log-odds space, which is evidently less well-suited to typical protein landscapes.

¹⁷We find similar behavior for ConFit; see Table 21.

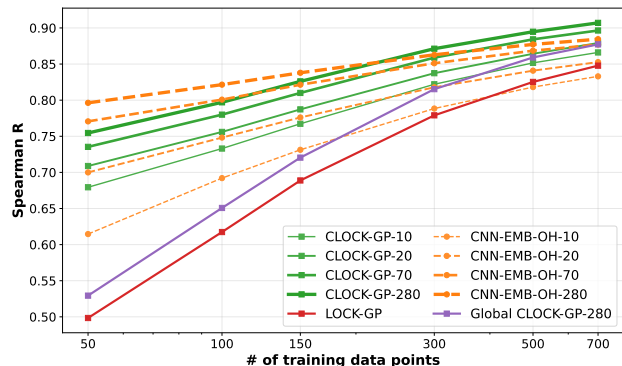


Figure 5. We depict how multi-task model performance changes as we vary the number of training landscapes from 10 to 280. Spearman R is averaged across 100 train/test splits in 50 held-out landscapes. CLOCK-GP performs best in the low landscape regime: e.g. CLOCK-GP-10, which is trained on 10 landscapes, roughly matches the performance of CNN-EMB-OH-20. See Sec. 5.4 for details and Fig. 20-27 for more figures.

dimensional positional structure embeddings from Chroma. In addition to CLOCK-GP we consider two GP baselines: i) LOCK-GP with a BLOSUM50 substitution matrix; and ii) Global CLOCK-GP in which we fit a global correlation matrix that is deployed at all positions.¹⁸ Finally we include ridge regression on one-hot sequences (Ridge-OH) as well as CLOCK-GP augmented with an additional RBF kernel that leverages the CNN zero-shot predictor (CLOCK-GP+CNN-ZS). For additional details see Sec. A.16.

The value of a structure-conditioned function prior is clearly demonstrated by comparing CLOCK-GP to Ridge-OH: the former model trained on $N = 50$ (landscape-local) data points outperforms the latter trained on $N = 700$ data points; see Fig. 4. Likewise the Spearman R of CLOCK-GP at $N = 50$ is 0.755, while it is only 0.498 for LOCK-GP. Setting aside the hybrid CLOCK-GP+CNN-ZS, the best performing model in the scarce data regime is CNN-EMB-OH, while CLOCK-GP is the strongest performer once the number of training data points increases ($N \gtrsim 200$). Not surprisingly, we see the best performance from CLOCK-GP+CNN-ZS, which combines the flexibility of neural networks with the carefully constructed function prior of CLOCK-GP. Notably at $N = 700$ Global CLOCK-GP, which does not use structure embeddings, matches the performance of CNN-EMB-OH, which does. As expected LOCK-GP is quite effective, although it is outperformed by Global CLOCK-GP which learns a global correlation matrix adapted to thermostability. See Table 24 for additional quantitative results.

Landscape Subsampling Above we trained models on 280 landscapes, whereas in many applications we may have

¹⁸That is we remove positional indices from Eqn. 14 and learn a single $\mathbf{z} \in \mathbb{R}^{A,A-1}$ that does *not* depend on $\mathbf{h}_{1:L}$.

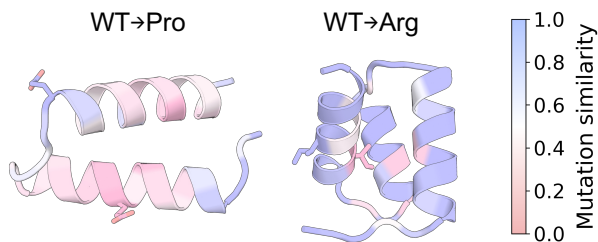


Figure 6. Structure-conditioned amino acid correlations learned by CLOCK on representative structures from Tsuboyama et al. (2023). Left (PDB ID: 2JVD): correlation between the wild-type amino acid and proline at each position. Right (PDB ID: 2JVD): correlation between the wild-type amino acid and arginine. Each structure highlights a pair of sites (namely those with sidechains pictured) with the *same* amino-acid identity but sharply different kernel correlation to the reference amino acid, illustrating how amino acids with identical BLOSUM-derived correlation can have distinct structure-conditioned correlations. Blue (resp., red) indicates substitutions judged (dis)similar to the reference residue.

significantly less data. This motivates a simple question: can we make do with (far) fewer landscapes? To investigate this systematically we train models on $n_{\text{landscapes}} \in \{10, 20, 35, 70, 140\}$. For the results see Fig. 5. Encouragingly, we find that CLOCK-GP performs best in the low-landscape regime. This is arguably expected, since GPs can be very data efficient. Put differently, GPs can still provide good predictive performance with a suboptimal kernel, and a modest number of landscapes may be sufficient to learn a kernel that is “good enough.” Indeed with just 10 landscapes¹⁹ CLOCK-GP outperforms Global CLOCK-GP trained on all 280 landscapes, highlighting the value of structure-conditioned correlation matrices (even imperfect ones). We highlight that LOCK-GP—a landscape-local model that uses a pre-canned substitution matrix—outperforms CNN-EMB-OH trained on 10 landscapes for $N \gtrsim 500$. By contrast CLOCK-GP trained on 10 landscapes outperforms LOCK-GP across the board. Notably all multi-task models improve as the number of training landscapes increases (although the improvements for Global CLOCK-GP are modest, see Fig. 22).

Interpretation As shown in Fig. 6, the correlations learned by CLOCK-GP reveal interpretable structural patterns: protein 2JVD illustrates that CLOCK captures a well-known preference to substitute proline at the N-terminal region of alpha helices (Richardson & Richardson, 1988), while protein 2JWD shows that substitutions to arginine are preferred on the protein surface rather than in the core. See Sec. A.7 for additional analysis. For visual representations of the correlation matrices learned by (Global) CLOCK-GP see Fig. 17-19, which highlight the special role played by proline in thermostability. Fig. 17-19 also reveal that the

¹⁹Since $L \sim 50$ this corresponds to ~ 500 local contexts $\mathbf{h}_\ell$.

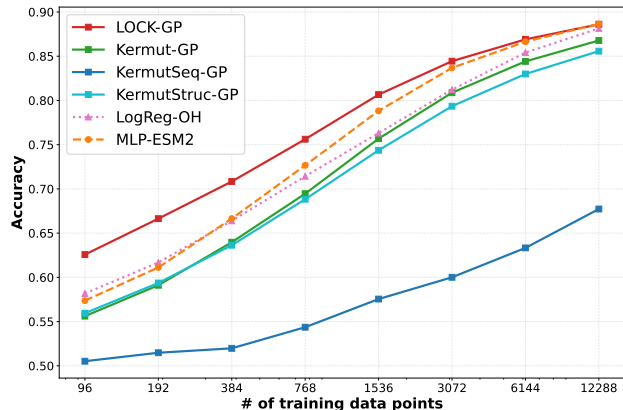


Figure 7. We report binary classification accuracy for models trained on (quantized) fluorescence readouts from Gonzalez Somer-meyer et al. (2022) as a function of the number of training data points. Accuracy is averaged across three replicates. LogReg-OH is logistic regression on one-hot features and is the direct analog of Ridge-OH in Sec. 5.3.

thermostability data contain sequences with gap tokens, confirming the ability of (C)LOCK-GPs to handle landscapes with variable length sequences.

5.5. Thompson Sampling

We demonstrate the utility of LOCK-GP uncertainty estimates by using Thompson sampling (TS) to trade off between exploration and exploitation in an optimization-based protein design problem with parallel candidate selection. See Sec. A.6 and Fig. 8 for the results, which illustrate how TS offers a natural lever for modulating sequence diversity. We note that Thompson sampling of GP surrogate models is by no means the only way to leverage uncertainty estimates in the context of protein design, and we refer the reader to the growing literature on this topic for alternative approaches (Stanton et al., 2022; Hu et al., 2023; Hawkins-Hooker et al., 2023; Khan et al., 2023; Rapp et al., 2024).

5.6. Binary Classification

We demonstrate that LOCK-GP can readily be applied to landscapes with binary labels. In our benchmark we evaluate most of the model classes considered in Sec. 5.3. See Fig. 7 for the results and Sec. A.11 and A.16.3 for additional details.

6. Discussion

We have seen that Gaussian processes are a powerful tool for modeling protein property landscapes, at least once sufficient care is taken to build a fit-to-purpose kernel. Notably, LOCK-GP frequently outperforms more complex methods that rely on foundation models with millions of parameters.

Moreover, the technology introduced in Sec. 3 is of wide application, as we have illustrated by, for example, training hybrid GPs that utilize neural zero-shot predictors as well as structure-conditioned kernels that leverage foundation model embeddings. A limitation of our kernels is their restriction to settings in which individual landscapes are either fixed-length or can be aligned to a common length. Extending our approach to relax this requirement is a natural direction for future work, as it would make (C)LOCK kernels applicable in a wider range of settings.

In Sec. 5.4 we introduced multi-task GPs that employ zero-shot structure-conditioned kernels. While these models do not inherit the extreme parameter efficiency of LOCK-GP—since they leverage the rich features provided by foundation models to enable cross-landscape learning—they can decisively outperform landscape-local models with as few as 10 landscapes. In some applications, however, even 10 landscapes may be unavailable. This raises the intriguing question of whether CLOCK kernels can be pre-trained on a given set of property landscapes and then fine-tuned on partially related landscapes—for example, transferring learned thermostability kernels to binding affinity landscapes. Since $\mathbf{W}$ in Eqn. 15 is a structured tensor of shape $A \times A - 1 \times D_{\text{emb}}$ —with $\sim 54\text{k}$ parameters in our experiments—it seems plausible that low-rank updates may be sufficient to adapt $\mathbf{W}$ to a new set of landscapes.²⁰ We provide a proof-of-concept demonstration of this general approach in Sec. A.9 by pre-training $\mathbf{W}$ on thermostability data and fine-tuning it on AAV capsid data (see Fig. 10-11). This initial positive result is encouraging, and we believe this broader line of inquiry is a fruitful direction for future work.

Acknowledgements

We thank the anonymous reviewers for their constructive feedback and suggestions.

Impact Statement

This work aims to improve data-efficient prediction of protein properties, with potential benefits for therapeutic discovery and industrial enzyme engineering. Like other methods that accelerate protein design, it could potentially be misused to facilitate harmful applications. Responsible use, appropriate oversight, and adherence to relevant regulations are therefore important.

²⁰Alternatively, one could instead fine-tune the foundation model that generates the embeddings $\mathbf{h}_{1:L}$.

References

- Alley, E. C., Khimulya, G., Biswas, S., AlQuraishi, M., and Church, G. M. Unified rational protein engineering with sequence-only deep representation learning. *Nature Methods*, 16(12):1315–1322, 2019. doi: 10.1038/s41592-019-0598-1. URL <https://www.nature.com/articles/s41592-019-0598-1>.
- Altschul, S. F. Amino acid substitution matrices from an information theoretic perspective. *Journal of molecular biology*, 219(3):555–565, 1991.
- Amin, A. N., Marks, D. S., and Weinstein, E. N. Biological sequence kernels with guaranteed flexibility. *Journal of Machine Learning Research*, 26(216):1–63, 2025.
- Balandat, M., Karrer, B., Jiang, D. R., Daulton, S., Letham, B., Wilson, A. G., and Bakshy, E. BoTorch: A framework for efficient monte-carlo bayesian optimization. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems 33: NeurIPS 2020 (virtual)*, 2020.
- Benjamins, C., Surana, S., Bent, O., Lindauer, M., and Duckworth, P. Bayesian optimisation for protein sequence design: Gaussian processes with zero-shot protein language model prior mean. In *Machine Learning in Structural Biology Workshop at NeurIPS*, volume 2024, pp. 4, 2024.
- Berg, C., Christensen, J. P. R., and Ressel, P. *Harmonic Analysis on Semigroups: Theory of Positive Definite and Related Functions*. Springer-Verlag, 1984.
- Bikias, T., Stamkopoulos, E., and Reddy, S. T. Plmfit: Benchmarking transfer learning with protein language models for protein engineering. *Briefings in Bioinformatics*, 26(4):bbaf381, 2025. doi: 10.1093/bib/bbaf381. URL <https://academic.oup.com/bib/article/26/4/bbaf381/8219059>.
- Brandes, N., Ofer, D., Peleg, Y., Rappoport, N., and Linial, M. ProteinBERT: a universal deep-learning model of protein sequence and function. *Bioinformatics*, 38(8):2102–2110, 2022. doi: 10.1093/bioinformatics/btac020. URL <https://academic.oup.com/bioinformatics/article/38/8/2102/6502274>.
- Cazelles, E. and Tobar, F. Efficient gaussian process learning via subspace projections. In *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, 2026.
- Chen, Y., Hu, R., Li, K., Zhang, Y., Fu, L., Zhang, J., and Si, T. Deep mutational scanning of an oxygen-independent fluorescent protein creilov for comprehensive profiling of mutational and epistatic effects. *ACS Synthetic Biology*, 12(5):1461–1473, April 2023. ISSN 2161-5063. doi: 10.1021/acssynbio.2c00662. URL <http://dx.doi.org/10.1021/acssynbio.2c00662>.
- Chungyoun, M., Ruffolo, J., and Gray, J. Flab: Benchmarking deep learning methods for antibody fitness prediction. *BioRxiv*, pp. 2024–01, 2024.
- Dauparas, J., Anishchenko, I., Bennett, N., Bai, H., Ragotte, R. J., Milles, L. F., Wicky, B. I., Courbet, A., de Haas, R. J., Bethel, N., et al. Robust deep learning-based protein sequence design using ProteinMPNN. *Science*, 378(6615):49–56, 2022.
- Dayhoff, M. O. A model of evolutionary change in proteins. *Atlas of protein sequence and structure*, 5:89–99, 1972.
- Ding, D., Shaw, A. Y., Sinai, S., Rollins, N., Prywes, N., Savage, D. F., Laub, M. T., and Marks, D. S. Protein design using structure-based residue preferences. *Nature Communications*, 15(1), February 2024. ISSN 2041-1723. doi: 10.1038/s41467-024-45621-4. URL <http://dx.doi.org/10.1038/s41467-024-45621-4>.
- Elnaggar, A., Heinzinger, M., Dallago, C., Rehawi, G., Wang, Y., Jones, L., Gibbs, T., Feher, T., Angerer, C., Steinegger, M., Bhowmik, D., and Rost, B. Prottrans: Toward understanding the language of life through self-supervised learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10):7112–7127, 2022. doi: 10.1109/TPAMI.2021.3095381.
- Falcon, W. and The PyTorch Lightning team. PyTorch lightning. Zenodo, March 2019. URL <https://doi.org/10.5281/zenodo.3828935>. Version 1.4. Project site: <https://www.pytorchlightning.ai>.
- FitzGerald, C. H. and Horn, R. A. On fractional hadamard powers of positive definite matrices. *Journal of Mathematical Analysis and Applications*, 61(3):633–642, 1977. doi: 10.1016/0022-247X(77)90167-6.
- Fox, R., Roy, A., Govindarajan, S., Minshull, J., Gustafsson, C., Jones, J. T., and Emig, R. Optimizing the search algorithm for protein engineering by directed evolution. *Protein Engineering, Design & Selection*, 16(8):589–597, 2003. doi: 10.1093/protein/gzg077. URL <https://academic.oup.com/peds/article/16/8/589/1501069>.
- Frazer, J., Notin, P., Dias, M., Gomez, A., Min, J. K., Brock, K. P., Gal, Y., and Marks, D. S. Disease variant prediction with deep generative models of evolutionary data. *Nature*, 599(7883):91–95, 2021. doi: 10.1038/s41586-021-04043-8. URL <https://www.nature.com/articles/s41586-021-04043-8>.

- Gardner, J. R., Pleiss, G., Weinberger, K. Q., Bindel, D., and Wilson, A. G. GPyTorch: Blackbox matrix-matrix gaussian process inference with GPU acceleration. In Bengio, S., Wallach, H. M., Larochelle, H., Grauman, K., Cesa-Bianchi, N., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 31: NeurIPS 2018, Montréal, Canada*, pp. 7587–7597, 2018.
- Gessner, A., Ober, S. W., Vickery, O., Oglič, D., and Uçar, T. Active learning for affinity prediction of antibodies. In *Proceedings of the Machine Learning in Structural Biology (MLSB) Workshop at NeurIPS 2024*, 2024. URL https://www.mlsb.io/papers_2024/Active_Learning_for_Affinity_Prediction_of_Antibodies.pdf.
- Gneiting, T. and Raftery, A. E. Strictly proper scoring rules, prediction, and estimation. *Journal of the American statistical Association*, 102(477):359–378, 2007.
- Gonzalez Somermeyer, L., Fleiss, A., Mishin, A. S., Bozhanova, N. G., Igolkina, A. A., Meiler, J., Alaball Pujol, M.-E., Putintseva, E. V., Sarkisyan, K. S., and Kon-drashov, F. A. Heterogeneity of the GFP fitness landscape and data-driven protein design. *eLife*, 11, May 2022. ISSN 2050-084X. doi: 10.7554/elife.75842.
- Groth, P. M., Kern, M. H., Olsen, L., Salomon, J., and Boomsma, W. Kermut: Composite kernel regression for protein variant effects. In *Advances in Neural Information Processing Systems*, 2024. Spotlight.
- Hawkins-Hooker, A., Duckworth, P., and Bent, O. Preferential bayesian optimisation for protein design with ranking-based fitness predictors. In *Machine Learning in Structural Biology Workshop at NeurIPS*, 2023.
- Hawkins-Hooker, A., Surana, S., Simons, J., Kmec, J., Bent, O., and Duckworth, P. Likelihood-based fine-tuning of protein language models for few-shot fitness prediction and design. *bioRxiv*, pp. 2024–05, 2024.
- Henikoff, S. and Henikoff, J. G. Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences*, 89(22):10915–10919, 1992.
- Hensman, J., Fusi, N., and Lawrence, N. D. Gaussian processes for big data. In *Proceedings of the Twenty-Ninth Conference on Uncertainty in Artificial Intelligence (UAI 2013)*, 2013.
- Hsu, C., Nisonoff, H., Fannjiang, C., and Listgarten, J. Learning protein fitness models from evolutionary and assay-labeled data. *Nature biotechnology*, 40(7):1114–1122, 2022.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- Hu, R., Fu, L., Chen, Y., Chen, J., Qiao, Y., and Si, T. Protein engineering via bayesian optimization-guided evolutionary algorithm and robotic experiments. *Briefings in Bioinformatics*, 24(1):bbac570, 2023.
- Ingraham, J. B., Baranov, M., Costello, Z., Barber, K. W., Wang, W., Ismail, A., Frappier, V., Lord, D. M., Ng-Thow-Hing, C., Van Vlack, E. R., et al. Illuminating protein space with a programmable generative model. *Nature*, 623(7989):1070–1078, 2023.
- Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Židek, A., Potapenko, A., Bridgland, A., Meyer, C., Kohli, S. A. A., Ballard, A. J., Cowie, A., Romera-Paredes, B., Nikolov, S., Jain, R., Adler, J., Back, T., Petersen, S., Reiman, D., Clancy, E., Zielinski, M., Steinegger, M., Pacholska, M., Berghammer, T., Bodenstein, S., Silver, D., Vinyals, O., Senior, A. W., Kavukcuoglu, K., Kohli, P., and Hassabis, D. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021. doi: 10.1038/s41586-021-03819-2.
- Khan, A., Cowen-Rivers, A. I., Grosnit, A., Deik, D.-G.-X., Robert, P. A., Greiff, V., Smorodina, E., Rawat, P., Akbar, R., Dreczkowski, K., et al. Toward real-world automated antibody design with combinatorial bayesian optimization. *Cell Reports Methods*, 3(1), 2023.
- Koshi, J. M. and Goldstein, R. A. Context-dependent optimal substitution matrices. *Protein Engineering, Design and Selection*, 8(7):641–645, 1995.
- Kunzmann, P., Müller, T. D., Greil, M., Krumbach, J. H., Anter, J. M., Bauer, D., Islam, F., and Hamacher, K. Biotite: new tools for a versatile python bioinformatics library. *BMC Bioinformatics*, 24:1–19, 2023. doi: 10.1186/s12859-023-05345-6.
- Li, F.-Z., Amini, A. P., Yue, Y., Yang, K. K., and Lu, A. X. Feature reuse and scaling: Understanding transfer learning with protein language models. In Salakhutdinov, R., Kolter, Z., Heller, K., Weller, A., Oliver, N., Scarlett, J., and Berkenkamp, F. (eds.), *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 27351–27375. PMLR, 21–27 Jul 2024. URL <https://proceedings.mlr.press/v235/li24a.html>.
- Lin, Z., Akin, H., Rao, R., Hie, B., Zhu, Z., Lu, W., Smetanin, N., Verkuil, R., Kabeli, O., Shmueli, Y.,

- dos Santos Costa, A., Fazel-Zarandi, M., Sercu, T., Candido, S., and Rives, A. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. doi: 10.1126/science.ade2574.
- Meier, J., Rao, R., Verkuil, R., Liu, J., Sercu, T., and Rives, A. Language models enable zero-shot prediction of the effects of mutations on protein function. In *Advances in Neural Information Processing Systems*, 2021.
- Moore, C. J., Chua, A. J., Berry, C. P., and Gair, J. R. Fast methods for training gaussian processes on large datasets. *Royal Society open science*, 3(5):160125, 2016.
- Moulana, A., Dupic, T., Phillips, A. M., Chang, J., Roffler, A. A., Greaney, A. J., Starr, T. N., Bloom, J. D., and Desai, M. M. The landscape of antibody binding affinity in SARS-CoV-2 omicron ba.1 evolution. *eLife*, 12, February 2023. ISSN 2050-084X. doi: 10.7554/elife.83442. URL <http://dx.doi.org/10.7554/eLife.83442>.
- Notin, P., Kollasch, A., Ritter, D., Van Niekerk, L., Paul, S., Spinner, H., Rollins, N., Shaw, A., Orenbuch, R., Weitzman, R., et al. Proteingym: Large-scale benchmarks for protein fitness prediction and design. *Advances in Neural Information Processing Systems*, 36:64331–64379, 2023a.
- Notin, P., Weitzman, R., Marks, D., and Gal, Y. ProteinNPT: Improving protein property prediction and design with non-parametric transformers. *Advances in Neural Information Processing Systems*, 36:33529–33563, 2023b.
- Ober, S. W., Rasmussen, C. E., and van der Wilk, M. The promises and pitfalls of deep kernel learning. In *Uncertainty in Artificial Intelligence*, pp. 1206–1216. PMLR, 2021.
- Otwinowski, J., McCandlish, D. M., and Plotkin, J. B. Inferring the shape of global epistasis. *Proceedings of the National Academy of Sciences*, 115(32):E7550–E7558, 2018.
- Overington, J., Donnelly, D., Johnson, M. S., Šali, A., and Blundell, T. L. Environment-specific amino acid substitution tables: tertiary templates and prediction of protein folds. *Protein Science*, 1(2):216–226, 1992.
- Pacesa, M., Nickel, L., Schellhaas, C., Schmidt, J., Pyatova, E., Kissling, L., Barendse, P., Choudhury, J., Kapoor, S., Alcaraz-Serna, A., et al. One-shot design of functional protein binders with BindCraft. *Nature*, pp. 1–10, 2025.
- Park, J.-S. and Baek, J. Efficient computation of maximum likelihood estimators in a spatial linear model with power exponential covariogram. *Computers & Geosciences*, 27(1):1–7, 2001.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E. Z., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. PyTorch: An imperative style, high-performance deep learning library. In Wallach, H. M., Larochelle, H., Beygelzimer, A., d’Alché-Buc, F., Fox, E. B., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 32: NeurIPS 2019, Vancouver, BC, Canada*, pp. 8024–8035, 2019.
- Phillips, A. M., Maurer, D. P., Brooks, C., Dupic, T., Schmidt, A. G., and Desai, M. M. Hierarchical sequence-affinity landscapes shape the evolution of breadth in an anti-influenza receptor binding site antibody. *eLife*, 12, January 2023. ISSN 2050-084X. doi: 10.7554/elife.83628. URL <http://dx.doi.org/10.7554/eLife.83628>.
- Pokusaeva, V. O., Usmanova, D. R., Putintseva, E. V., Espinar, L., Sarkisyan, K. S., Mishin, A. S., Bogatyreva, N. S., Ivankov, D. N., Akopyan, A. V., Avvakumov, S. Y., Povolotskaya, I. S., Fillion, G. J., Carey, L. B., and Kondrashov, F. A. An experimental assay of the interactions of amino acids from orthologous sequences shaping a complex fitness landscape. *PLOS Genetics*, 15(4):e1008079, April 2019. ISSN 1553-7404. doi: 10.1371/journal.pgen.1008079. URL <http://dx.doi.org/10.1371/journal.pgen.1008079>.
- Rao, R., Bhattacharya, N., Thomas, N., Duan, Y., Chen, X., Canny, J., Abbeel, P., and Song, Y. S. Evaluating protein transfer learning with TAPE. *Advances in Neural Information Processing Systems*, 32, 2019.
- Rapp, J. T., Bremer, B. J., and Romero, P. A. Self-driving laboratories to autonomously navigate the protein fitness landscape. *Nature chemical engineering*, 1(1):97–107, 2024.
- Rasmussen, C. E. Gaussian processes in machine learning. In *Summer School on Machine Learning*, pp. 63–71. Springer, 2003.
- Richardson, J. S. and Richardson, D. C. Amino acid preferences for specific locations at the ends of α helices. *Science*, 240(4859):1648–1652, 1988.
- Riesselman, A. J., Ingraham, J. B., and Marks, D. S. Deep generative models of genetic variation capture the effects of mutations. *Nature Methods*, 15(10):816–822, 2018. doi: 10.1038/s41592-018-0138-4. URL <https://www.nature.com/articles/s41592-018-0138-4>.
- Rives, A., Meier, J., Sercu, T., Goyal, S., Lin, Z., Liu, J., Guo, D., Ott, M., Zitnick, C. L., Ma, J., and Ferguson, R. Biological structure and function emerge from

- scaling unsupervised learning to 250 million protein sequences. *Proceedings of the National Academy of Sciences*, 118(15):e2016239118, 2021. doi: 10.1073/pnas.2016239118. URL <https://www.pnas.org/doi/pdf/10.1073/pnas.2016239118>.
- Romero, P. A., Krause, A., and Arnold, F. H. Navigating the protein fitness landscape with gaussian processes. *Proceedings of the National Academy of Sciences*, 110(3): E193–E201, 2013.
- Sarkisyan, K. S., Bolotin, D. A., Meer, M. V., Usmanova, D. R., Mishin, A. S., Sharonov, G. V., Ivankov, D. N., Bozhanova, N. G., Baranov, M. S., Soylemez, O., Bogatyreva, N. S., Vlasov, P. K., Egorov, E. S., Logacheva, M. D., Kondrashov, A. S., Chudakov, D. M., Putintseva, E. V., Mamedov, I. Z., Tawfik, D. S., Lukyanov, K. A., and Kondrashov, F. A. Local fitness landscape of the green fluorescent protein. *Nature*, 533(7603):397–401, May 2016. ISSN 1476-4687. doi: 10.1038/nature17995.
- Schmirler, R., Heinzinger, M., and Rost, B. Fine-tuning protein language models boosts predictions across diverse tasks. *Nature Communications*, 15(1):7407, 2024.
- Schoenberg, I. J. Metric spaces and positive definite functions. *Transactions of the American Mathematical Society*, 44(3):522–536, 1938.
- Shanker, V. R., Bruun, T. U. J., Hie, B. L., and Kim, P. S. Unsupervised evolution of protein and antibody complexes with a structure-informed language model. *Science*, 385(6704):46–53, July 2024. ISSN 1095-9203. doi: 10.1126/science.adk8946. URL <http://dx.doi.org/10.1126/science.adk8946>.
- Sinai, S., Jain, N., Church, G. M., and Kelsic, E. D. Generative AAV capsid diversification by latent interpolation. *bioRxiv*, April 2021. URL <https://doi.org/10.1101/2021.04.16.440236>. preprint.
- Sledzieski, S., Kshirsagar, M., Baek, M., Dodhia, R., Lavista Ferres, J., and Berger, B. Democratizing protein language models with parameter-efficient fine-tuning. *Proceedings of the National Academy of Sciences*, 121(26): e2405840121, 2024.
- Staller, M. V., Holehouse, A. S., Swain-Lenz, D., Das, R. K., Pappu, R. V., and Cohen, B. A. A high-throughput mutational scan of an intrinsically disordered acidic transcriptional activation domain. *Cell Systems*, 6(4):444–455.e6, April 2018. ISSN 2405-4712. doi: 10.1016/j.cels.2018.01.015. URL <http://dx.doi.org/10.1016/j.cels.2018.01.015>.
- Stanton, S., Maddox, W., Gruver, N., Maffettone, P., Delaney, E., Greenside, P., and Wilson, A. G. Accelerating bayesian optimization for biological sequence design with denoising autoencoders. In *International conference on machine learning*, pp. 20459–20478. PMLR, 2022.
- Su, J., Han, C., Zhou, Y., Shan, J., Zhou, X., and Yuan, F. SaProt: Protein language modeling with structure-aware vocabulary. In *International Conference on Learning Representations*, volume 2024, pp. 6987–7009, 2024.
- Tripp, A., Bacallado, S., Singh, S., and Hernández-Lobato, J. M. Tanimoto random features for scalable molecular machine learning. *Advances in Neural Information Processing Systems*, 36:33656–33686, 2023.
- Tsuboyama, K., Dauparas, J., Chen, J., Laine, E., Mohseni Behbahani, Y., Weinstein, J. J., Mangan, N. M., Ovchinnikov, S., and Rocklin, G. J. Mega-scale experimental analysis of protein folding stability in biology and design. *Nature*, 620(7973):434–444, July 2023. ISSN 1476-4687. doi: 10.1038/s41586-023-06328-6. URL <http://dx.doi.org/10.1038/s41586-023-06328-6>.
- Ucar, T. and Sormanni, P. BLOSUM is all you learn—generative antibody models reflect evolutionary priors. *bioRxiv*, pp. 2025–10, 2025.
- Vieira, L. C., Handojo, M. L., and Wilke, C. O. Medium-sized protein language models perform well at transfer learning on realistic datasets. *Scientific Reports*, 15(1): 21400, 2025. doi: 10.1038/s41598-025-05674-x.
- Wang, K., Pleiss, G., Gardner, J., Tyree, S., Weinberger, K. Q., and Wilson, A. G. Exact gaussian processes on a million data points. *Advances in neural information processing systems*, 32, 2019.
- Wells, J. A. Additivity of mutational effects in proteins. *Biochemistry*, 29(37):8509–8517, 1990.
- Wenzel, F., Galy-Fajou, T., Donner, C., Kloft, M., and Oppel, M. Efficient Gaussian process classification using Pólya-Gamma data augmentation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pp. 5417–5424, 2019.
- Wu, N. C., Dai, L., Olson, C. A., Lloyd-Smith, J. O., and Sun, R. Adaptation in protein fitness landscapes is facilitated by indirect paths. *elife*, 5:e16965, 2016.
- Yoshikawa, Y. and Iwata, T. Gaussian process regression with interpretable sample-wise feature weights. *IEEE Transactions on Neural Networks and Learning Systems*, 34:5789–5803, 2023. doi: 10.1109/TNNLS.2021.3131234. URL <https://ieeexplore.ieee.org/document/9646444>.
- Zhang, R., Liu, X., and Liu, Q. A langevin-like sampler for discrete distributions. In *International Conference on Machine Learning*, pp. 26375–26396. PMLR, 2022.

- Zhao, J., Zhang, C., and Luo, Y. Contrastive fitness learning: Reprogramming protein language models for low-n learning of protein fitness landscape. In *International Conference on Research in Computational Molecular Biology*, pp. 470–474. Springer, 2024.
- Zhou, J. and McCandlish, D. M. Minimum epistasis interpolation for sequence-function relationships. *Nature communications*, 11(1):1782, 2020.
- Zhou, J., Martí-Gómez, C., Petti, S., and McCandlish, D. M. Learning sequence-function relationships with scalable, interpretable gaussian processes. *eLife*, 14:RP108964, 2025. doi: 10.7554/eLife.108964.1. URL <https://doi.org/10.7554/eLife.108964.1>. Reviewed preprint.
- Zhou, Z., Zhang, L., Yu, Y., Wu, B., Li, M., Hong, L., and Tan, P. Enhancing efficiency of protein language models with minimal wet-lab data through few-shot learning. *Nature Communications*, 15(1):5566, 2024.

A. Appendix

Table of Contents

A.1 Infinitely Divisible Matrices	15
A.2 Local Linearity	17
A.3 Detailed LOCK Kernel Description	18
A.4 Epistasis and LOCK	19
A.5 GP Ablation Experiment	20
A.6 Injecting Diversity with Thompson Sampling	22
A.7 CLOCK Kernel Interpretation	24
A.8 CLOCK Training Objective	24
A.9 Proof-of-concept CLOCK Fine-tuning Experiment	26
A.10 Generalizing CLOCK	27
A.11 On the scalability of (C)LOCK GPs	28
A.12 Inventory of Substitution Matrices	29
A.13 Kermut Kernel	29
A.14 Description of Datasets and Evaluation Strategies	31
A.15 Speed and Memory Usage Benchmark	33
A.16 Experimental Details	34
A.17 Additional Figures and Tables for the Local Learning Experiment	38
A.18 Additional Figures and Tables for the Multi-task Learning Experiment	43
A.19 Code Snippets	43

A.1. Infinitely Divisible Matrices

A.1.1. DEFINITIONS AND USEFUL FACTS

We begin with some definitions.

Definition A.1 (Positive semidefinite matrix). A real symmetric matrix $M \in \mathbb{R}^{n \times n}$ is *positive semidefinite* or PSD if

$$x^\top M x \geq 0 \quad \text{for all } x \in \mathbb{R}^n.$$

We write $M \succeq 0$.

Definition A.2 (Elementwise exponential). For $A \in \mathbb{R}^{n \times m}$, the elementwise exponential $\exp^\circ(A) \in \mathbb{R}^{n \times m}$ is defined by

$$(\exp^\circ(A))_{ij} = e^{A_{ij}} \quad (1 \leq i \leq n, 1 \leq j \leq m).$$

It is important to emphasize that this is *not* the matrix exponential.

Definition A.3 (Hadamard (elementwise) power). Let $A = (A_{ij}) \in \mathbb{R}^{n \times m}$ and let $t > 0$. The Hadamard (elementwise) power $A^{\circ t} \in \mathbb{R}^{n \times m}$ is defined by

$$(A^{\circ t})_{ij} = (A_{ij})^t \quad (1 \leq i \leq n, 1 \leq j \leq m),$$

i.e., exponentiation is applied to each entry separately. For integer t , this is defined for any real A ; for non-integer real t , we require $A_{ij} \geq 0$ to remain in the real numbers.

Definition A.4 (Schur (elementwise) logarithm). Let $A = (A_{ij}) \in \mathbb{R}^{n \times m}$ have strictly positive entries ($A_{ij} > 0$). The Schur (elementwise) logarithm of A is

$$\log^\circ(A) \in \mathbb{R}^{n \times m}, \quad (\log^\circ(A))_{ij} \equiv \log(A_{ij}).$$

It is defined elementwise and should be distinguished from the matrix logarithm.

Definition A.5 (Infinitely divisible matrix). A symmetric matrix $K \in \mathbb{R}^{n \times n}$ with strictly positive entries is said to be *infinitely divisible* if its Hadamard (elementwise) powers $K^{\circ t}$ are positive semidefinite for all $t > 0$.

Definition A.6 (Conditionally positive semidefinite matrix). A real symmetric matrix $K \in \mathbb{R}^{n \times n}$ is *conditionally positive semidefinite* or CPSD if

$$x^\top K x \geq 0 \quad \text{for all } x \in \mathbb{R}^n \text{ with } \sum_{i=1}^n x_i = 0.$$

Equivalently, letting $J \equiv I - \frac{1}{n} \mathbf{1}\mathbf{1}^\top$ denote the orthogonal projector onto the zero-sum subspace, K is CPSD iff $JKJ \succeq 0$. A real symmetric matrix K is *conditionally negative semidefinite* or CNSD if $-K$ is CPSD.

Finally, we state Schoenberg's theorem, as we will use it below.

Theorem A.7 (Schoenberg (1938)). Let ψ be symmetric with $\psi(i, i) = 0$. Then the following are equivalent:

1. ψ is conditionally negative semidefinite;
2. for every $t > 0$, the kernel $k_t(i, j) = \exp(-t\psi(i, j))$ is positive semidefinite;
3. for finite index sets, ψ is a squared Euclidean distance.

A.1.2. CHARACTERIZING INFINITELY DIVISIBLE MATRICES

With these definitions in hand, we can give a precise characterization of infinitely divisible matrices.

Lemma A.8 (Berg et al. (1984)). Let K be a symmetric matrix with strictly positive entries. Then K is infinitely divisible if and only if $\log^\circ K$ is conditionally positive semidefinite, i.e.

$$x^\top (\log^\circ K) x \geq 0 \quad \text{for all } x \in \mathbb{R}^n \text{ with } \sum_i x_i = 0.$$

Proof sketch. We give two standard arguments; see Berg et al. (1984) for a detailed proof.

(CPSD $\Rightarrow$ ID via Schoenberg) Assume $A \equiv \log^\circ K$ is CPSD. Define

$$d(i, j) \equiv \frac{A_{ii} + A_{jj}}{2} - A_{ij}.$$

Then $d(i, i) = 0$, and for any x with $\sum_i x_i = 0$,

$$\sum_{i,j} x_i x_j d(i, j) = -x^\top A x \leq 0,$$

so d is CNSD. By Schoenberg's theorem, the kernel $B^{(t)}$ with entries $B_{ij}^{(t)} = \exp(-t d(i, j))$ is PSD for every $t > 0$. Noting the factorization

$$B_{ij}^{(t)} = \exp\left(-\frac{t}{2} A_{ii}\right) \exp(t A_{ij}) \exp\left(-\frac{t}{2} A_{jj}\right) = (D_t K^{\circ t} D_t)_{ij}, \quad D_t \equiv \text{diag}(e^{-t A_{11}/2}, \dots, e^{-t A_{nn}/2}),$$

we have $B^{(t)} = D_t K^{\circ t} D_t$. Since sandwiching with D_t^{-1} preserves PSD, $K^{\circ t} \succeq 0$ for every $t > 0$, i.e., K is infinitely divisible.

(ID $\Rightarrow$ CPSD) Assume K is infinitely divisible. Fix $x \in \mathbb{R}^n$ with $\sum_i x_i = 0$ and set

$$g_x(t) \equiv x^\top K^{\circ t} x = \sum_{i,j} x_i x_j K_{ij}^t \quad (t \geq 0).$$

For $t > 0$, $K^{\circ t} \succeq 0$ so $g_x(t) \geq 0$. As $t \downarrow 0$, $K^{\circ t} \rightarrow \mathbf{1}\mathbf{1}^\top$ elementwise, hence $g_x(0) \equiv \lim_{t \downarrow 0} g_x(t) = (\sum_i x_i)^2 = 0$. Thus g_x has a right minimum at $t = 0$, so $g'_x(0^+) \geq 0$. Differentiating termwise at $t = 0$ we obtain

$$g'_x(0^+) = \sum_{i,j} x_i x_j \log K_{ij} = x^\top (\log^\circ K) x \geq 0,$$

which is exactly conditional positive semidefiniteness of $\log^\circ K$. $\square$

A.1.3. CHARACTERIZING INFINITELY DIVISIBLE CORRELATION MATRICES

We can also characterize infinitely divisible *correlation* matrices.

Theorem A.9 (Characterization via squared Euclidean distances). *Let $K \in \mathbb{R}^{A \times A}$ be a correlation matrix with strictly positive entries. The following are equivalent:*

1. K is infinitely divisible.
2. $\log^\circ K$ is CPSD (equivalently $D \equiv -\log^\circ K$ is CNSD).
3. There exist points $x_1, \dots, x_A$ in $\mathbb{R}^m$ (for some $m \leq A - 1$) such that

$$K_{ij} = \exp(-\|x_i - x_j\|^2) \quad \text{for all } i, j.$$

In particular, every infinitely divisible correlation matrix is of the form $K = \exp^\circ(-D)$ where D is a squared Euclidean distance matrix.

Proof. By Lemma A.8, (1) $\Leftrightarrow$ (2). By Schoenberg's Theorem A.7, (2) $\Leftrightarrow$ (3): D is CNSD iff e^{-tD} is PSD for all $t > 0$, and for finite sets such a CNSD D is precisely a squared Euclidean distance. $\square$

Remark A.10. For all $n \times n$ doubly nonnegative matrices A (i.e. PSD matrices with all nonnegative entries), the Hadamard power $A^{\circ t}$ preserves positive semidefiniteness iff $t \in \mathbb{N}$ or $t \geq n - 2$ (FitzGerald & Horn, 1977). This severe dimension-dependent restriction on t motivates our use of infinitely divisible matrices, which enable learning the exponent t with gradient methods for all positive t .

A.2. Local Linearity

Suppose $\mathbf{x} \in \mathbb{R}^D$ and let $f(\mathbf{x}) = \beta(\mathbf{x}) \cdot \mathbf{x} = \sum_{d=1}^D \beta_d(\mathbf{x}) x_d$. We place an independent GP prior on each component $\beta_d(\mathbf{x})$ of $\beta(\mathbf{x})$ with zero mean and with kernel $k_d(\cdot, \cdot)$, i.e. $\langle \beta_d(\mathbf{x}) \rangle = 0$ and $\langle \beta_d(\mathbf{x}) \beta_d(\mathbf{y}) \rangle = k_d(\mathbf{x}, \mathbf{y})$. Since each $\beta_d(\mathbf{x})$ is Gaussian distributed and $f(\mathbf{x})$ is linear in $\beta(\mathbf{x})$, $f(\mathbf{x})$ is evidently Gaussian. To determine the GP prior that controls $f(\mathbf{x})$ we need to compute the corresponding mean and covariance. First compute the mean

$$\langle f(\mathbf{x}) \rangle = \langle \beta(\mathbf{x}) \cdot \mathbf{x} \rangle = \langle \beta(\mathbf{x}) \rangle \cdot \mathbf{x} = 0 \quad (16)$$

Next compute the covariance

$$\langle f(\mathbf{x}) f(\mathbf{y}) \rangle = \langle (\beta(\mathbf{x}) \cdot \mathbf{x}) (\beta(\mathbf{y}) \cdot \mathbf{y}) \rangle = \sum_{d,d'} x_d y_{d'} \langle \beta_d(\mathbf{x}) \beta_{d'}(\mathbf{y}) \rangle = \sum_d x_d y_d \langle \beta_d(\mathbf{x}) \beta_d(\mathbf{y}) \rangle = \sum_d x_d y_d k_d(\mathbf{x}, \mathbf{y}) \quad (17)$$

If we specialize to the case that $k_d(\mathbf{x}, \mathbf{y}) = k_{\text{nl}}(\mathbf{x}, \mathbf{y})$ for $d = 1, \dots, D$ then the kernel in Eqn. 17 simplifies to $k_{\text{nl}}(\mathbf{x}, \mathbf{y}) k_{\text{lin}}(\mathbf{x}, \mathbf{y})$ where $k_{\text{lin}}(\mathbf{x}, \mathbf{y}) \equiv \mathbf{x}^\top \mathbf{y} = \mathbf{x} \cdot \mathbf{y}$ is the canonical linear kernel. So we conclude that $f(\mathbf{x}) = \beta(\mathbf{x}) \cdot \mathbf{x}$ is a zero-mean Gaussian process governed by the product kernel $k_{\text{nl}}(\mathbf{x}, \mathbf{y}) k_{\text{lin}}(\mathbf{x}, \mathbf{y})$.

Now suppose that $\beta(\mathbf{x})$ is a vector-valued Gaussian process with zero mean and with kernel $\langle \beta_d(\mathbf{x})\beta_{d'}(\mathbf{y}) \rangle = U_{dd'}k_{\text{nl}}(\mathbf{x}, \mathbf{y})$ where $\mathbf{U}$ is a PSD matrix. Then Eqn. 17 becomes

$$\langle f(\mathbf{x})f(\mathbf{y}) \rangle = \sum_{d,d'} x_d y_{d'} \langle \beta_d(\mathbf{x})\beta_{d'}(\mathbf{y}) \rangle = \sum_{d,d'} x_d y_{d'} U_{dd'} k_{\text{nl}}(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{U} \mathbf{y}) k_{\text{nl}}(\mathbf{x}, \mathbf{y}) \quad (18)$$

In other words the resulting kernel is still a product of a linear kernel and k_{nl} , only now the linear kernel is $\mathbf{x}^T \mathbf{U} \mathbf{y}$ instead of the isotropic kernel $\mathbf{x}^T \mathbf{y}$. This is precisely the scenario we have in Sec. 3.3, where $\mathbf{U}$ is a large block diagonal covariance matrix of size $(LA) \times (LA)$ where there are L blocks of size $A \times A$ and each block is a correlation matrix $\mathbf{C}_\ell$, see Eqn. 11.²¹

Finally, we consider the case where $\beta(\mathbf{x})$ is a vector-valued Gaussian process with zero mean and with kernel given by

$$\langle \beta_d(\mathbf{x})\beta_{d'}(\mathbf{y}) \rangle = U_{dd'} (\kappa + k_{\text{nl}}(\mathbf{x}, \mathbf{y})) \quad (19)$$

where $\kappa > 0$ is a fixed constant and we note that a constant covariance function is equivalent to including an intercept term in the function class (with its prior covariance set by κ). In this case we find that the effective kernel for $f(\mathbf{x})$ is given by

$$(\mathbf{x}^T \mathbf{U} \mathbf{y}) (\kappa + k_{\text{nl}}(\mathbf{x}, \mathbf{y})) = \kappa (\mathbf{x}^T \mathbf{U} \mathbf{y}) + k_{\text{nl}}(\mathbf{x}, \mathbf{y}) (\mathbf{x}^T \mathbf{U} \mathbf{y}) \quad (20)$$

In other words we have a kernel that is of the form “linear kernel plus non-linear kernel times linear kernel.” This is precisely the form of the LOCK kernel in Eqn. 22, where the only difference is that in the latter we have decoupled the hyperparameters used in each kernel in the sum.²² This decoupling does not dramatically expand the class of functions under consideration, but it does make the kernel a bit more flexible. Put differently, we note that, depending on the application, it may be undesirable for the covariance function that controls $\beta(\mathbf{x})$ to revert to zero far away from the training data. By including κ we instead revert to a learnable constant $\kappa \mathbf{U}$, and the result is an ‘extra’ linear kernel in Eqn. 20.

Whatever its precise form, if the kernel $k_{\text{nl}}(\mathbf{x}, \mathbf{y})$ is such that $\beta(\mathbf{x})$ varies slowly across the input space $\mathbf{x}$, then $f(\mathbf{x})$ is naturally viewed as locally linear. We note that nothing in our derivation requires $k_{\text{nl}}(\mathbf{x}, \mathbf{y})$ to be non-linear, but this is the most natural ansatz in practice.

A.3. Detailed LOCK Kernel Description

We restate the definitions of the two base kernels used in LOCK (see Eqn. 11-12)

$$k_{\text{lin}}^{\text{LOCK}}(\mathbf{x}, \mathbf{y}) = \sum_{\ell=1}^L \mathbf{x}_\ell^T \mathbf{C}_\ell^{\alpha_\ell} \mathbf{y}_\ell \quad k_{\text{nl}}^{\text{LOCK}}(\mathbf{x}, \mathbf{y}) = \prod_{\ell=1}^L \mathbf{x}_\ell^T \mathbf{C}_\ell^{\alpha_\ell} \mathbf{y}_\ell \quad (21)$$

as well as the full LOCK kernel

$$k^{\text{LOCK}}(\mathbf{x}, \mathbf{y}) = \sigma_1^2 k_{\text{nl}}^{\text{LOCK}}(\mathbf{x}, \mathbf{y}) k_{\text{lin}}^{\text{LOCK}}(\mathbf{x}, \mathbf{y}) + \sigma_2^2 \tilde{k}_{\text{lin}}^{\text{LOCK}}(\mathbf{x}, \mathbf{y}) \quad (22)$$

The first linear kernel $k_{\text{lin}}^{\text{LOCK}}$ is equipped with a global exponent $\alpha^{(1)}$, while the second linear kernel $\tilde{k}_{\text{lin}}^{\text{LOCK}}$ is equipped with a global exponent $\alpha^{(2)}$. The non-linear kernel $k_{\text{nl}}^{\text{LOCK}}$ is equipped with positionwise exponents α_ℓ . To be explicit the first linear kernel is given by

$$k_{\text{lin}}^{\text{LOCK}}(\mathbf{x}, \mathbf{y}) = \sum_{\ell=1}^L \mathbf{x}_\ell^T \mathbf{C}_\ell^{\alpha^{(1)}} \mathbf{y}_\ell \quad (23)$$

and the second linear kernel is given by

$$\tilde{k}_{\text{lin}}^{\text{LOCK}}(\mathbf{x}, \mathbf{y}) = \sum_{\ell=1}^L \mathbf{x}_\ell^T \mathbf{C}_\ell^{\alpha^{(2)}} \mathbf{y}_\ell \quad (24)$$

while the non-linear kernel is as in Eqn. 21. In the regression case we also have the noise scale σ_n (see Eqn. 2). Thus we have $L + 5$ hyperparameters: $\{\sigma_1, \sigma_2, \sigma_n, \alpha^{(1)}, \alpha^{(2)}, \alpha_{1:L}\}$. While it would of course be possible to introduce local exponents for each linear kernel—or even additional positive multiplicative parameters that modulate the overall scale of each correlation matrix $\mathbf{C}_\ell$ —we find that this choice strikes a good balance between flexibility and parsimony.

²¹Of course in the LOCK kernel correlation matrices $\mathbf{C}_\ell$ also appear in the non-linear kernel Eqn. 12.

²²In particular we introduce separate exponents $\alpha^{(1)}$, $\alpha^{(2)}$, and α_ℓ ; see Sec. A.3.

	Spearman R			MAE		
	LOCK-GP	Kermut-GP	MLP-ESM2-LastLayer	LOCK-GP	Kermut-GP	MLP-ESM2-LastLayer
Cross-validation ($N = 48$)	0.22	0.13	0.25	0.31	0.32	0.27
Cross-validation ($N = 1536$)	0.42	0.45	0.53	0.26	0.36	0.14
Extrapolation ($N = 128$)	0.32	0.21	0.36	0.41	1.14	0.42
Extrapolation ($N = 512$)	0.34	0.26	0.44	0.62	1.03	0.33

Table 3. We report Spearman R and MAE metrics for the GB1 dataset in both the cross-validation and extrapolation regime, with the number of training data points ranging from $N = 48$ to $N = 1536$. The extrapolation training and test sets are defined using a Hamming distance cutoff of $D = 2$; c.f. Sec. A.14.

A.3.1. NORMALIZATION

In practice we use the following normalization scheme for correlation matrices. First we start with a raw substitution matrix, e.g. BLOSUM50, denoted by $\mathbf{S}$ (assumed to be exponentiated, i.e. not in log-odds space). We then define the corresponding correlation matrix $C_{aa'} \equiv S_{aa'} / \sqrt{S_{aa} S_{a'a'}}$. In order to make it straightforward to define priors over exponent hyperparameters we do one final normalization step. In particular we raise $\mathbf{C}$ to the unique power that makes the median off-diagonal entry of $\mathbf{C}$ equal to $\exp(-\frac{1}{4})$. In other words $\mathbf{C} \rightarrow \mathbf{C}^{ot}$ where $t = -\frac{1}{4 \log m}$ and m is the median of the off-diagonal entries of $\mathbf{C}$. An example of such a normalized correlation matrix can be seen in the right panel of Fig. 1.

A.3.2. HYPERPARAMETER PRIORS

We place $\text{Gamma}(2, 2)$ priors on both kernel variances σ_1^2 and σ_2^2 and place a $\text{Gamma}(2, 2)$ on the observation noise variance σ_n^2 .²³ We place a $\text{LogNormal}(0, 1)$ prior on global exponents $\alpha^{(1)}$ and $\alpha^{(2)}$. We use an overcomplete parameterization for local exponents, i.e. $\alpha_\ell = \tilde{\alpha} \tilde{\alpha}_\ell$, where $\tilde{\alpha}$ is a scalar and where $\tilde{\alpha}_{1:L} \in \mathbb{R}^L$. We then place a $\text{LogNormal}(0, 1)$ prior on the scalar $\tilde{\alpha}$ and a $\text{LogNormal}(0, \frac{1}{4})$ prior on each $\tilde{\alpha}_\ell$. Marginally, this choice corresponds to a $\text{LogNormal}(0, \frac{5}{4})$ prior on α_ℓ . We note that choosing an overcomplete parameterization has an impact on the *optimization dynamics*; in particular by introducing $\tilde{\alpha}$ we expect it to be easier to take larger steps in α_ℓ space. These priors were chosen based on regression experiments conducted with datasets disjoint from those used in our experiments in Sec. 5.

A.4. Epistasis and LOCK

The LOCK base kernels in Eqn. 11-12 are defined residuewise and do not introduce explicit learnable parameters for specific pairwise or higher-order interactions. Nevertheless, the full LOCK kernel in Eqn. 13 can induce strongly non-additive behavior, as follows. First, the non-linear kernel k_{nl} has a multiplicative structure across positions, which couples residuewise contributions and makes motif-level effects natural to represent. Second, because the full kernel is locally linear, the effective coefficient of a mutation can vary smoothly with sequence context. In this sense, LOCK is best viewed as modeling smooth, context-dependent epistasis rather than explicit sparse interaction terms. As described in Sec. 3.4, the linear kernel uses rotated residue-level basis functions, while the non-linear kernel uses their tensor products. Consequently, observing that a particular multi-residue motif is associated with high fitness tends to increase predicted fitness for nearby motifs composed of biophysically similar residues. This allows LOCK to capture a broad class of higher-order effects that are mediated by shared residue similarities and local smoothness. Accordingly, LOCK is particularly well-suited to modeling epistasis between classes of biophysically similar substitutions, rather than to arbitrary interactions between mutations. Notably, LOCK does not separately parameterize each pairwise, three-way, or higher-order interaction. The induced epistatic structure is constrained: it is determined implicitly by the residuewise correlation matrices $\mathbf{C}_{1:L}$, the learned exponents, and the multiplicative/local-linear kernel structure. This is arguably a feature in the low-data regime, where attempting to learn unconstrained higher-order epistasis directly is generally infeasible. A useful analogy might be the following. Suppose z is a gaussian random variable with mean zero and covariance matrix Σ . Then higher-order correlations like $\langle z_i z_j z_k z_l \rangle$ are entirely determined by $\Sigma_{ij} = \langle z_i z_j \rangle$. Similarly, in LOCK, the induced higher-order epistatic structure is governed by the lower-order correlation matrices $\mathbf{C}_{1:L}$.

In Table 3 we benchmark LOCK-GP against two baselines on the GB1 dataset (Wu et al., 2016), which has four variable positions and is known to exhibit significant pairwise as well as higher-order epistasis in these positions. We find that all three

²³We also use these priors for Tanimoto-GP; for Kermut we use the priors in Groth et al. (2024).

models struggle to provide good fits, especially in the low-data regime, although the neural model MLP-ESM2-LastLayer achieves somewhat better performance, at least for $N = 1536$ training data points. These results highlight the challenge of obtaining good predictive models for highly epistatic landscapes in the low-data regime.

A.5. GP Ablation Experiment

Model	SpearmanR	PearsonR	MAE	RMSE	CRPS	NLL	ECE
LOCK	0.610	0.622	0.591	0.823	0.456	2.560	0.113
LOCK-45	0.607	0.620	0.592	0.824	0.457	2.570	0.113
LOCK-62	0.609	0.621	0.592	0.825	0.458	2.624	0.115
LOCK-80	0.609	0.621	0.592	0.824	0.457	2.604	0.113
LOCK-WeakPrior	0.574	0.575	0.591	0.884	0.491	8.933	0.161
LOCK-NoLinear	0.609	0.622	0.596	0.824	0.456	3.004	0.106
LOCK-DoublyLocal	0.612	0.623	0.587	0.821	0.454	2.554	0.117
LOCK-LinearOnly-Global	0.527	0.538	0.670	0.890	0.503	2.525	0.114
LOCK-LinearOnly-Local	0.534	0.544	0.664	0.890	0.497	2.665	0.107
LOCK-NonLinearOnly-Global	0.598	0.607	0.625	0.842	0.473	2.735	0.118
LOCK-NonLinearOnly-Local	0.608	0.622	0.599	0.824	0.459	3.304	0.108
Linear	N/A	N/A	N/A	N/A	N/A	N/A	N/A
RBF	0.565	0.564	1.710	6.509	1.544	8.3×10^8	0.176
Tanimoto	0.555	0.560	0.675	0.881	0.499	2.586	0.125
Tanimoto-50	0.556	0.562	0.674	0.880	0.498	2.497	0.124
Tanimoto-Orig	0.547	0.552	0.678	0.883	0.501	2.550	0.127
Kermut	0.629	0.632	0.617	0.827	0.465	2.773	0.097
KermutSeq	0.532	0.537	0.681	0.894	0.507	46.657	0.109
KermutStruc	0.614	0.613	0.634	0.846	0.478	2.982	0.104

Table 4. Performance metrics in the unseen mutations regime with 96 training data points for the GP ablation experiment. RMSE is root mean squared error; CRPS is continuous ranked probability score (Gneiting & Raftery, 2007); NLL is negative log likelihood; ECE is expected calibration error. Note that we obtained constant predictions for some Linear predictors in this regime, so we do not include results for this model. See Sec. A.5 for discussion.

To better understand the effect of different choices that enter into the LOCK kernel in Eqn. 13, we perform a systematic ablation experiment in which we train 14 additional models. For the results see Tables 4-6. To make cross-comparison easier, we also report results for the three Kermut variants and Tanimoto-GP. In the following we dissect the results in detail.

While we use the BLOSUM50 substitution matrix by default, there are many more BLOSUM variants (see Table 7). In the ablation we compare to BLOSUM45, BLOSUM62, and BLOSUM80. We see that the differences are generally quite small, which is probably to be expected since the normalization step in Eqn. 10 substantially reduces the differences between different BLOSUM variants. While we do not show the results here, we find that this weak dependence also holds for individual landscapes.

Next we consider the hyperparameter priors described in Sec. 3.5, focusing on the exponents α_ℓ that enter $k_{\text{nl}}^{\text{LOCK}}$. Instead of removing the priors entirely, we instead place a much weaker prior on α_ℓ , as removing the prior entirely leads to numerical instabilities.²⁴ Under the weak prior—see LOCK-WeakPrior—predictive performance is impaired across the board.

Next we consider the single linear kernel that enters into the LOCK kernel in Eqn. 13, i.e. the term $\sigma_2^2 \tilde{k}_{\text{lin}}^{\text{LOCK}}(\mathbf{x}, \mathbf{y})$. We remove this kernel component entirely, resulting in LOCK-NoLinear. We find that while uncertainty-unaware metrics like SpearmanR and MAE are very similar for LOCK and LOCK-NoLinear, the NLL of the latter is degraded for the most challenging OOD regimes. In particular in the unseen mutations regime (resp., extrapolation regime with 128 data points) LOCK obtains a lower NLL than LOCK-NoLinear on 15/21 (resp., 14/21) landscapes. For this reason, and for the reasons outlined in Sec. 3.4, we prefer to keep the term $\sigma_2^2 \tilde{k}_{\text{lin}}^{\text{LOCK}}(\mathbf{x}, \mathbf{y})$, although we would generally expect to get good predictive performance without it.

With reference to Sec. A.3, we see that the only ‘local’—i.e. residuewise—parameters in LOCK are the exponents α_ℓ that modulate the non-linear kernel $k_{\text{nl}}^{\text{LOCK}}$. In LOCK-DoublyLocal we also include residuewise exponents in the single linear kernel $\sigma_2^2 \tilde{k}_{\text{lin}}^{\text{LOCK}}(\mathbf{x}, \mathbf{y})$. From Tables 4-6 we can see that this additional flexibility can lead to slightly improved performance in some (but not all) cases. Since the differences are small, and since this change approximately doubles the total number of

²⁴In particular we place a LogNormal(0, 16) prior on the scalar $\tilde{\alpha}$ and a LogNormal(0, 16) prior on each $\tilde{\alpha}_\ell$; c.f. Sec. A.3.2.

Flexible Kernels for Protein Property Prediction

	128 training data points						512 training data points					
	SpearmanR	PearsonR	MAE	CRPS	NLL	ECE	SpearmanR	PearsonR	MAE	CRPS	NLL	ECE
LOCK	0.669	0.711	0.592	0.442	1.954	0.120	0.759	0.807	0.440	0.333	1.262	0.097
LOCK-45	0.668	0.710	0.593	0.444	1.957	0.125	0.757	0.805	0.442	0.334	1.270	0.095
LOCK-62	0.668	0.710	0.594	0.443	1.952	0.121	0.758	0.806	0.441	0.334	1.258	0.098
LOCK-80	0.668	0.710	0.596	0.445	1.963	0.125	0.757	0.805	0.444	0.336	1.271	0.101
LOCK-WeakPrior	0.649	0.687	0.568	0.466	9.912	0.161	0.740	0.784	0.428	0.335	2.228	0.110
LOCK-NoLinear	0.668	0.709	0.591	0.443	2.043	0.126	0.759	0.805	0.441	0.333	1.295	0.098
LOCK-DoublyLocal	0.671	0.712	0.589	0.438	1.942	0.112	0.753	0.800	0.447	0.336	1.263	0.084
LOCK-LinearOnly-Global	0.609	0.617	0.868	0.668	3.074	0.137	0.713	0.714	0.822	0.646	3.962	0.150
LOCK-LinearOnly-Local	0.611	0.620	0.867	0.668	3.193	0.134	0.716	0.717	0.810	0.637	4.020	0.155
LOCK-NonLinearOnly-Global	0.659	0.682	0.663	0.497	2.158	0.113	0.749	0.786	0.507	0.385	1.632	0.100
LOCK-NonLinearOnly-Local	0.667	0.709	0.595	0.447	2.259	0.131	0.759	0.806	0.440	0.333	1.376	0.098
Linear	0.586	0.596	0.916	0.700	3.092	0.144	0.698	0.698	0.854	0.667	3.974	0.149
RBF	0.631	0.668	0.671	0.512	3.123	0.193	0.646	0.683	7.484	7.363	1.5×10^{10}	0.189
Tanimoto	0.632	0.654	0.762	0.573	2.119	0.120	0.739	0.769	0.574	0.424	1.552	0.087
Tanimoto-50	0.633	0.655	0.755	0.568	2.109	0.117	0.744	0.772	0.558	0.414	1.543	0.088
Tanimoto-Orig	0.630	0.652	0.764	0.575	2.115	0.122	0.736	0.767	0.582	0.431	1.587	0.088
Kermut	0.639	0.654	0.845	0.657	3.867	0.159	0.750	0.767	0.670	0.498	2.297	0.108
KermutSeq	0.518	0.505	0.836	0.635	159.803	0.138	0.605	0.614	0.714	0.540	2.982	0.122
KermutStruc	0.624	0.641	0.847	0.665	4.243	0.159	0.706	0.713	0.847	0.670	4.808	0.167

Table 5. Performance metrics in the Hamming-distance-based extrapolation regime for the GP ablation experiment, with the number of training data points equal to either 128 or 512. RMSE is root mean squared error; CRPS is continuous ranked probability score (Gneiting & Raftery, 2007); NLL is negative log likelihood; ECE is expected calibration error. See Sec. A.5 for discussion.

	48 training data points						1536 training data points					
	SpearmanR	PearsonR	MAE	CRPS	NLL	ECE	SpearmanR	PearsonR	MAE	CRPS	NLL	ECE
LOCK	0.653	0.678	0.498	0.361	0.971	0.056	0.867	0.914	0.210	0.160	0.112	0.108
LOCK-45	0.653	0.678	0.499	0.361	0.973	0.055	0.866	0.913	0.211	0.161	0.117	0.108
LOCK-62	0.652	0.676	0.500	0.362	0.975	0.055	0.867	0.914	0.210	0.160	0.110	0.108
LOCK-80	0.652	0.677	0.500	0.362	0.974	0.055	0.867	0.914	0.210	0.160	0.113	0.109
LOCK-WeakPrior	0.619	0.634	0.478	0.394	121.508	0.135	0.860	0.908	0.205	0.160	0.368	0.113
LOCK-NoLinear	0.655	0.680	0.494	0.358	0.967	0.060	0.867	0.915	0.210	0.161	0.099	0.108
LOCK-DoublyLocal	0.654	0.679	0.497	0.360	0.971	0.053	0.867	0.914	0.210	0.160	0.108	0.109
LOCK-LinearOnly-Global	0.554	0.563	0.586	0.414	1.116	0.047	0.809	0.806	0.415	0.298	0.749	0.040
LOCK-LinearOnly-Local	0.562	0.573	0.578	0.410	1.111	0.046	0.811	0.807	0.412	0.296	0.746	0.039
LOCK-NonLinearOnly-Global	0.623	0.636	0.548	0.389	1.054	0.059	0.855	0.902	0.254	0.189	0.240	0.076
LOCK-NonLinearOnly-Local	0.651	0.675	0.496	0.359	0.974	0.061	0.867	0.915	0.210	0.161	0.107	0.107
Linear	0.506	0.521	0.603	0.424	1.142	0.054	0.802	0.800	0.420	0.302	0.762	0.040
RBF	0.610	0.602	0.561	0.421	3.7×10^5	0.102	N/A	N/A	N/A	N/A	N/A	N/A
Tanimoto	0.514	0.511	0.592	0.415	1.106	0.075	0.846	0.888	0.272	0.201	0.281	0.068
Tanimoto-50	0.515	0.513	0.590	0.414	1.103	0.075	0.846	0.888	0.271	0.200	0.275	0.068
Tanimoto-Orig	0.510	0.507	0.593	0.415	1.106	0.074	0.845	0.887	0.273	0.202	0.282	0.067
Kermut	0.636	0.638	0.549	0.398	1.415	0.048	0.850	0.888	0.285	0.212	0.358	0.066
KermutSeq	0.540	0.537	0.619	0.443	45.840	0.053	0.794	0.838	0.338	0.248	0.476	0.061
KermutStruc	0.615	0.618	0.569	0.413	1.752	0.050	0.809	0.805	0.417	0.300	0.763	0.040

Table 6. Performance metrics in the cross-validation regime for the GP ablation experiment, with the number of training data points ranging from 48 to 1536. RMSE is root mean squared error; CRPS is continuous ranked probability score (Gneiting & Raftery, 2007); NLL is negative log likelihood; ECE is expected calibration error. Note we did not obtain complete results for the RBF kernel due to numerical issues. See Sec. A.5 for discussion.

parameters (from $L + 5$ to $2L + 4$), we prefer the more parsimonious choice in LOCK.

Next we consider the base kernels $k_{\text{nl}}^{\text{LOCK}}$ and $k_{\text{lin}}^{\text{LOCK}}$ in isolation, considering both global and local versions of each (i.e. with scalar and residuewise exponents, respectively; c.f. Sec. 3.5). First we note that it is generally advantageous to include the additional flexibility of residuewise parameters. Next we note the superiority of the non-linear kernel $k_{\text{nl}}^{\text{LOCK}}$ to the linear kernel $k_{\text{lin}}^{\text{LOCK}}$. Finally we note that while $k_{\text{nl}}^{\text{LOCK}}$ in isolation approaches the performance of LOCK for uncertainty-unaware metrics like SpearmanR and MAE, it has noticeably worse NLL for the more challenging OOD regimes. In particular in the unseen mutations regime (resp., extrapolation regime with 128 data points) LOCK obtains a lower NLL than LOCK-NonLinearOnly-Local on 18/21 (resp., 14/21) landscapes. For this reason we prefer the locally linear LOCK kernel, although we would generally expect good performance from $k_{\text{nl}}^{\text{LOCK}}$ alone (with the caveat that using $k_{\text{nl}}^{\text{LOCK}}$ in isolation results in mean-reverting predictions away from the training data, which may be undesirable depending on the application). Taken together the results in Tables 4-6 demonstrate that the most crucial ingredient in LOCK is arguably fit-to-purpose incorporation of biophysical similarity as encoded by (infinitely divisible) BLOSUM substitution matrices.

Next we consider an isotropic linear kernel as in Eqn. 5. We find that Linear is uniformly outperformed by LOCK-LinearOnly-Global, highlighting the value of incorporating substitution matrices into the kernel—and as will be confirmed further in the non-linear case in the following paragraph.

Next we consider an RBF kernel as in Eqn. 7 with learnable lengthscales for each residue. We place a $\text{Gamma}(4, 2)$ prior on each lengthscale τ_ℓ , i.e. the prior probability is maximized for $\tau_\ell = 2$ so that the typical ‘similarity’ of nonidentical amino acids is $\sim \exp(-1/4) \approx 0.78$. The anisotropic RBF kernel performs relatively poorly across the board, emphasizing the value of incorporating substitution matrices into the kernel (we note that these improvements tend to be larger than in the linear case explored in the previous paragraph). Indeed on some datasets the NLL is catastrophically bad ($> 10^8$). Moreover the RBF kernel can be numerically unstable: note that we did not obtain complete results for the RBF kernel in Table 6.

Finally we consider whether the Tanimoto kernel exhibits strong dependence on the BLOSUM substitution matrix used. We consider BLOSUM62 (the default) as well as BLOSUM50. We find little difference in predictive performance. We also compare to the original Tanimoto kernel (i.e. without eigenvalue clamping) described in Gessner et al. (2024) (see Sec. A.16.1 for discussion), which we find to perform slightly worse on average.

Additional Discussion We have also considered LOCK variants that utilize matrix exponentiation instead of Hadamard exponentiation to modulate correlation matrices $\mathbf{C}_\ell$. While we find that this is viable, it is less numerically stable. As such we prefer using Hadamard exponentiation; this choice is particularly important for CLOCK, where numerical stability is crucial for robust learning of kernels parameterized by tens of thousands of parameters.

We have also considered LOCK variants analogous to LOCK-DoublyLocal described above in which the residuewise parameters are tied together. We find that this can work well on some datasets, but works poorly on others. Consequently we prefer the simpler LOCK.

More broadly, instead of selecting a single LOCK variant, we could do a small kernel search and choose the best kernel architecture for each dataset. While we do not pursue this option here, it is likely that by doing so we could squeeze some additional predictive performance from LOCK, at least on some datasets.

A.6. Injecting Diversity with Thompson Sampling

We consider the ParD3 dataset from Ding et al. (2024), which contains sequences of length $L = 93$ and has a variable region of length $L' = 10$. In the following in silico exercise we set out to generate ParD3 antitoxin variants that exhibit improved neutralization of the cognate toxin ParE3. To do so we train a LOCK GP predictor on the experimental growth rates and use the resulting predictor as a surrogate for neutralization ability in an optimization problem.

Before considering Thompson sampling, we first consider a multi-sequence optimization problem of the form

$$\max_{\mathbf{x}_{1:N}} \Phi(\mathbf{x}_{1:N}) \quad \text{with} \quad \Phi(\mathbf{x}_{1:N}) \equiv \Psi(\mathbf{x}_{1:N}) + \sum_{n=1}^N \phi(\mathbf{x}_n) \quad (25)$$

defined on $N = 800$ sequences $\mathbf{x}_{1:N} \equiv \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$. We limit the optimization to the L' residues of the variable region (i.e. all other residues are fixed to the wild-type amino acid) and include all 20 canonical amino acids in the design space. Here ϕ is a LOCK GP predictor trained on all the data from Ding et al. (2024), and higher values of $\phi(\cdot)$ correspond to

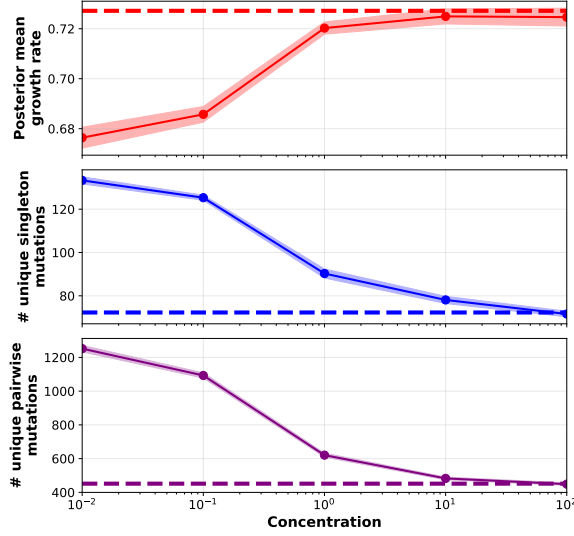


Figure 8. We depict results for the Thompson sampling experiment in Sec. A.6. Solid lines depict properties of the sequences obtained from solving Eqn. 26 at five distinct concentrations α ; uncertainty bands reflect standard errors. Dashed lines depict properties of the sequences obtained from solving Eqn. 25, which corresponds to the $\alpha \rightarrow \infty$ limit. We average across 10 replicate experiments. As the concentration α goes to zero and thus the diversity of the Thompson ensemble increases, the sequence diversity—as measured by the total number of unique singleton and pairwise mutations w.r.t. the wild-type sequence—increases substantially, with e.g. the number of pairwise mutations approximately tripling. This increased diversity comes at a relatively small cost in average predicted growth rate.

higher predicted growth rates. Furthermore, $\Psi(\mathbf{x}_{1:N})$ is a diversity-promoting objective that serves the following role:

1. it heavily penalizes sets of sequences $\mathbf{x}_{1:N}$ that contain duplicate (i.e. non-unique) sequences
2. it strongly encourages the sequences $\mathbf{x}_{1:N}$ to spread out in a balanced manner across a nested set of 8 Hamming shells centered around the wild-type sequence $\mathbf{x}_{\text{wt}}$

In particular $\Psi(\mathbf{x}_{1:N})$ strongly encourages exactly 100 of the 800 sequences to reside in the 2-Hamming shell of $\mathbf{x}_{\text{wt}}$, exactly 100 of the 800 sequences to reside in the 3-Hamming shell of $\mathbf{x}_{\text{wt}}$, and so on through the 9-Hamming shell of $\mathbf{x}_{\text{wt}}$. As such $\Psi(\mathbf{x}_{1:N})$ encourages mutational diversity in solutions to Eqn. 25. Nevertheless, since our design space is extremely large,²⁵ the diversity effects of $\Psi(\mathbf{x}_{1:N})$ are somewhat modest. Indeed consider two stylized putative solutions. In solution A, the 100 unique sequences placed in the 9-Hamming shell of $\mathbf{x}_{\text{wt}}$ are very different from one another, with typical pairwise Hamming distances of $\sim 8 - 9$. In solution B, the 100 unique sequences placed in the 9-Hamming shell of $\mathbf{x}_{\text{wt}}$ are quite similar to one another, with typical pairwise Hamming distances of ~ 2 . By assumption both A and B optimally meet the conditions set by Ψ and so the properties of $\phi(\cdot)$ determine whether solution A or B is preferred. In other words, the diversity of the solution to Eqn. 25 can vary dramatically as $\phi(\cdot)$ changes. Consequently, from the point of view of diversity the optimization problem in Eqn. 25 is somewhat undetermined. If ensuring sequence diversity in the solution is important, additional mechanisms for injecting diversity must be considered.

Here we explore how to use (pseudo-)Thompson sampling to inject a controllable amount of diversity into the optimization problem in Eqn. 25. To do so we generalize Eqn. 25 to leverage $N = 800$ predictors $\{\phi_n\}$ instead of a single common ϕ :

$$\max_{\mathbf{x}_{1:N}} \Phi(\mathbf{x}_{1:N}|\alpha) \quad \text{with} \quad \Phi(\mathbf{x}_{1:N}|\alpha) \equiv \Psi(\mathbf{x}_{1:N}) + \sum_{n=1}^N \phi_n(\mathbf{x}_n|\alpha) \quad (26)$$

Here α is a hyperparameter that controls the diversity of the ensemble $\{\phi_n\}$. By construction the diversity of the solution to Eqn. 26 increases as the diversity of the ensemble increases. While the diversity in $\{\phi_n\}$ could come from any number of sources, in the following each ϕ_n is obtained by a (pseudo-)Thompson sampling procedure that we now describe.

²⁵The design space for a single sequence consists of $20^{10} \approx 10^{13}$ sequences, and we consider a joint design problem defined over $N = 800$ sequences.

Let $\alpha > 0$ be the concentration parameter of a symmetric Dirichlet distribution on the $N - 1$ simplex. We draw a sample $\omega \sim \text{Dir}(\alpha)$ so that each ω_k has mean $\frac{1}{N}$. We then define $\tilde{\omega} \equiv \omega / \text{median}(\omega)$. Thus typically $\tilde{\omega}_k \approx 1$ and the spread around 1 increases as $\alpha \rightarrow 0$. We then modify the GP posterior mean prediction given in Eqn. 3 as follows

$$\mu_{\mathbf{f}}(\mathbf{x}^*|\tilde{\omega}) \equiv k_{*\mathbf{X}}^T (K_{\mathbf{X}\mathbf{X}} + \sigma_n^2 \text{diag}(\tilde{\omega}))^{-1} \mathbf{t} \quad (27)$$

where $\text{diag}(\tilde{\omega})$ is the $N \times N$ diagonal matrix with entries $\tilde{\omega}_k$ along the diagonal. In other words we stochastically modulate the assumed observation noise of each training data point in $(\mathbf{X}, \mathbf{t})$, while keeping other hyperparameters fixed. We do this for each predictor ϕ_n , i.e. each ϕ_n uses the formula in Eqn. 27 with a distinct sample $\tilde{\omega}_n$. This procedure results in an ensemble of predictors, each of which modulates the amount of smoothing in different amounts of sequence space as controlled by $\tilde{\omega}$, but in a way that does not differ dramatically from the canonical posterior mean prediction with $\tilde{\omega}_k \rightarrow 1$.²⁶

To demonstrate the effect of modulating the concentration α , we do an experiment in which we obtain approximate solutions to Eqn. 26 using a MCMC-and-annealing-based optimization algorithm that leverages the gradient-based proposal from Zhang et al. (2022). See Fig. 8 for the results. As expected, as the concentration goes to zero and the variability in $\tilde{\omega}$ increases, the sequence diversity w.r.t. the wild-type sequence increases substantially. While this comes at a cost in average predicted growth rate $\phi(\cdot)$, this cost is moderate. For example at $\alpha = 0.1$ the average predicted growth rate—as estimated by the canonical GP posterior mean—is reduced by about 0.04. This should be compared to the typical standard deviation of predicted growth rates within each of the eight Hamming shells, which is about 0.30. In other words the shift of 0.04 is a small fraction of the variability in the predicted growth rates for the 800 sequences in $\mathbf{x}_{1:N}$.

Thus we have demonstrated the viability of using the uncertainty information encoded by LOCK GP to trade-off exploration and exploitation in optimization-based protein design. The essential point is that Thompson sampling allows us to inject a controllable amount of *model-based* diversity into generation in a way that reflects data-driven uncertainty about predicted fitness. Being able to tunably trade-off exploration and exploitation is a key component of many high-performing Bayesian optimization algorithms, and so it is attractive to have a predictive model like LOCK GP that can provide high-quality uncertainty estimates—see e.g. Fig. 3 and Table 22—that can be leveraged in design.

A.7. CLOCK Kernel Interpretation

See Fig. 9 for a visual representation of CLOCK correlations. The top row illustrate that the CLOCK kernel has learned to prefer a substitution to proline in loops rather than secondary structure elements such as helices and sheets. The examples in the second row illustrate that the CLOCK kernel has learned to prefer a substitution to arginine on the surface of the protein rather than in the interior of the protein or at positions pointing inwards. The first structure in the final row illustrates that the CLOCK kernel has learned to prefer substitutions to glycine at the C-terminal region of alpha helices over the interior of alpha helices (Richardson & Richardson, 1988). The second and third structures illustrate the general preference for glycine in loops over beta sheets and alpha helices.

We also note that our structure-conditioned substitution matrices can be seen as an ML-driven descendent of the ‘environment-specific substitution tables’ (ESSTs) derived in Overington et al. (1992)—see also Koshi & Goldstein (1995). While in our case substitution matrices are regressed on continuous-valued embeddings $\mathbf{h}_\ell$, ESSTs are defined for a small number of manually-chosen semantic categories derived from e.g. secondary structure or solvent accessibility. The general motivation, however, is similar.

A.8. CLOCK Training Objective

Take the standard GP marginal log likelihood:

$$\log p(\mathbf{t}|\mathbf{X}) = \log \mathcal{N}(\mathbf{t}|\mathbf{0}, K_{\mathbf{X}\mathbf{X}} + \sigma_n^2 \mathbf{1}_N) \quad (28)$$

²⁶An alternative procedure would be to introduce hyperpriors on the kernel hyperparameters and do MCMC inference over the kernel hyperparameters. One could then sample hyperparameters from the approximate posterior and define GP posterior mean predictors conditioned on different hyperparameter samples. While this procedure would be perfectly viable, for simplicity we adopt the noise-modulating procedure instead. In either case the non-parametric nature of GPs makes exact Thompson sampling computationally intractable: we simply cannot sample posterior GP function values across the entirety of the sequence space in our optimization problem, since it has cardinality 20^{10} .

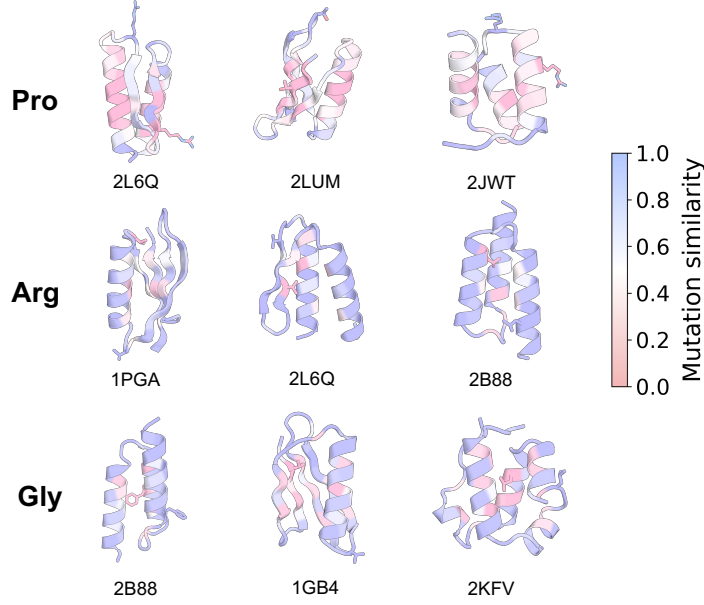


Figure 9. Structure-conditioned amino acid correlations learned by the CLOCK kernel. We show representative structures from Tsuboyama et al. (2023), coloring the wild-type residue at each position by its CLOCK correlation $C_{\ell a_{ref}}$ (Eqn. 14) to one of three reference amino acids (Pro, Arg, Gly; rows). Each structure highlights a pair of sites (namely those with sidechains pictured) with the *same* amino-acid identity but sharply different kernel correlation to the reference amino acid, illustrating how the learned mapping $\mathbf{W}$ (Eqn. 15) encodes amino-acid preferences that reflect local structure. Blue indicates substitutions judged similar to the reference residue; red denotes dissimilar, penalized substitutions. The PDB ID associated with each fitness landscape is provided directly under the structure of the corresponding protein. See Sec. A.7 for discussion.

and write

$$K_{\mathbf{X}\mathbf{X}} + \sigma_n^2 \mathbf{1}_N = \sigma_f^2 \left(\hat{K}_{\mathbf{X}\mathbf{X}} + \hat{\sigma}_n^2 \mathbf{1}_N \right) \quad (29)$$

i.e. we factor out the overall kernel scale σ_f^2 and introduce the inverse signal-to-noise ratio $\hat{\sigma}_n^2 = \sigma_n^2 / \sigma_f^2$. The MLE estimator for σ_f^2 can be computed in closed-form:

$$\hat{\sigma}_f^2 = \frac{1}{N} \mathbf{t}^T \left(\hat{K}_{\mathbf{X}\mathbf{X}} + \hat{\sigma}_n^2 \mathbf{1}_N \right)^{-1} \mathbf{t} \quad (30)$$

If this is plugged into Eqn. 28 and we drop irrelevant constants, we obtain

$$\log p(\mathbf{t}|\mathbf{X}) = -\frac{N}{2} \log (2\pi \hat{\sigma}_f^2) - \frac{1}{2} \log \left| \hat{K}_{\mathbf{X}\mathbf{X}} + \hat{\sigma}_n^2 \mathbf{1}_N \right| - \frac{N}{2} \quad (31)$$

$$\Rightarrow -\frac{N}{2} \log \hat{\sigma}_f^2 - \frac{1}{2} \log \left| \hat{K}_{\mathbf{X}\mathbf{X}} + \hat{\sigma}_n^2 \mathbf{1}_N \right| \quad (32)$$

in which we have effectively reduced the total number of kernel hyperparameters by one by eliminating σ_f . We use this ‘concentrated’ log likelihood as the training objective for CLOCK. This is very convenient in our context, since it obviates the need to learn the overall kernel scale for each landscape. Instead we can focus on learning the signal-to-noise ratio. Empirically on the data from Tsuboyama et al. (2023) we find that we can get away with learning a global signal-to-noise ratio that is shared by all landscapes. It’s plausible that this ansatz may be less effective for other datasets, but we generically expect this approach to work well in many scenarios, since we do not need the kernel scale and the noise scale to be perfectly specified in order to extract useful signal from the data. In any case learning landscape-specific signal-to-noise ratios is easy, and we find equally good performance when we do so for the Tsuboyama et al. (2023) dataset.

We note that the approach of profiling out σ_f has been adopted by many authors in the literature, see e.g. Park & Baek (2001); Moore et al. (2016); Ober et al. (2021). Indeed in Moore et al. (2016) the authors go a step further and place a prior on σ_f , in which case the overall kernel scale can be integrated out analytically.

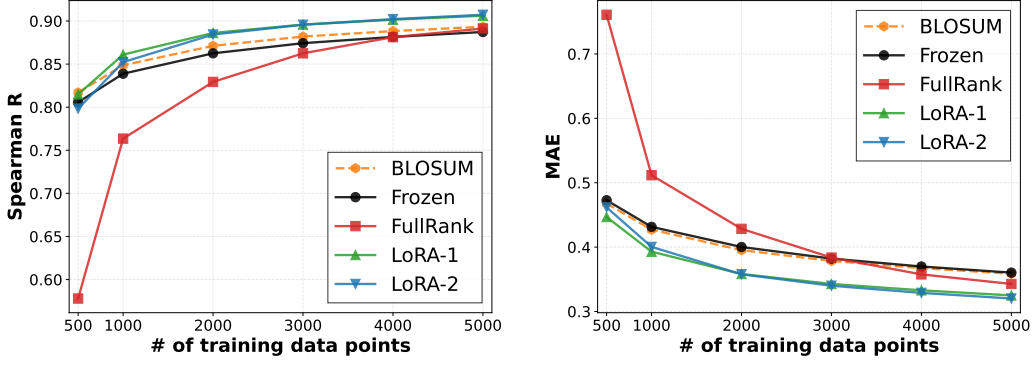


Figure 10. We compare several GPs trained on the AAV dataset from Sinai et al. (2021). We plot both Spearman R (left) and MAE (right); metrics are averaged across 10 i.i.d. train/test splits. All GPs are CLOCK-GPs except for the BLOSUM GP, which uses the same correlation matrix at each position. All CLOCK-GPs use a $\mathbf{W}$ tensor (see Eqn. 15) that is pre-trained on thermostability data (see Sec. 5.4). All CLOCK-GPs apart from Frozen are then fine-tuned on the AAV data. For $N \gtrsim 1000$ the LoRA-fine-tuned CLOCK-GPs outperform the BLOSUM GP, suggesting performance-enhancing transfer learning from the thermostability data.

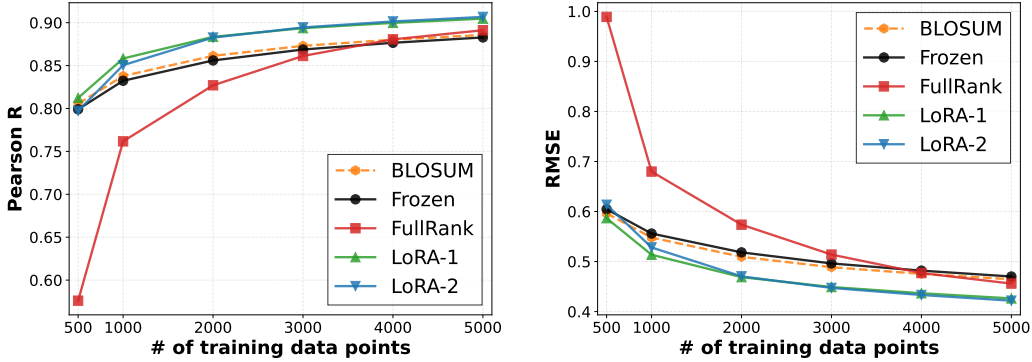


Figure 11. We compare several GPs trained on the AAV dataset from Sinai et al. (2021). We plot both Pearson R (left) and RMSE (right); metrics are averaged across 10 i.i.d. train/test splits. All GPs are CLOCK-GPs except for the BLOSUM GP, which uses the same correlation matrix at each position. All CLOCK-GPs use a $\mathbf{W}$ tensor (see Eqn. 15) that is pre-trained on thermostability data (see Sec. 5.4). All CLOCK-GPs apart from Frozen are then fine-tuned on the AAV data. For $N \gtrsim 1000$ the LoRA-fine-tuned CLOCK-GPs outperform the BLOSUM GP, suggesting performance-enhancing transfer learning from the thermostability data.

A.9. Proof-of-concept CLOCK Fine-tuning Experiment

We investigate the basic viability of fine-tuning the $\mathbf{W}$ tensor in CLOCK (see Eqn. 15) and adapting it to new landscapes. We would expect the most promise for such an approach in one of two scenarios: either i) the new landscape is for a property that is closely related to the property that $\mathbf{W}$ is pre-trained on; or ii) we can jointly fine-tune $\mathbf{W}$ on a considerable number of closely related landscapes. For simplicity, here we choose to focus on an a priori more difficult scenario in which the property in the new landscape is not particularly closely related to the property used in pre-training and in which we only have a single new landscape. In particular we focus on the AAV dataset from Sinai et al. (2021), where the property is a measure of capsid viability. We note that the length of the protein in this dataset is $L = 735$, whereas for the thermostability data $L \sim 50$.

To fine-tune $\mathbf{W}$ we simply optimize the vanilla GP training objective, i.e. the marginal log likelihood (see Eqn. 2). We use the Adam optimizer with an initial learning rate of 0.01. We train for 1500 steps and decimate the learning rate after 500 and 1000 steps. For simplicity in all cases we consider GPs with a non-linear kernel $k_{\text{nl}}^{\text{LOCK}}$ (see Eqn. 12) together with a learned global exponent (in addition to the noise scale σ_n and the overall kernel scale). We consider 5 different methods. In BLOSUM we utilize a BLOSUM50 correlation matrix at all positions. In Frozen we use $\mathbf{W}$ obtained from the thermostability pre-training and leave $\mathbf{W}$ fixed during training. In FullRank we initialize with $\mathbf{W}$ obtained from the thermostability pre-training and train $\mathbf{W}$ jointly with the kernel hyperparameters. In LoRA- k we use $\mathbf{W}$ obtained from the thermostability pre-training and train a rank- k LoRA adapter (Hu et al., 2022) jointly with the kernel hyperparameters (so

that $\mathbf{W}$ remains fixed).²⁷ These parameter-efficient adapters have 548 (for $k = 1$) and 1096 (for $k = 2$) trainable parameters, much reducing the risk of overfitting. For the results see Fig. 10-11.

As we might expect, FullRank performs poorly unless there is sufficient training data from the AAV landscape, since it is easy to overfit the $\sim 54k$ parameters in $\mathbf{W}$. Frozen performs quite well—and comparably to BLOSUM—which is perhaps surprising, since thermostability is a priori quite different from capsid viability. For $N \gtrsim 1000$ the strongest performers are LoRA-1 and LoRA-2, suggesting performance-enhancing transfer learning from the thermostability data. While the performance improvement is relatively small, we emphasize that good performance on LoRA- k requires mapping previously unseen structural contexts $\mathbf{h}_{1:L}$ to correlation matrices that are adapted to the AAV landscape. For example this presumably requires ‘unlearning’ the idiosyncratic role played by proline in the thermostability data, see Fig. 18. Thus these empirical results demonstrate the basic viability of fine-tuning $\mathbf{W}$ to adapt to new landscapes.

A.10. Generalizing CLOCK

We highlight some of the general features of the CLOCK kernel introduced in Sec. 3.6 and describe how it can be readily generalized to other kernel constructions. In more detail CLOCK can be understood as a special case of the following more general framework. For each landscape and corresponding reference structure $\mathcal{S}$ we either compute structure embeddings $\mathbf{h}_{1:L}(\mathcal{S})$ ²⁸ or sequence-and-structure embeddings $\mathbf{h}_{1:L}(\mathcal{S}, \mathbf{x})$. It is important to emphasize that doing inference with models that rely on structure embeddings will generally be much faster than those that rely on sequence-and-structure embeddings, since structure embeddings are shared across the entire landscape. We treat each of these two possibilities in turn.

Structure Embeddings We use a sequence-to-sequence model to map $\mathbf{h}_{1:L}$ to transformed embeddings $\tilde{\mathbf{h}}_{1:L}$ with some mapping $\mathcal{T}$. We then formulate a kernel that aggregates information from residue-level embeddings $\tilde{\mathbf{h}}_{1:L}$ in a way that naturally supports variable length sequences. In particular we can leverage kernel addition or kernel multiplication or some combination of the two:

- Each embedding $\tilde{\mathbf{h}}_\ell$ is mapped to a residue-specific subkernel k_ℓ —i.e. an $A \times A$ PSD matrix—and the full kernel is given by addition, i.e. $k = \sum_\ell k_\ell$
- Each embedding $\tilde{\mathbf{h}}_\ell$ is mapped to a residue-specific subkernel k_ℓ —i.e. an $A \times A$ PSD matrix—and the full kernel is given by multiplication, i.e. $k = \prod_\ell k_\ell$

Sequence-and-structure Embeddings We use a sequence-to-sequence model to map $\mathbf{h}_{1:L}$ to transformed embeddings $\tilde{\mathbf{h}}_{1:L}$ with some mapping $\mathcal{T}$.²⁹ We treat $\tilde{\mathbf{h}}_{1:L}$ as features and feed them into any kernel that takes euclidean features as input,³⁰ e.g. an RBF or Tanimoto kernel.³¹ That is in the case of sequence-and-structure embeddings the essential aggregation step happens at the feature level instead of at the kernel level.

Kernel Learning In both cases to learn the mapping $\mathcal{T}$ as well as any other parameters that feed into the kernel construction, we use gradient-based methods to optimize the concentrated form of the standard GP training objective in which the overall kernel scale is ‘profiled’ out. See Sec. A.8 and Sec. A.16.2.

Discussion CLOCK is a special case of this broader framework in which the sequence-to-sequence model $\mathcal{T}$ is given by applying the $\mathbf{W}$ matrix in Eqn. 15 residue-by-residue to obtain correlation matrices $\mathbf{C}_{1:L}$ using Eqn. 14. These correlation matrices are then used to define both linear and non-linear subkernels as in Eqn. 11 and Eqn. 12. These subkernels can then be combined as in LOCK, i.e. as in Eqn. 13, which utilizes both kernel addition and kernel multiplication to achieve

²⁷While $\mathbf{W}$ is most naturally viewed as a tensor of shape $A \times A - 1 \times D_{\text{emb}}$, here we conceptualize it as a two-dimensional tensor of shape $A(A - 1) \times D_{\text{emb}}$ when applying LoRA. Of course other low-rank parameterizations are also possible.

²⁸Alternatively, we compute structure-agnostic embeddings $\mathbf{h}_{1:L}(\mathbf{x}_{\text{ref}})$. The essential point is that we get a single set of embeddings $\mathbf{h}_{1:L}$ that is shared across the entire landscape.

²⁹In the case of sequence-and-structure embeddings we note that $\mathcal{T}$ can also depend explicitly on $\mathbf{x}$.

³⁰In more detail suppose we have features $\mathbf{z}_\ell \in \mathbb{R}^D$ where ℓ ranges across positions and where each positional feature is of dimension D . We can flatten $\{\mathbf{z}_\ell\}$ to get a vector $\mathbf{Z}$ of length LD and provide $\mathbf{Z}$ as an input to a, say, Tanimoto kernel (Tripp et al., 2023). Alternatively, we can define L Tanimoto kernels k_ℓ , where each subkernel takes features $\mathbf{z}_\ell$ as input; the complete kernel is then formed via kernel addition or multiplication.

³¹Indeed we have experimented with both of these constructions on the thermostability data from Tsuboyama et al. (2023). Both kernels resulted in good predictive models, but we found that CLOCK performed better at larger training sizes.

local linearity.³² Alternatively we can discard the linear kernels and use only the non-linear kernels or vice versa; indeed we explicitly benchmark against these constructions in Sec. A.5 in the non-structure-conditioned case.

This general framework makes it evident that there is considerable room for defining additional CLOCK-like variants. For example, instead of using a mapping $\mathcal{T}$ that is applied independently at each residue, we could use a true sequence-to-sequence model like a RNN or Transformer.

A.11. On the scalability of (C)LOCK GPs

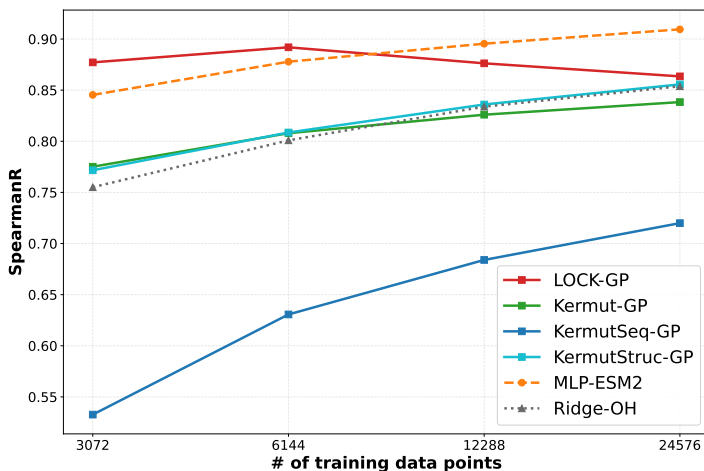


Figure 12. We report Spearman R for models trained on the amacGFP dataset from Gonzalez Somermeyer et al. (2022) as a function of the number of training data points. Spearman R is averaged across three replicates.

Apart from the binary classification experiment in Sec. 5.6, the GP experiments in Sec. 5 focus on the regression setting with small-to-moderate sized datasets, a regime in which exact inference is viable. In this section we briefly touch upon some of the possibilities for scaling LOCK GPs to much larger datasets.

For regression, using the conjugate gradient (CG) method as in Wang et al. (2019) formally reduces training from $\mathcal{O}(N^3)$ to $\mathcal{O}(N^2)$ and can result in large speed-ups in practice, making $N \sim 10^5 - 10^6$ viable on commercial GPUs. For the LOCK kernel in particular, this approach can be made significantly more memory efficient by utilizing kernel-vector multiplies that are implemented in a streaming fashion.³³

Beyond CG, there are many powerful approximate inference methods that leverage so-called inducing points (Hensman et al., 2013). Many of these methods are mini-batch-friendly so that they can be pushed to arbitrarily large N in principle. We provided an example of this kind of approach in the binary classification experiment in Sec. 5.6 and Fig. 7, which considered up to $N = 12288$ training data points and which uses the inference algorithm from Wenzel et al. (2019). For the regression setting we suggest that the method from Cazelles & Tobar (2026) is particularly well-suited for LOCK GPs, since it avoids the introduction of inducing points.³⁴ While we do not report any results with this method here, we find that it can work well in practice.

As a demonstration of the scalability of LOCK GPs in the regression setting, we report the results of an additional experiment that mirrors the binary classification experiment in Sec. 5.6. In particular we use CG to train four GPs (and two additional baselines) on the amacGFP dataset from Gonzalez Somermeyer et al. (2022) on up to 24576 training data points, see Fig. 12. This is done using a NVIDIA A10G GPU, which has 24 GB of memory, i.e. only 12.5% of the memory available on a modern NVIDIA B200 GPU.

³²An interesting feature of CLOCK is that we learn linear kernels at training time, but at inference time we can deploy both linear and non-linear kernels. This is one of the advantages of unifying around correlation matrices, which can be readily deployed in various ways. Since linear kernel learning is straightforward, this flexibility contributes to the speed and robustness of CLOCK training. See Sec. A.16.2.

³³See <https://github.com/getkeops/keops>.

³⁴These methods work by utilizing M (with $M \ll N$) inducing points that provide a ‘compressed’ view of the N training inputs. As such they can be a poor fit for discrete input spaces. This is for at least two reasons: i) the discrete nature of the input space makes it challenging to optimize the inducing points; and ii) the compression might be excessively lossy (consider e.g. rare mutations).

Finally, we emphasize that these general considerations also translate to CLOCK. In more detail, we note that in CLOCK (see Sec. A.16.2 for training details) we randomly subsample within landscapes during training of the matrix $\mathbf{W}$ (see Eqn. 15) so that each landscape has 512 training data points. As such during training of $\mathbf{W}$, the cubic cost of exact GP inference is not an issue. When deploying CLOCK on a new (potentially large) landscape, the various approaches described above—for example CG—can be used to fit any kernel hyperparameters such as the overall kernel scale to the landscape at hand. As such CLOCK can be readily deployed in the large N setting.

A.12. Inventory of Substitution Matrices

See Table 7.

Substitution Matrix	ID	Substitution Matrix	ID	Substitution Matrix	ID
BLOSUM100	✓	BLOSUM85	✓	PAM180	
BLOSUM30		BLOSUM90	✓	PAM190	
BLOSUM35		BLOSUMN	✓	PAM20	
BLOSUM40	✓	CorBLOSUM49_5.0	✓	PAM200	
BLOSUM45	✓	CorBLOSUM57_13p	✓	PAM210	
BLOSUM50	✓	CorBLOSUM57_14.3	✓	PAM250	
BLOSUM50_13p	✓	CorBLOSUM61_5.0	✓	PAM30	
BLOSUM50_14.3	✓	CorBLOSUM66_13p	✓	PAM40	
BLOSUM50_5.0	✓	CorBLOSUM67_14.3	✓	PAM50	
BLOSUM55	✓	DAYHOFF		PAM60	
BLOSUM60	✓	PAM10		PAM70	
BLOSUM62	✓	PAM100		PAM80	
BLOSUM62_13p	✓	PAM110		PAM90	
BLOSUM62_14.3	✓	PAM120		RBLOSUM52_5.0	✓
BLOSUM62_5.0	✓	PAM130		RBLOSUM59_13p	✓
BLOSUM65	✓	PAM140		RBLOSUM59_14.3	✓
BLOSUM70	✓	PAM150		RBLOSUM64_5.0	✓
BLOSUM75	✓	PAM160		RBLOSUM69_13p	✓
BLOSUM80	✓	PAM170		RBLOSUM69_14.3	✓

Table 7. We provide an inventory of 57 substitution matrices available from Biotite (Kunzmann et al., 2023), focusing on the 20×20 principle submatrices that correspond to the 20 canonical amino acids. We list only those matrices that are PSD. For each matrix we indicate whether it is infinitely divisible (ID). We note that all listed BLOSUM matrices are ID except for BLOSUM30 and BLOSUM35.

A.13. Kermut Kernel

The Kermut kernel has two components, a sequence kernel $k_{\text{seq}}(\mathbf{x}, \mathbf{y})$ and a structure kernel $k_{\text{struct}}(\mathbf{x}, \mathbf{y})$. The former is an isotropic RBF kernel defined on mean-pooled ESM-2 embeddings. Let $M(\mathbf{x}) \subseteq \{1, \dots, L\}$ denote the mutated positions of sequence $\mathbf{x}$ w.r.t. the user-specified reference (e.g. wild-type) sequence $\mathbf{x}_{\text{ref}}$. Then the structure kernel is a set kernel defined over pairs of mutations:

$$k_{\text{struct}}(\mathbf{x}, \mathbf{y}) = \sum_{i \in M(\mathbf{x})} \sum_{j \in M(\mathbf{y})} k'_{\text{struct}}(\mathbf{x}_i, \mathbf{y}_j) \quad (33)$$

where $k'_{\text{struct}}(\mathbf{x}_i, \mathbf{y}_j)$ is a base kernel that acts on pairs of residues. In Kermut $k'_{\text{struct}}(\mathbf{x}_i, \mathbf{y}_j)$ is a product kernel made up of three subkernels and leverages ProteinMPNN inverse folding probabilities and 3d structure coordinates. In addition Kermut GP utilizes an ESM-2-derived zero-shot mean function. For more details see Groth et al. (2024).

Here we would like to draw attention to some of the counterintuitive properties of $k_{\text{struct}}(\mathbf{x}, \mathbf{y})$:

- **Combinatorial explosion:** k_{struct} scales quadratically as the number of mutations increases, which tends to inflate similarities for high-order mutants.
- **Variable behavior as we go further away from the reference sequence:** Choose a sequence $\mathbf{y}$ that is everywhere mutated away from the reference sequence, i.e. the Hamming distance is given by $d_{\text{H}}(\mathbf{x}_{\text{ref}}, \mathbf{y}) = L$. Now let $\{\mathbf{x}^{(0)} \equiv \mathbf{x}_{\text{ref}}, \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(L)} \equiv \mathbf{y}\}$ be a series of sequences that interpolate from $\mathbf{x}_{\text{ref}}$ to $\mathbf{y}$ with the property that $d_{\text{H}}(\mathbf{x}_{\text{ref}}, \mathbf{x}^{(\ell)}) = \ell$. In other words $\mathbf{x}^{(\ell)}$ is like $\mathbf{x}_{\text{ref}}$ except that it has adopted ℓ mutations from $\mathbf{y}$. Then under the assumption that $k'_{\text{struct}}(\mathbf{x}_i^{(\ell)}, \mathbf{x}_j^{(\ell')})$ is always strictly greater than zero—which is the case for Kermut—we find that

$$k_{\text{struct}}(\mathbf{x}^{(0)}, \mathbf{x}^{(1)}) < k_{\text{struct}}(\mathbf{x}^{(1)}, \mathbf{x}^{(2)}) < \dots < k_{\text{struct}}(\mathbf{x}^{(L-1)}, \mathbf{x}^{(L)}) \quad (34)$$

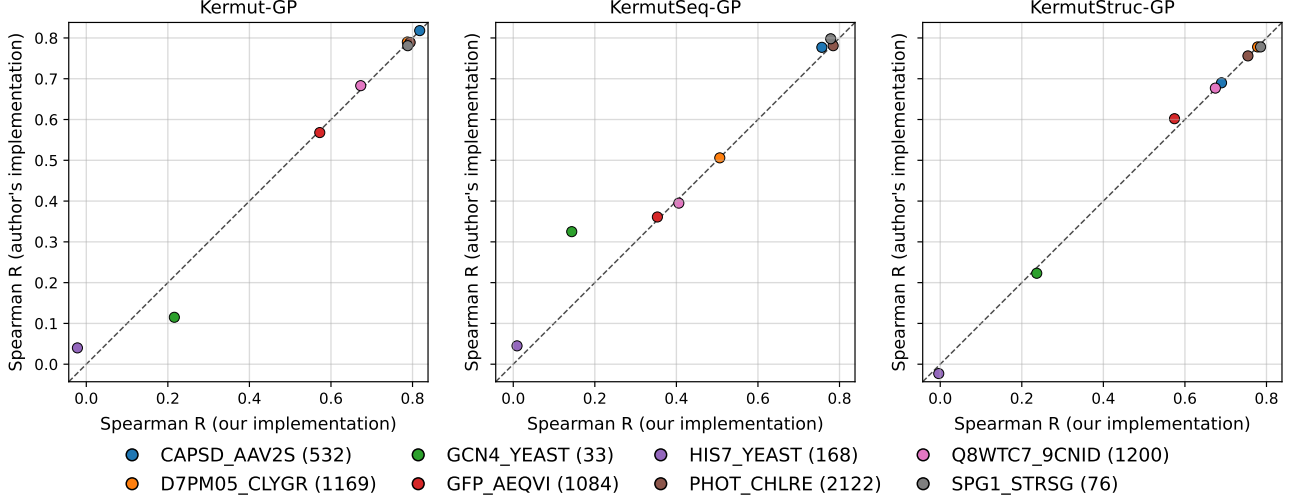


Figure 13. We depict Spearman R metrics obtained for 8 ProteinGym datasets for all three Kermut variants, comparing our implementation to the results provided by Groth et al. (2024). The number in parentheses after each landscape indicates the total number of data points in the given landscape.

since the sum that defines $k_{\text{struct}}(\mathbf{x}^{(\ell)}, \mathbf{x}^{(\ell+1)})$ contains all the terms that enter into $k_{\text{struct}}(\mathbf{x}^{(\ell-1)}, \mathbf{x}^{(\ell)})$ plus new (positive) terms. This means, for example, that $k_{\text{struct}}(\mathbf{x}^{(L-1)}, \mathbf{x}^{(L)})$ is generally going to be *much* larger than $k_{\text{struct}}(\mathbf{x}^{(0)}, \mathbf{x}^{(1)})$, even though both similarities are between pairs of sequences that differ by a single mutation. This highly non-intuitive behavior would seem to suggest that, while Kermut may perform well for datasets where most sequences are in the vicinity of the reference sequence, it may exhibit unusual behavior for datasets with lots of higher-order mutants.³⁵ Indeed this may be a contributing factor to the relatively poor performance of Kermut in the extrapolation setting, see e.g. Table 1.

- **Wild-type degeneracy:** $k_{\text{struct}}(\mathbf{x}_{\text{ref}}, \mathbf{y}) = 0$ for all sequences $\mathbf{y}$. One consequence is that—in the absence of the sequence kernel—the posterior mean prediction at $\mathbf{x}_{\text{ref}}$ is given by the prior mean (cf. Eqn. 3), i.e. is largely uninformed by other sequences.³⁶ Moreover posterior (function) uncertainty **collapses to 0** at $\mathbf{x}_{\text{ref}}$ (see Eqn. 4). By contrast the LOCK-GP functions as a *smoother*, just like GPs with familiar kernels like the RBF kernel.³⁷ That is predictions for an observed sequence $\mathbf{x}$ are influenced not only by the observed value t at $\mathbf{x}$ but also by the observed function values of nearby sequences (as determined by the kernel), and the function uncertainty is generically non-zero everywhere.

Implementation We use our own implementation of Kermut that is lightly adapted from the authors’ code at <https://github.com/petergroth/kermut>. Since Kermut is one of our strongest baseline methods, we take special care to verify that our implementation is equivalent to the authors’. To do so we perform a head-to-head comparison of the two implementations, comparing Spearman R metrics obtained with our implementation against metrics provided by the authors. We do so on 8 ProteinGym datasets, using the exact same train/test splits and metric computations. See Fig. 13 for the results. We find very good agreement, with the largest difference appearing for the dataset with only 33 data points, where we find more variability from training run to training run. In addition we checked our computation of ESM-2 embeddings as well as masked marginal zero-shot pseudolikelihoods against those provided by the authors and found excellent agreement.

Discussion of Kermut Log Likelihoods We briefly discuss the Kermut uncertainty results summarized in Table 20 and Table 22. If we define a “catastrophic NLL” as one larger than 100, we find that LOCK-GP and Tanimoto-GP do not exhibit catastrophic NLLs for any datasets, while all three Kermut variants demonstrate catastrophic NLLs on at least one dataset. Moreover, we find examples of this in all three evaluation regimes: cross-validation, unseen mutations, and extrapolation.

³⁵As such this behavior is essentially a special case of the ‘combinatorial explosion’ mentioned above.

³⁶Other sequences enter only insofar as they influence learned parameters in the prior mean. In this case there are two such parameters, a shift and a scale, which are used to adjust the zero-shot pseudo-likelihood from ESM-2.

³⁷Here we assume noisy observations like in Eqn. 1.

Table 8. We summarize the datasets used in experiments in Sec. 5.2 and Sec. 5.3. We selected datasets that are sufficiently large and have considerable sequence diversity, including an abundance of higher-order mutants (see Table 9). Datasets range in size from 1,812 sequences to 496,137 sequences. The 9 datasets from ProteinGym (Notin et al., 2023a) are referred to by their ProteinGym IDs (i.e. from D7PM05_CLYGR_Somermeyer_2022 to GCN4_YEAST_Staller_2018). Assay types are categorized by their corresponding ProteinGym category: activity, binding, organismal fitness, stability, and expression; note that we did not include any expression datasets.

Name	N	Ref. structure	Readout type	Structure in complex	Reference
D7PM05_CLYGR_Somermeyer_2022	24515	Predicted	Activity	No	Gonzalez Somermeyer et al. (2022)
Q6WV12_9MAXI_Somermeyer_2022	31401	Predicted	Activity	No	Gonzalez Somermeyer et al. (2022)
Q8WTC7_9CNID_Somermeyer_2022	33510	Predicted	Activity	No	Gonzalez Somermeyer et al. (2022)
GFP_AEQVI_Sarkisyan_2016	51714	Predicted	Activity	No	Sarkisyan et al. (2016)
CAPSD_AAV2S_Sinai_2021	42328	Predicted	OrganismalFitness	No	Sinai et al. (2021)
F7YBW8_MESOW_Ding_2023	7922	Predicted	OrganismalFitness	No	Ding et al. (2024)
HIS7_YEAST_Pokusaeva_2019	496137	Predicted	OrganismalFitness	No	Pokusaeva et al. (2019)
PHOT_CHLRE_Chen_2023	167529	Predicted	Activity	No	Chen et al. (2023)
GCN4_YEAST_Staller_2018	2638	Predicted	Activity	No	Staller et al. (2018)
CH65 - MA90	65530	Experimental	Binding	Yes	Phillips et al. (2023)
CH65 - SI06	64619	Experimental	Binding	Yes	Phillips et al. (2023)
CH65 - G189E	63840	Experimental	Binding	Yes	Phillips et al. (2023)
CR9114 - H1	65093	Experimental	Binding	Yes	Shanker et al. (2024)
CR9114 - H3	65093	Experimental	Binding	Yes	Shanker et al. (2024)
CR6261 - H1	1812	Experimental	Binding	Yes	Shanker et al. (2024)
CR6261 - H9	1812	Experimental	Binding	Yes	Shanker et al. (2024)
Moulana Regn10987	30479	Experimental	Binding	Yes	Moulana et al. (2023)
Moulana CB6	32112	Experimental	Binding	Yes	Moulana et al. (2023)
Moulana Cov555	29892	Experimental	Binding	Yes	Moulana et al. (2023)
Tsuboyama - 2B88	2937	Predicted	Stability	No	Tsuboyama et al. (2023)
Tsuboyama - 2HBB	4494	Predicted	Stability	No	Tsuboyama et al. (2023)

These catastrophic NLLs result from very narrow posterior predictive distributions, which appears to be driven by several distinct phenomena. In some cases, the lengthscale of the RBF kernel collapses; in other cases, the observation noise σ_n becomes anomalously small. It is possible that this undesirable behavior could be rescued by modifying the hyperpriors and/or hyperparameter initialization used by Kermut. However, as we can see from Table 20, the poor average NLL for the three Kermut variants is not solely due to a few outliers. Indeed LOCK-GP has the lowest NLL on 17/21 landscapes in the cross-validation regime with 192 training data points. It also has the lowest NLL on 12/21 landscapes in the extrapolation regime with 128 training data points as well as the lowest NLL on 13/21 landscapes in the extrapolation regime with 96 training data points.

A.14. Description of Datasets and Evaluation Strategies

See Table 8 and Table 9 for details on each dataset, including summary statistics like sequence length.

Dataset Pre-processing We minimally processed each ProteinGym dataset (Notin et al., 2023a), downloaded as DMS substitution datasets from the ProteinGym website directly in August 2025. For each dataset, we removed residues from either end of the associated predicted structure or reference sequence that were not present in the other to ensure their sequence lengths matched. We additionally log10-normalized DMS scores for the three Somermeyer et al. datasets (D7PM05_CLYGR_Somermeyer_2022, Q6WV12_9MAXI_Somermeyer_2022, and Q8WTC7_9CNID_Somermeyer_2022; (Gonzalez Somermeyer et al., 2022)). For all other ProteinGym datasets, we performed no pre-processing on DMS scores.

We processed non-ProteinGym datasets separately. For the three Phillips et al. datasets (CH65 - MA90, CH65 - SI06, and CH65 - G189E), we associated reported negative log KD scores with the reference 5UGY PDB structure (Phillips et al., 2023). For the three Moulana et al. datasets (Moulana Regn10987, Moulana CB6, and Moulana Cov555), we associated reported log KD scores with their respective reference PDB structure - 6XDG for Regn10987, 7C01 for CB6, and 7KMG for COV555 (Moulana et al., 2023). To define full-length reference sequences for these datasets, we added constant residues to each set of originally-reported wildtype residues based on the constant residues in each respective structure’s sequence. For the two Tsuboyama et al. datasets (Tsuboyama et al., 2023), we associated each dataset with its respective AlphaFold-predicted structure after taking the mean stability score across duplicated sequences.

Lastly, for the four Shanker et al. datasets (CR9114 - H1, CR9114 - H3, CR6261 - H1, CR6261 - H9), we associated each dataset with their respective PDB structure - 3GBN for CR6261 datasets and 4FQI for CR9114 datasets (Shanker et al., 2024). While these are experimental structures, it is important to note that the measured experimental values for these datasets do not precisely match the subtype of the complexed influenza virus measured in the structure for all experiments—please

Table 9. We provide detailed summary statistics for the 21 datasets used for the experiments in Sec. 5.2 and Sec. 5.3. Each dataset has a maximum Hamming distance from reference (d_H) of at least 6, an average d_H of at least 2.7, and at least 10 variable positions.

Name	Avg. d_H	Max d_H	Total sequence length	# of variable positions
D7PM05_CLYGR_Somermeyer_2022	3.0	23	235	234
Q6WV12_9MAXI_Somermeyer_2022	2.7	13	222	221
Q8WTC7_9CNID_Somermeyer_2022	3.1	43	238	237
GFP_AEQVI_Sarkisyan_2016	3.9	15	238	233
CAPSD_AAV2S_Sinai_2021	4.7	28	735	28
F7YBW8_MESOW_Ding_2023	5.4	10	93	10
HIS7_YEAST_Pokusaeva_2019	7.0	28	220	171
PHOT_CHLRE_Chen_2023	8.0	15	118	118
GCN4_YEAST_Staller_2018	17.1	44	281	44
CH65 - MA90	8.0	16	760	16
CH65 - SI06	8.0	16	760	16
CH65 - G189E	8.0	16	760	16
CR9114 - H1	8.0	16	730	16
CR9114 - H3	8.0	16	730	16
CR6261 - H1	5.5	11	731	11
CR6261 - H9	5.5	11	731	11
Moulana Regn10987	7.5	15	1043	15
Moulana CB6	7.5	15	1256	15
Moulana Cov555	7.6	15	1255	15
Tsuboyama - 2B88	3.6	6	54	54
Tsuboyama - 2HBB	3.3	7	48	48

refer to [Shanker et al. \(2024\)](#) for more information. For the reference 3GBN structure, we also removed the first residue of the sequence as it is unstructured. As above, we then added constant residues to each set of originally-reported wildtype residues based on the constant residues in each respective structure’s sequence to define full-length reference sequences. For the CR9114 and CR6261 datasets, respectively, we also removed sequences with any missing score in the associated pair of H1/H3 or H1/H9 scores.

Cross-validation We do 7-fold cross-validation. That is for each landscape we randomly partition the data into 7 disjoint and approximately-equally-sized subsets. We then define 7 train/test sets, where each test set is one of the 7 subsets and the training set is the remaining data points. This procedure is done once for each landscape. Then, to modulate the number of training data points, we do i.i.d. subsampling within each training set, while simultaneously keeping the test set fixed. To compute metrics we aggregate test predictions across all 7 test sets, making sure that predictions are computed in the same target space. We then compute predictive metrics like Spearman R or MAE on these aggregated predictions. In this manner we obtain a single metric for each landscape and for each given number of training data points. For ease of comparison of metrics across landscapes, we use the standard deviation of all target scores in the given landscape to normalize true and predicted scores as well as the predictive distribution when available. This additional normalization step has no effect on metrics like Spearman or Pearson R, but it ensures that scale-dependent metrics like MAE and CRPS are more readily comparable across landscapes. For $N = 48$ training data points we do three subsampling replicates to mitigate against random variability that is more prevalent for small training data sets;³⁸ for $N > 48$ we do a single replicate.

Unseen Mutations In order to investigate the ability of different models to generalize to unseen mutations, we devise train/test splits in which all test sequences have at least one mutation that is not present in the corresponding training set. Given the diverse nature of our 21 datasets, we utilize two distinct strategies to do so.

The first strategy is used for landscapes that have 20 or fewer variable positions and for which the majority amino acid at each position has a frequency of at least 50%. These happen to be the 10 antibody landscapes: i) 3 CH65 landscapes; ii) 4 CR landscapes; and iii) 3 Moulana landscapes. For these landscapes in each distinct train/test split we randomly select 4 distinct residues. The initial training set consists of all sequences that have the majority amino acid at the selected residues. The initial test set consists of all remaining sequences. We then subsample each training set down to exactly 96 data points and remove all sequences in the test set that do not have any unseen mutations, i.e. mutations not present in the training set. We define 3 distinct train/test splits for each landscape, ensuring that there is minimal overlap between the ‘selected’ residues between the three splits for each landscape. We also require that the standard deviation of the training targets is at least a tenth of the standard deviation of the targets across the entire landscape.

For the remaining landscapes we utilize a different strategy. First we identify “rare” mutations by ranking mutations by

³⁸For the GP ablation experiments in Sec. A.5 we do a single replicate.

their frequency and choosing those in the bottom decile. We then construct a training set with 96 data points by drawing sequences from a weighted distribution that heavily favors sequences with one or more rare mutations. The test set is then defined as all remaining sequences that have at least one unseen mutation, i.e. mutations not present in the training set. As above we define 3 distinct train/test splits for each landscape and require that the standard deviation of the training targets is at least a tenth of the standard deviation of the targets across the entire landscape.

With these choices each of the 21×3 test sets has between 1684 and 175,693 sequences. Metrics are computed for each train/test split and for each landscape we report averages across the three distinct splits.

Extrapolation For each landscape we determine the reference/wild-type sequence $\mathbf{x}_{\text{ref}}$. For ProteinGym datasets this is the `target_seq` field in the `DMS_substitutions.csv` file, while for the remaining datasets we take the reference sequence from the corresponding publication. For each variant $\mathbf{x}$ in a given landscape we compute its Hamming distance to the target, $d_H(\mathbf{x}, \mathbf{x}_{\text{ref}})$. We adaptively select a Hamming distance cutoff D for each landscape as the smallest integer satisfying $3 \leq D \leq 5$ such that at least 512 sequences have $d_H(\mathbf{x}, \mathbf{x}_{\text{ref}}) \leq D$ and at least 384 sequences have $d_H(\mathbf{x}, \mathbf{x}_{\text{ref}}) > D$. The initial training set is all sequences within a Hamming distance of D of the target sequence (i.e. we choose $\mathbf{x}$ such that $d_H(\mathbf{x}, \mathbf{x}_{\text{ref}}) \leq D$). The training set is then subsampled uniformly at random to the desired number of training data points (either 128 or 512). The test set is all sequences more than a Hamming distance of D from the target sequence (i.e. we choose $\mathbf{x}$ such that $d_H(\mathbf{x}, \mathbf{x}_{\text{ref}}) > D$) and is not subsampled. By construction all test sets have at least 384 sequences.

A.15. Speed and Memory Usage Benchmark

Training time			Training peak memory		
Model	$N = 1024$	$N = 16384$	Model	$N = 1024$	$N = 16384$
LOCK-GP	12.8 s	1.8 m	LOCK-GP	154 MB	11.0 GB
Kermut-GP	1.8 m	13.9 m	Kermut-GP	5.8 GB	16.6 GB
MLP-ESM2-8M	40.7 s	5.8 m	MLP-ESM2-8M	146 MB	358 MB
MLP-ESM2-650M	1.7 m	11.3 m	MLP-ESM2-650M	3.2 GB	3.4 GB
MLP-ESM2-LastLayer	2.7 m	41.1 m	MLP-ESM2-LastLayer	215 MB	361 MB
MLP-SaProt-LastLayer	21.0 m	249.3 m	MLP-SaProt-LastLayer	425 MB	562 MB

Inference time			Inference peak memory		
Model	$N = 1024$	$N = 16384$	Model	$N = 1024$	$N = 16384$
LOCK-GP	0.2 s	0.7 s	LOCK-GP	83 MB	5.2 GB
Kermut-GP	40.3 s	43.1 s	Kermut-GP	5.8 GB	14.3 GB
MLP-ESM2-8M	1.5 s	1.5 s	MLP-ESM2-8M	141 MB	152 MB
MLP-ESM2-650M	20.6 s	19.7 s	MLP-ESM2-650M	3.2 GB	3.3 GB
MLP-ESM2-LastLayer	1.1 s	1.1 s	MLP-ESM2-LastLayer	1.6 GB	1.7 GB
MLP-SaProt-LastLayer	4.0 s	4.0 s	MLP-SaProt-LastLayer	846 MB	976 MB

Table 10. We report training and inference times as well as the corresponding peak GPU memory usage for six models on the Chen et al. (2023) dataset. We vary the number of training points as $N \in \{1024, 16384\}$ and keep the inference workload fixed at 1024 predictions. Results are obtained using a NVIDIA A10G GPU.

We benchmark the compute time³⁹ and peak memory usage of six representative models; see Table 10 for the results. We find that LOCK-GP training is $\sim 3 - 140\times$ faster than the other five models, all of which make use of at least one foundation model. Similarly LOCK-GP inference is $\sim 2 - 80\times$ faster than the other models considered. By contrast the picture for peak memory usage is more complex, and in any case this quantity can depend on the mini-batch size used for computing foundation model embeddings—this applies to both training and inference—as well as other implementation details. Notably peak memory usage for GP models becomes substantial for $N \gtrsim 10^4$. While this can be ameliorated by using inducing point methods or other approximate GP inference techniques (see Sec. A.11), kernel methods can be prone to significant memory usage unless care is taken to avoid realizing large kernel matrices. We note that we can estimate the inference time of other models in Table 21 by considering ESM-2 usage and comparing to Table 10. For example, the inference time of Ridge-ESM2-650M is roughly comparable to MLP-ESM2-650M, since computing ESM-2 embeddings is the dominant

³⁹Note that to benchmark inference compute time we skip any computations that can be cached and subsequently reused. For example, computing ProteinMPNN inverse folding logits for Kermut-GP need only be done once, since the reference structure $\mathcal{S}$ is assumed fixed. Similarly, for training we make sure to avoid unnecessary recomputation wherever possible.

computational cost.

We also note that a similar inference time hierarchy exists for the multi-task models benchmarked in Sec. 5.4. In particular CLOCK-GP is dramatically faster at inference time than CNN-EMB-OH, since the positional structure embeddings $\mathbf{h}_{1:L}$ used by CLOCK-GP can be computed once and cached, while for CNN-EMB-OH positional structure-and-sequence embeddings must be computed for each new test sequence.

A.16. Experimental Details

In our experiments we leverage Pytorch (Paszke et al., 2019), Gpytorch (Gardner et al., 2018), Botorch (Balandat et al., 2020), and Pytorch Lightning (Falcon & The PyTorch Lightning team, 2019). In particular, unless noted otherwise we use `fit_gpytorch_mll` from Botorch to fit GPs, which makes use of the quasi-Newton method L-BFGS under the hood.

A.16.1. LOCAL LEARNING EXPERIMENT

We describe the various baselines used in the experiment in Sec. 5.3. We also describe some additional baselines for which we did not report results in the main text but which appear in Table 21:

- | | | | |
|----------------------|--------------------------|------------------------|------------------------|
| • ESM2-650M-ZeroShot | • Ridge-ESM2-8M-MeanPool | • MLP-ESM2-8M-MeanPool | • MLP-OH |
| • BLOSUM50-ZeroShot | • Ridge-OH-ESM2-650M-Aug | • MLP-OH-ESM2-Aug | • ConFit |
| • Ridge-ESM2-650M | • MLP-ESM2-650M | • MLP-ESM2-8M-RandInit | • MLP-SaProt-LastLayer |

Tanimoto-GP We provide a brief description of the kernel in Gessner et al. (2024). Let $\log^\circ \mathbf{S} \in \mathbb{R}^{A \times A}$ be a BLOSUM substitution matrix in log-odds space. Compute its eigendecomposition $\log^\circ \mathbf{S} = \mathbf{U} \mathbf{D} \mathbf{U}^\top$. Since $\log^\circ \mathbf{S}$ is not PSD, we cannot take the square root of $\mathbf{D}$. In Gessner et al. (2024) the authors replace $\mathbf{D}$ with its entrywise absolute value $|\mathbf{D}|$ before taking the square root. We find slightly better performance (see Sec. A.5) by using elementwise clamping instead: $\mathbf{D}_+ = \max(\mathbf{D}, 0)$. For one-hot sequence $\mathbf{x} \in \mathbb{R}^{L \times A}$ we then encode $\mathbf{x}$ as $\phi(\mathbf{x}) = \mathbf{x} \mathbf{U} \sqrt{\mathbf{D}_+} \in \mathbb{R}^{L \times A}$. We then define the resulting Tanimoto kernel (Tripp et al., 2023) as

$$k_{\text{Tani}}(\mathbf{x}, \mathbf{x}') = \frac{\langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle_F}{\|\phi(\mathbf{x})\|_F^2 + \|\phi(\mathbf{x}')\|_F^2 - \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle_F} \quad (35)$$

where $\langle \cdot, \cdot \rangle_F$ is the Frobenius inner product. Gessner et al. (2024) does not specify which BLOSUM matrix is used; we use BLOSUM62. As shown in Sec. A.5, there is only weak dependence on the particular substitution matrix used.

Kermut-GP We follow the detailed model specification described in Groth et al. (2024). The authors’ released code initializes the two parameters that define the prior mean by sampling both the scalar weight and bias from $\mathcal{N}(0, 1)$. We instead sample the weight from $\mathcal{N}(0, 0.01)$ and initialize the bias to 0.

KermutSeq-GP Like Kermut-GP but without the structure kernel.

KermutStruc-GP Like Kermut-GP but without the RBF sequence kernel. See Sec. A.13 for an extended discussion of the structure kernel. As described in Groth et al. (2024), the structure kernel includes an overall kernel scale. The authors’ released code omits this scaling for the structure-only kernel. Following the paper, we include this learnable scale in our implementation.

Ridge-OH We use RidgeCV from sklearn on one-hot encoded sequences, which selects from 32 ridge parameters α_{ridge} that are geometrically evenly spaced between 10^{-4} and 10^4 (i.e. `numpy.logspace(-4, 4, 32)`).

Ridge-ESM2 We use RidgeCV from sklearn on (non-mean-pooled) features from ESM-2-8M. We extract features by selecting the variable positions from the last hidden layer, flattening into a vector, and then elementwise centering and unit-scaling. We use 32 ridge parameters α_{ridge} that are geometrically evenly spaced between 10^{-4} and 10^4 (i.e. `numpy.logspace(-4, 4, 32)`). In Table 21 we also reports results for Ridge-ESM2-650M.

MLP-ESM2-8M and MLP-ESM2-650M We train a single-layer neural network with 128 hidden units and a ReLU non-linearity on (non-mean-pooled) features from ESM-2. We extract features by selecting the variable positions from the last hidden layer, flattening into a vector, and then elementwise centering and unit-scaling. We use a dropout probability of 0.5 and a L2 regularizer $\lambda = 0.1$ (i.e. relatively strong weight decay). We train for 1000 epochs in batch mode using Adam with default settings (i.e. $1r = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$) and a linear learning-rate schedule that decays to 0 over 1000 epochs.

MLP-ESM2-LastLayer We use the same feature extraction pipeline as MLP-ESM2-8M but fine-tune the last layer of ESM-2-8M jointly with the MLP head. We use 32 hidden units, a dropout probability of 0.5, and a LeakyReLU non-linearity with negative slope 0.1. We train for 100 epochs using AdamW with a batch size of 24, gradient clipping with a maximum norm of 0.5, and a learning-rate schedule with 30 epochs of linear warmup (starting at a factor of 0.01) followed by cosine decay to a minimum of 10^{-6} . We use a peak learning rate of 10^{-4} for training ESM-2 parameters and 10^{-3} for the MLP head, with weight decay 0.1 (except for LayerNorm parameters, which have no weight decay); other AdamW settings are set to default values.

SigGLM-OH The model is of the form $f(\mathbf{x}) = a \sigma(\beta \cdot \mathbf{x} + b) + c$ where $\sigma(\cdot)$ is a logistic link function (i.e. `torch.sigmoid`), $a, b, c \in \mathbb{R}$ are scalars, $\beta \in \mathbb{R}^{L \times A}$ are coefficients, and $\mathbf{x}$ is a one-hot encoded sequence. We fit the model by minimizing a MSE loss function that also includes a L2 regularizer on β ; we use the Adam optimizer. The strength of the L2 regularizer is chosen via cross-validation.

ConFit ConFit (Contrastive Fitness Learning) fine-tunes a pre-trained PLM using a contrastive Bradley–Terry loss and parameter-efficient LoRA adapters (Zhao et al., 2024). Like (Zhao et al., 2024) we use an ensemble of five distinct ESM-1v checkpoints (Meier et al., 2021); we do not use MSA context retrieval. Our implementation is closely based on the authors’ implementation at <https://github.com/luo-group/ConFit>, with a few adaptations. We removed the KL regularization term from the loss, since we found no benefit from including it. Since zero-shot ESM-1v performance is quite poor on most of our datasets, this is not unexpected. Since some of the datasets in Sec. 5.1 exhibit ties or near ties w.r.t. the targets (i.e. fitness scores), we modify the Bradley–Terry loss as follows: i) we explicitly model near-ties (defined with a threshold of 10^{-5}) using a soft target of 0.5; and ii) we suppress the influence of ties by downweighting their contribution to the loss by a factor of 10^3 . We validated our implementation by running the author’s evaluation script and comparing the resulting Spearman correlations to those obtained with our implementation, finding that the Spearman correlations were within 0.001 (or better) than those obtained with the authors’ implementation.

Since ConFit uses a ranking-based loss instead of a regression loss, it does not yield calibrated fitness predictions; indeed the absolute scale of the predictions is arbitrary. Consequently, in order to compute reasonable regression metrics like MAE and RMSE, we apply an additional linear calibration step to ConFit predictions in which we fit two parameters—a scalar intercept and scalar coefficient—to the raw ConFit predictions.⁴⁰ We fit these parameters using RidgeCV from sklearn. This calibration step has minimal impact on Pearson or Spearman correlation metrics.

ESM2-650M-ZeroShot Zero-shot scores are computed using the masked marginal scoring method as described in Meier et al. (2021).

BLOSUM50-ZeroShot For each sequence $\mathbf{x}$, zero-shot scores are computed by summing BLOSUM50 (log-odds) substitution scores⁴¹ position-by-position against the reference sequence $\mathbf{x}_{\text{ref}}$ (Altschul, 1991).

Ridge-OH-ESM2-Aug Like Ridge-OH above except the linear model uses one-hot features concatenated with the normalized (scalar) zero-shot scores used in ESM2-650M-ZeroShot. This baseline is inspired by Hsu et al. (2022).

MLP-OH We use a single-layer MLP like in MLP-ESM2-8M above except with one-hot features and with 32 instead of 128 hidden units.

⁴⁰Additionally, we normalize the raw predictions from each ensemble member by their standard deviation before averaging predictions across the ensemble.

⁴¹That is we compute the sum using entries $\log S_{aa_{\text{ref}}}$ for amino acids a and a_{ref} , where we note that in our notation $\mathbf{S}$ is the exponentiated substitution matrix and $\log^\circ \mathbf{S}$ is the raw matrix with log-odds.

MLP-OH-ESM2-Aug We use a single-layer MLP like in MLP-ESM2-8M above and with the same zero-shot-augmented features as in Ridge-OH-ESM2-Aug. We use 32 instead of 128 hidden units.

Ridge-ESM2-8M-MeanPool Like Ridge-ESM2 above, except the input features are mean-pooled across variable positions, rather than being flattened.

MLP-ESM2-8M-MeanPool Like MLP-ESM2-8M above, except the MLP uses 32 hidden units and the input features are mean-pooled across variable positions, rather than being flattened.

MLP-SaProt-LastLayer Like MLP-ESM2-LastLayer above, except the structure-conditioned PLM SaProt-35M (Su et al., 2024) is used instead of ESM-2-8M.

PLM Usage ESM-2 and ESM-1v were pre-trained on single chain sequences, while some of our datasets consist of multi-chain proteins. Therefore, when computing ESM-2 embeddings or fine-tuning ESM-1v we proceed as follows. For each sequence we: i) obtain the chain boundaries from the associated reference structure; ii) compute independent forward passes for each chain; and iii) concatenate the outputs. Note that we use two ESM-2 variants, namely a smaller model with 8 million parameters and a larger model with 650 million parameters.

SaProt is used as follows. Residues with incomplete backbone coordinates are masked using the # token. If only $C\beta$ is missing, mini3di reconstructs a virtual $C\beta$ from the N, $C\alpha$, and C coordinates. For predicted structures, pLDDT-based masking with a threshold of 70 is applied after 3Di encoding: tokens are first computed for all structurally valid residues, and then positions with low pLDDT are replaced with #. This preserves accurate 3Di tokens for neighboring residues.

Brief Discussion of Additional Baselines The ESM-2-based zero-shot baseline ESM2-650M-ZeroShot has very variable performance, with some Pearson/Spearman correlation coefficients being markedly negative; see Tables 11-19. This kind of variable performance is hardly surprising given the complex relationship between evolutionary preferences and fitness, especially for antibodies, which are subject to stochastic recombination and somatic hypermutation; see (Chungyoun et al., 2024) for similar observations. By contrast BLOSUM50-based zero-shot scores are consistently positive except for the two thermostability landscapes from Tsuboyama et al. (2023) and some of the antibody landscapes from Moulana et al. (2023). Swapping ESM2-8M for ESM-650M in ridge regression and MLP models has mixed effects on performance. While Ridge-ESM2-650M outperforms Ridge-ESM2-8M in the cross-validation setting, predictive performance is roughly comparable in the two OOD regimes. Models that leverage scalar zero-shot features like Ridge-OH-ESM2-Aug and MLP-OH-ESM2-Aug generally outperform their non-augmented counterparts, i.e. Ridge-OH and MLP-OH. We find that mean-pooling features like in Ridge-ESM2-8M-MeanPool and MLP-ESM2-8M-MeanPool degrades performance. We find that ConFit and MLP-ESM2-8M perform similarly to MLP-ESM2-LastLayer, although MLP-ESM2-LastLayer performs best over all. In particular all three of these methods perform quite well in the cross-validation setting but exhibit degraded performance in OOD settings. As we would expect, since ConFit is a ranking-based method, it exhibits poorer Pearson R and MAE metrics. We find that using ESM2-8M as a random feature generator as in MLP-ESM2-8M-RandInit leads to surprisingly good results—for example, often outperforming MLP-ESM2-8M-MeanPool—suggesting that a non-trivial portion of the good performance of methods like MLP-ESM2-8M should be attributed to the general representational capacity of high-dimensional transformer-based sequence-aware n-gram-like features that is orthogonal to any particular learnings from global protein data obtained via pre-training. See (Li et al., 2024) for related benchmarks that involve randomly initialized protein language models. We also find that, while the overall picture is somewhat mixed, MLP-OH outperforms Ridge-OH in most regimes w.r.t. MAE, highlighting the value of making the predictive model non-linear. Finally in MLP-SaProt-LastLayer we include an additional model beyond Kermut-GP and KermutStruc-GP that utilizes the reference structure $\mathcal{S}$. We find that, while MLP-SaProt-LastLayer outperforms MLP-ESM2-LastLayer on average, especially in the OOD setting, the gains are quite modest overall.

A.16.2. MULTI-TASK EXPERIMENT

We first describe the general training and evaluation setup. We assume a set of training landscapes $\{\mathcal{D}_k\}$ where each $\mathcal{D}_k = \{\mathbf{X}_k, \mathbf{t}_k, \mathcal{S}_k\}$ consists of N_k sequences $\mathbf{X}_k = \{\mathbf{x}_{k,n}\}$, N_k properties/targets $\mathbf{t}_k = \{t_{k,n}\}$, and a reference structure $\mathcal{S}_k$. We are given a heldout landscape $\mathcal{D}' = \{\mathbf{X}', \mathbf{t}', \mathcal{S}'\}$ with N' data points. Then we use the training landscapes $\{\mathcal{D}_k\}$ to fit $\mathbf{W}$ (in the case of CLOCK-GP) or neural network weights (in the case of CNN-EMB-OH). The resulting model artifact is then used in conjunction with a subset of data from $\mathcal{D}'$ (say $\{\mathbf{X}'_{\text{train}}, \mathbf{t}'_{\text{train}}, \mathcal{S}'\}$ with $N'_{\text{train}} < N'$ data points) to make

predictions on test data from $\mathcal{D}'$ (in particular the complement $\{\mathbf{X}'_{\text{test}}, \mathbf{t}'_{\text{test}}, \mathcal{S}'\}$ with $N'_{\text{test}} = N' - N'_{\text{train}}$ data points). It is the performance of these model predictions for N'_{test} test data points that we report.

In more detail, we randomly split the 371 landscapes into 280 training landscapes, 41 validation landscapes, and 50 test landscapes. The split is i.i.d. except that we require test landscapes to have at least 800 sequences to enable training on subsets of size 700. The target label we use is ddG_{ML}. We use predicted structures provided by Tsuboyama et al. (2023). The thermostability data has a small number of duplicate sequences; we aggregate duplicates by computing the mean ddG_{ML}. To make aggregate MAE/RMSE metrics easier to interpret, we preprocess all landscapes so that their targets are zero-centered and have unit standard deviation.

For evaluation we define two i.i.d. train/test splits for each of the 50 heldout landscapes for a total of 100 unique train/test pairs. For example, if a given landscape has 1000 data points, and the training size is $N = 100$, then the test set is given by the remaining 900 data points. The model is then trained on $N = 100$ data points and evaluated on 900 data points. For this reason the test set changes as N increases; this is why the zero-shot prediction metrics are not constant w.r.t. the training data size (though they are nearly constant). The same 100 train/test splits are used for all model evaluations.

For the landscape subsampling experiment we train models on i.i.d. subsets of the 280 training landscapes and consider $n_{\text{landscapes}} \in \{10, 20, 35, 70, 140\}$. For models trained on fewer than 280 landscapes, we train two models on distinct random subsets of the 280 training landscapes, in which case metrics are averaged across both models. This reduces variance w.r.t. which of the 280 training landscapes are chosen.

CLOCK-GP Variants A basic implementation of the PyTorch module that we use to compute correlation matrices is provided in Fig. 30. A basic implementation of the concentrated-likelihood-based loss function is provided in Fig. 31. The signal-to-noise ratio is initialized to 40. We use the Adam optimizer and train for 300 epochs. Chroma embeddings are computed at diffusion time $t = 0.01$. We use an initial learning rate of 0.005 and decimate the learning rate twice (once after epoch 100 and once after epoch 200). In each epoch we evaluate the loss on two mini-batches and thus take two gradient steps per epoch. Each mini-batch consists of a random subset of half the available landscapes. We randomly choose 512 sequences from each landscape, and we center the targets in each sampled landscape. In contrast to the CNN, we do not use the validation data in any way, as we did not observe any evidence for overfitting.⁴² Indeed we found the CLOCK-GP very robust to various choices, for example: i) whether the signal-to-noise ratio is fixed or learned; ii) whether the signal-to-noise ratio is global or landscape-specific; iii) optimizer settings; iv) number of landscapes that enter into each gradient step; etc.

The RBF kernel used in CLOCK-GP+CNN-ZS is given by $k_{\text{rbf}}(\mathbf{x}, \mathbf{y}) = \exp(-\frac{1}{2}(\text{pred}(\mathbf{x}) - \text{pred}(\mathbf{y}))^2/\tau^2)$, where $\tau > 0$ is a learnable length scale and $\text{pred}(\cdot)$ is the CNN-based zero-shot predictor. This kernel is equipped with a learnable kernel scale and added to the LOCK kernel in Eqn. 13.

CNN Baselines The CNN baseline architecture takes positional structure-sequence embeddings of dimension 128 and projects them to 256 channels via a 1×1 convolution. Next multi-scale convolutional processing is performed using three parallel branches with kernel sizes of 3, 5, and 7, where each branch applies two sequential convolutional layers with batch normalization, ReLU activation functions, and dropout (with a rate of 0.05). The outputs from all three branches are globally average-pooled and concatenated, yielding a 768 feature vector that is passed through a two-layer MLP ($768 \rightarrow 512 \rightarrow 256 \rightarrow 1$) to produce the final fitness prediction. We note that while CNN-EMB-OH does not obviously *need* its feature space to be augmented with one-hot sequences—since the positional structure-sequence embeddings from Chroma (computed at diffusion time $t = 0.01$) explicitly depend on sequence information—we found that explicit augmentation results in better performance. Similarly, we also considered variants in which the CNN uses structure-only embeddings (as is done in CLOCK-GP) together with one-hot sequence features, but found that this performs worse—as we would expect. In other words performance differences between the CNN-based methods and CLOCK-GP cannot be explained by the quality of the embeddings used.

Ridge-OH We use RidgeCV from sklearn on one-hot encoded sequences, which selects from 32 ridge parameters α_{ridge} that are geometrically evenly spaced between 10^{-4} and 10^4 (i.e. `numpy.logspace(-4, 4, 32)`).

⁴²This conclusion may not hold in the low landscape regime with $\lesssim 20$ landscapes (where more additional regularization or early stopping may be prudent), but we have not systematically investigated overfitting in this regime.

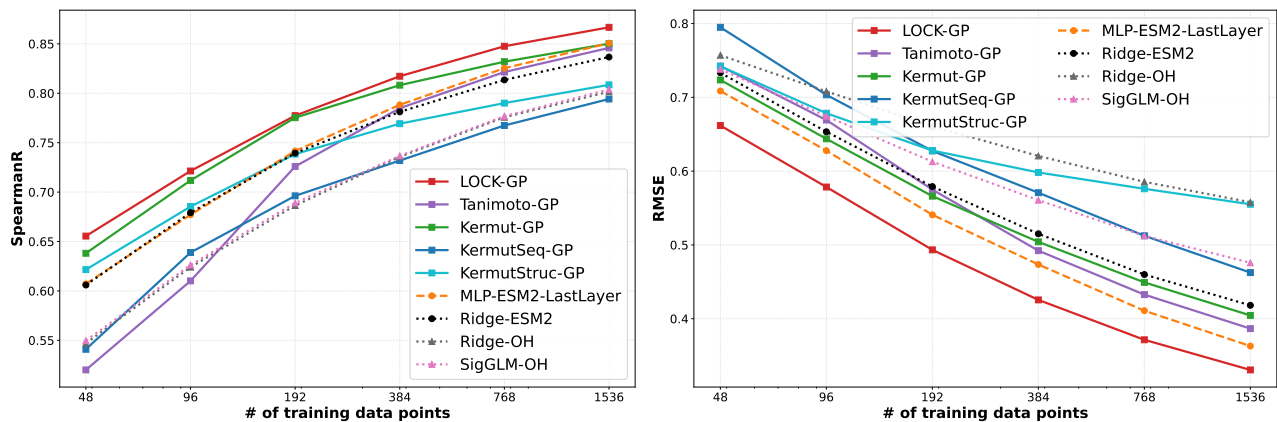


Figure 14. Predictive performance as a function of the number of training data points in the cross-validation setting. We depict Spearman R (left) and RMSE (right); metrics are averaged across 21 datasets. See Sec. 5.3 for discussion and Sec. A.16.1 for details on each model.

Landscape	LOCK-GP	Tanimoto-GP	MLP-ESM2-LastLayer	Kermut-GP	KermutSeq-GP	KermutStruc-GP	Ridge-ESM2	Ridge-OH	SigGLM-OH	BLOSUM50-ZeroShot	ESM2-650M-ZeroShot
CAPSD_AAV2S_Sinai_2021	0.748	0.699	0.663	0.695	0.633	0.610	0.655	0.581	0.585	0.297	0.280
Q8WTC7_9CNID_Somermeyer_2022	0.650	0.393	0.468	0.604	0.175	0.625	0.515	0.532	0.535	0.252	-0.042
CH65 - G189E	0.948	0.936	0.932	0.886	0.878	0.841	0.890	0.842	0.880	0.543	-0.394
CH65 - MA90	0.969	0.967	0.963	0.947	0.943	0.857	0.950	0.856	0.872	0.532	-0.443
CH65 - S106	0.952	0.926	0.945	0.881	0.876	0.854	0.882	0.854	0.892	0.478	-0.359
CR6261 - H1	0.975	0.957	0.960	0.970	0.968	0.864	0.948	0.855	0.932	0.665	-0.738
CR6261 - H9	0.957	0.926	0.928	0.953	0.951	0.885	0.917	0.884	0.899	0.683	-0.687
CR9114 - H1	0.949	0.908	0.933	0.835	0.853	0.788	0.898	0.786	0.884	0.428	-0.338
CR9114 - H3	0.877	0.710	0.901	0.533	0.602	0.494	0.702	0.499	0.782	0.399	-0.179
D7PM05_CLYGR_Somermeyer_2022	0.695	0.458	0.472	0.637	0.360	0.621	0.496	0.426	0.431	0.523	0.054
GCN4_YEAST_Staller_2018	0.223	0.194	0.339	0.420	0.433	0.182	0.345	0.116	0.124	0.107	0.159
GFP_AEQVL_Sarkisyan_2016	0.738	0.617	0.597	0.725	0.536	0.692	0.630	0.502	0.502	0.619	0.114
HIS7_YEAST_Pokusaeva_2019	0.739	0.691	0.679	0.735	0.722	0.735	0.706	0.626	0.628	0.234	0.401
PHOT_CHLRE_Chen_2023	0.933	0.922	0.918	0.927	0.922	0.913	0.918	0.910	0.912	0.421	0.730
F7YBW8_MESOW_Ding_2023	0.814	0.803	0.789	0.775	0.777	0.752	0.768	0.758	0.755	0.486	0.364
Q6WV12_9MAXI_Somermeyer_2022	0.772	0.636	0.612	0.688	0.308	0.700	0.610	0.633	0.636	0.358	0.023
Tsuboyama - 2B88	0.721	0.721	0.804	0.815	0.618	0.740	0.827	0.643	0.641	0.064	0.089
Tsuboyama - 2HBB	0.721	0.717	0.711	0.809	0.752	0.697	0.738	0.630	0.630	0.013	0.327
Moulana CB6	0.989	0.986	0.991	0.981	0.957	0.980	0.982	0.980	0.987	0.268	-0.206
Moulana Cov555	0.965	0.921	0.961	0.861	0.816	0.854	0.918	0.854	0.938	0.401	-0.123
Moulana Regn10987	0.953	0.919	0.946	0.887	0.846	0.885	0.911	0.885	0.916	0.336	0.124

Table 11. Pearson R metric for each landscape for 192 training data points in the cross-validation regime. The best metric for each landscape is marked in bold. See Sec. 5.3 for discussion and Sec. A.16.1 for details on each model.

Ridge-EMB and Ridge-EMB-OH These are like Ridge-OH above but use different features: Ridge-EMB uses mean-pooled structure-and-sequence embeddings from Chroma, while Ridge-EMB-OH concatenates mean-pooled structure-and-sequence embeddings with one-hot encoded sequences.

A.16.3. BINARY CLASSIFICATION EXPERIMENT

We consider the Q8WTC7_9CNID_Somermeyer_2022 landscape from Gonzalez Somermeyer et al. (2022). We use the same cross-validation setup as in Sec. A.16.1 except we stratify to ensure class labels are approximately balanced. We quantize the continuous fitness score from Q8WTC7_9CNID_Somermeyer_2022 using the median so that, for example, 50% of binary labels are positive. For GP inference we use the method from Wenzel et al. (2019), closely following the GPyTorch implementation.⁴³ For N training data points we use $N_{\text{ind}} = \min(N, 2048)$ inducing point locations. Inducing points are chosen at random from the training data and kept fixed. To optimize the ELBO we use a maximum mini-batch size of 1024 and train for 30 epochs. We decimate the learning rate after 10 and 20 epochs.

A.17. Additional Figures and Tables for the Local Learning Experiment

We provide additional results for the local learning experiment in Sec. 5.3. For a companion figure to Fig. 2 that depicts Spearman R and RMSE metrics see Fig. 14. For a companion figure to Fig. 2 that depicts CRPS for GP models see Fig. 3. Select results for individual landscapes are reported in Tables 11-20. For an extended version of Table 1 with additional models see Table 21. For uncertainty metrics for GP models see Table 22. For average model ranks w.r.t. three metrics see Table 23.

⁴³See https://docs.gpytorch.ai/en/v1.12/examples/04_Variational_and_Approximate_GPs/PolyaGamma_Binary_Classification.html.

Flexible Kernels for Protein Property Prediction

Landscape	LOCK-GP	Tanimoto-GP	MLP-ESM2-LastLayer	Kermut-GP	KermutSeq-GP	KermutStruc-GP	Ridge-ESM2	Ridge-OH	SigGLM-OH	BLOSUM50-ZeroShot	ESM2-650M-ZeroShot
CAPSD_AAV2S_Sinai_2021	0.754	0.718	0.701	0.719	0.640	0.644	0.686	0.610	0.612	0.178	0.247
Q8WTC7_9CNID_Somermeyer_2022	0.549	0.290	0.328	0.501	0.164	0.519	0.386	0.385	0.386	0.265	-0.024
CH65 - G189E	0.961	0.952	0.958	0.917	0.908	0.877	0.928	0.878	0.876	0.558	-0.404
CH65 - MA90	0.966	0.965	0.965	0.952	0.947	0.857	0.959	0.856	0.856	0.531	-0.438
CH65 - SI06	0.881	0.877	0.885	0.874	0.870	0.855	0.878	0.854	0.856	0.498	-0.359
CR6261 - H1	0.967	0.938	0.949	0.947	0.941	0.912	0.921	0.917	0.916	0.680	-0.724
CR6261 - H9	0.971	0.958	0.967	0.965	0.963	0.924	0.951	0.927	0.926	0.710	-0.704
CR9114 - H1	0.913	0.870	0.896	0.833	0.833	0.846	0.854	0.843	0.846	0.532	-0.458
CR9114 - H3	0.527	0.512	0.529	0.488	0.472	0.491	0.510	0.493	0.516	0.396	-0.197
D7PM05_CLYGR_Somermeyer_2022	0.699	0.447	0.456	0.689	0.364	0.672	0.496	0.425	0.428	0.547	0.052
GCN4_YEAST_Staller_2018	0.277	0.226	0.416	0.481	0.493	0.234	0.404	0.113	0.135	0.209	0.206
GFP_AEQVI_Sarkisyan_2016	0.708	0.603	0.579	0.727	0.523	0.695	0.614	0.477	0.478	0.604	0.094
HIS7_YEAST_Pokusaeva_2019	0.693	0.645	0.630	0.720	0.702	0.720	0.659	0.593	0.596	0.254	0.387
PHOT_CHLRE_Chen_2023	0.941	0.933	0.940	0.936	0.930	0.925	0.930	0.923	0.924	0.398	0.717
F7YBW8_MESOW_Ding_2023	0.728	0.726	0.707	0.715	0.715	0.701	0.717	0.703	0.700	0.582	0.423
Q6WV12_9MAXI_Somermeyer_2022	0.660	0.520	0.475	0.573	0.273	0.586	0.497	0.492	0.494	0.333	0.009
Tsuboyama - 2B88	0.684	0.682	0.747	0.757	0.574	0.673	0.769	0.615	0.613	0.027	0.079
Tsuboyama - 2HBB	0.746	0.738	0.738	0.828	0.778	0.720	0.753	0.653	0.652	-0.010	0.365
Moulana CB6	0.918	0.915	0.920	0.914	0.881	0.915	0.912	0.915	0.917	0.287	-0.210
Moulana Cov555	0.857	0.832	0.858	0.830	0.785	0.826	0.827	0.828	0.834	0.420	-0.087
Moulana Regn10987	0.923	0.898	0.929	0.915	0.863	0.914	0.877	0.914	0.908	0.327	0.104

Table 12. Spearman R metric for each landscape for 192 training data points in the cross-validation regime. The best metric for each landscape is marked in bold. See Sec. 5.3 for discussion and Sec. A.16.1 for details on each model.

Landscape	LOCK-GP	Tanimoto-GP	MLP-ESM2-LastLayer	Kermut-GP	KermutSeq-GP	KermutStruc-GP	Ridge-ESM2	Ridge-OH	SigGLM-OH
CAPSD_AAV2S_Sinai_2021	0.528	0.579	0.611	0.573	0.618	0.644	0.612	0.672	0.661
Q8WTC7_9CNID_Somermeyer_2022	0.515	0.753	0.636	0.592	0.778	0.575	0.634	0.620	0.619
CH65 - G189E	0.220	0.252	0.254	0.354	0.363	0.429	0.359	0.429	0.352
CH65 - MA90	0.193	0.202	0.218	0.256	0.265	0.432	0.248	0.432	0.411
CH65 - SI06	0.206	0.282	0.240	0.364	0.383	0.407	0.373	0.405	0.332
CR6261 - H1	0.138	0.207	0.197	0.167	0.175	0.410	0.236	0.424	0.269
CR6261 - H9	0.196	0.285	0.260	0.208	0.220	0.372	0.315	0.377	0.335
CR9114 - H1	0.202	0.305	0.247	0.407	0.370	0.481	0.331	0.484	0.342
CR9114 - H3	0.220	0.418	0.197	0.504	0.414	0.538	0.463	0.532	0.387
D7PM05_CLYGR_Somermeyer_2022	0.568	0.911	0.761	0.643	0.853	0.653	0.738	0.794	0.792
GCN4_YEAST_Staller_2018	0.707	0.714	0.679	0.633	0.626	0.723	0.673	0.735	0.756
GFP_AEQVI_Sarkisyan_2016	0.529	0.696	0.662	0.549	0.723	0.578	0.624	0.738	0.735
HIS7_YEAST_Pokusaeva_2019	0.518	0.594	0.574	0.529	0.536	0.532	0.553	0.630	0.628
PHOT_CHLRE_Chen_2023	0.268	0.293	0.297	0.284	0.294	0.313	0.300	0.316	0.312
F7YBW8_MESOW_Ding_2023	0.435	0.453	0.455	0.472	0.476	0.498	0.490	0.495	0.500
Q6WV12_9MAXI_Somermeyer_2022	0.426	0.601	0.549	0.537	0.753	0.523	0.573	0.564	0.561
Tsuboyama - 2B88	0.457	0.470	0.423	0.410	0.572	0.476	0.401	0.523	0.526
Tsuboyama - 2HBB	0.497	0.508	0.527	0.437	0.491	0.540	0.498	0.576	0.576
Moulana CB6	0.067	0.102	0.076	0.140	0.217	0.145	0.132	0.144	0.103
Moulana Cov555	0.174	0.311	0.214	0.428	0.471	0.443	0.316	0.445	0.289
Moulana Regn10987	0.157	0.290	0.191	0.360	0.434	0.364	0.311	0.366	0.287

Table 13. MAE metric for each landscape for 192 training data points in the cross-validation regime. The best metric for each landscape is marked in bold. See Sec. 5.3 for discussion and Sec. A.16.1 for details on each model.

Landscape	LOCK-GP	Tanimoto-GP	MLP-ESM2-LastLayer	Kermut-GP	KermutSeq-GP	KermutStruc-GP	Ridge-ESM2	Ridge-OH	SigGLM-OH	BLOSUM50-ZeroShot	ESM2-650M-ZeroShot
CAPSD_AAV2S_Sinai_2021	0.701	0.652	0.595	0.653	0.565	0.593	0.637	0.536	0.540	0.138	0.225
Q8WTC7_9CNID_Somermeyer_2022	0.404	0.183	0.187	0.225	0.037	0.226	0.230	0.192	0.203	0.262	-0.033
CH65 - G189E	0.694	0.676	0.675	0.674	0.619	0.675	0.493	0.625	0.637	0.548	-0.400
CH65 - MA90	0.672	0.673	0.682	0.746	0.756	0.629	0.622	0.616	0.617	0.528	-0.447
CH65 - SI06	0.629	0.634	0.637	0.679	0.593	0.664	0.676	0.619	0.617	0.507	-0.352
CR6261 - H1	0.759	0.756	0.756	0.799	0.825	0.789	0.286	0.730	0.725	0.673	-0.719
CR6261 - H9	0.736	0.731	0.740	0.814	0.821	0.797	0.292	0.720	0.718	0.699	-0.690
CR9114 - H1	0.670	0.660	0.635	0.548	0.453	0.700	0.374	0.634	0.625	0.534	-0.458
CR9114 - H3	0.436	0.450	0.455	0.446	0.293	0.436	0.306	0.440	0.450	0.394	-0.201
D7PM05_CLYGR_Somermeyer_2022	0.635	0.446	0.326	0.597	0.182	0.597	0.367	0.310	0.310	0.546	0.049
GCN4_YEAST_Staller_2018	0.207	0.178	0.297	0.266	0.261	0.178	0.316	0.119	0.119	0.156	0.166
GFP_AEQVI_Sarkisyan_2016	0.656	0.521	0.485	0.636	0.491	0.682	0.519	0.364	0.367	0.598	0.092
HIS7_YEAST_Pokusaeva_2019	0.451	0.252	0.350	0.633	0.613	0.607	0.389	0.227	0.231	0.374	0.375
PHOT_CHLRE_Chen_2023	0.413	0.404	0.311	0.643	0.606	0.695	0.296	0.280	0.260	0.244	0.673
F7YBW8_MESOW_Ding_2023	0.712	0.710	0.719	0.708	0.693	0.707	0.711	0.704	0.705	0.613	0.485
Q6WV12_9MAXI_Somermeyer_2022	0.534	0.397	0.320	0.468	0.236	0.476	0.379	0.390	0.389	0.335	0.010
Tsuboyama - 2B88	0.502	0.423	0.637	0.608	0.477	0.438	0.655	0.434	0.436	-0.021	-0.078
Tsuboyama - 2HBB	0.566	0.492	0.481	0.706	0.644	0.610	0.501	0.432	0.436	0.008	-0.351
Moulana CB6	0.888	0.884	0.870	0.882	0.819	0.894	0.728	0.883	0.884	-0.287	0.212
Moulana Cov555	0.682	0.680	0.681	0.614	0.513	0.618	0.464	0.679	0.684	-0.421	0.089
Moulana Regn10987	0.854	0.843	0.887	0.874	0.682	0.879	0.756	0.870	0.867	-0.322	-0.107

Table 14. Spearman R metric for each landscape for 96 training data points in the unseen mutations regime. Metrics are averaged across three distinct train-test splits, and the best metric for each landscape is marked in bold. See Sec. 5.3 for discussion and Sec. A.16.1 for details on each model.

Flexible Kernels for Protein Property Prediction

Landscape	LOCK-GP	Tanimoto-GP	MLP-ESM2-LastLayer	Kermut-GP	KermutSeq-GP	KermutStruc-GP	Ridge-ESM2	Ridge-OH	SigGLM-OH	BLOSUM50-ZeroShot	ESM2-650M-ZeroShot
CAPSD_AAV2S_Sinai_2021	0.689	0.633	0.554	0.617	0.560	0.546	0.603	0.505	0.508	0.267	0.257
Q8WTC7_9CNID_Somermeyer_2022	0.445	0.225	0.273	0.282	0.045	0.282	0.325	0.252	0.266	0.250	-0.049
CH65 - G189E	0.656	0.655	0.649	0.648	0.593	0.651	0.479	0.606	0.636	0.535	-0.392
CH65 - MA90	0.681	0.683	0.689	0.753	0.758	0.640	0.639	0.628	0.636	0.530	-0.451
CH65 - SI06	0.629	0.632	0.624	0.665	0.573	0.656	0.677	0.611	0.613	0.492	-0.353
CR6261 - H1	0.687	0.707	0.709	0.795	0.842	0.769	0.297	0.688	0.717	0.660	-0.738
CR6261 - H9	0.694	0.684	0.698	0.786	0.802	0.770	0.291	0.689	0.692	0.674	-0.676
CR9114 - H1	0.637	0.637	0.600	0.516	0.430	0.633	0.352	0.588	0.635	0.429	-0.339
CR9114 - H3	0.656	0.550	0.602	0.426	0.374	0.412	0.333	0.420	0.572	0.400	-0.183
D7PM05_CLYGR_Somermeyer_2022	0.636	0.439	0.343	0.573	0.192	0.573	0.388	0.320	0.321	0.523	0.052
GCN4_YEAST_Staller_2018	0.162	0.121	0.209	0.216	0.212	0.138	0.243	0.080	0.080	0.081	0.126
GFP_AEQVI_Sarkisyan_2016	0.686	0.534	0.506	0.639	0.503	0.675	0.535	0.394	0.397	0.609	0.109
HIS7_YEAST_Pokusaeva_2019	0.471	0.295	0.381	0.584	0.565	0.572	0.404	0.265	0.267	0.349	0.368
PHOT_CHLRE_Chen_2023	0.291	0.269	0.295	0.658	0.622	0.676	0.284	0.237	0.241	0.232	0.685
F7YBW8_MESOW_Ding_2023	0.814	0.816	0.817	0.815	0.781	0.802	0.789	0.792	0.793	0.587	0.458
Q6WV12_9MAXI_Somermeyer_2022	0.641	0.504	0.436	0.553	0.263	0.577	0.484	0.507	0.507	0.361	0.024
Tsuboyama - 2B88	0.508	0.368	0.663	0.641	0.484	0.484	0.692	0.404	0.406	-0.061	-0.086
Tsuboyama - 2HBB	0.517	0.463	0.449	0.658	0.606	0.567	0.480	0.398	0.401	-0.014	-0.315
Moulana CB6	0.984	0.981	0.979	0.965	0.840	0.972	0.792	0.977	0.980	-0.269	0.206
Moulana Cov555	0.701	0.699	0.693	0.614	0.526	0.618	0.465	0.677	0.705	-0.401	0.126
Moulana Regn10987	0.877	0.874	0.873	0.862	0.706	0.861	0.739	0.859	0.870	-0.332	-0.129

Table 15. Pearson R metric for each landscape for 96 training data points in the unseen mutations regime. Metrics are averaged across three distinct train-test splits, and the best metric for each landscape is marked in bold. See Sec. 5.3 for discussion and Sec. A.16.1 for details on each model.

Landscape	LOCK-GP	Tanimoto-GP	MLP-ESM2-LastLayer	Kermut-GP	KermutSeq-GP	KermutStruc-GP	Ridge-ESM2	Ridge-OH	SigGLM-OH
CAPSD_AAV2S_Sinai_2021	0.618	0.670	0.738	0.661	0.689	0.743	0.670	0.756	0.759
Q8WTC7_9CNID_Somermeyer_2022	0.647	0.771	0.704	0.706	0.775	0.705	0.704	0.741	0.733
CH65 - G189E	0.666	0.676	0.672	0.672	0.699	0.655	0.870	0.727	0.693
CH65 - MA90	0.684	0.688	0.671	0.584	0.666	0.714	0.813	0.737	0.731
CH65 - SI06	0.577	0.593	0.591	0.670	0.881	0.688	0.835	0.620	0.598
CR6261 - H1	0.656	0.650	0.633	0.543	0.453	0.615	10.02	0.708	0.662
CR6261 - H9	0.695	0.705	0.684	0.632	0.545	0.676	8.580	0.731	0.711
CR9114 - H1	0.474	0.502	0.502	0.590	0.607	0.587	0.886	0.591	0.525
CR9114 - H3	0.392	0.520	0.435	0.638	0.549	0.648	1.386	0.647	0.558
D7PM05_CLYGR_Somermeyer_2022	0.636	0.971	0.842	0.704	0.937	0.699	0.815	0.881	0.873
GCN4_YEAST_Staller_2018	0.729	0.738	0.777	0.725	0.729	0.781	0.710	0.766	0.762
GFP_AEQVI_Sarkisyan_2016	0.599	0.957	0.739	0.633	0.760	0.608	0.710	0.850	0.848
HIS7_YEAST_Pokusaeva_2019	0.776	0.901	0.830	0.687	0.715	0.702	0.809	0.900	0.899
PHOT_CHLRE_Chen_2023	0.918	0.932	0.943	0.822	0.774	0.710	0.911	0.904	0.874
F7YBW8_MESOW_Ding_2023	0.515	0.515	0.504	0.506	0.528	0.535	0.552	0.539	0.538
Q6WV12_9MAXI_Somermeyer_2022	0.565	0.820	0.650	0.632	0.757	0.613	0.655	0.639	0.642
Tsuboyama - 2B88	0.599	0.666	0.517	0.544	0.621	0.651	0.508	0.655	0.658
Tsuboyama - 2HBB	0.665	0.735	0.740	0.600	0.618	0.644	0.731	0.823	0.818
Moulana CB6	0.111	0.197	0.155	0.261	0.542	0.220	0.842	0.154	0.135
Moulana Cov555	0.521	0.587	0.518	0.800	0.854	0.784	1.397	0.613	0.541
Moulana Regn10987	0.374	0.384	0.295	0.344	0.603	0.331	0.770	0.332	0.321

Table 16. MAE metric for each landscape for 96 training data points in the unseen mutations regime. Metrics are averaged across three distinct train-test splits, and the best metric for each landscape is marked in bold. See Sec. 5.3 for discussion and Sec. A.16.1 for details on each model.

Landscape	LOCK-GP	Tanimoto-GP	MLP-ESM2-LastLayer	Kermut-GP	KermutSeq-GP	KermutStruc-GP	Ridge-ESM2	Ridge-OH	SigGLM-OH	BLOSUM50-ZeroShot	ESM2-650M-ZeroShot
CAPSD_AAV2S_Sinai_2021	0.710	0.518	0.399	0.357	0.600	0.318	0.499	0.212	0.218	0.444	0.342
Q8WTC7_9CNID_Somermeyer_2022	0.444	0.303	0.279	0.349	0.100	0.348	0.330	0.352	0.351	0.224	-0.043
CH65 - G189E	0.857	0.846	0.849	0.802	0.689	0.801	0.836	0.792	0.777	0.548	-0.391
CH65 - MA90	0.896	0.879	0.895	0.806	0.737	0.797	0.893	0.795	0.782	0.520	-0.425
CH65 - SI06	0.823	0.822	0.791	0.810	0.759	0.810	0.849	0.808	0.804	0.485	-0.345
CR6261 - H1	0.947	0.919	0.922	0.928	0.924	0.896	0.889	0.877	0.894	0.596	-0.705
CR6261 - H9	0.918	0.918	0.923	0.917	0.910	0.874	0.902	0.882	0.880	0.564	-0.594
CR9114 - H1	0.702	0.627	0.659	0.730	0.486	0.730	0.624	0.664	0.641	0.524	-0.451
CR9114 - H3	0.517	0.518	0.524	0.520	0.424	0.520	0.512	0.520	0.523	0.382	-0.184
D7PM05_CLYGR_Somermeyer_2022	0.443	0.314	0.217	0.422	0.031	0.422	0.250	0.271	0.274	0.332	0.031
GCN4_YEAST_Staller_2018	0.189	0.255	0.286	0.463	0.463	0.177	0.331	0.194	0.194	0.123	0.160
GFP_AEQVI_Sarkisyan_2016	0.516	0.332	0.346	0.294	0.139	0.474	0.384	0.295	0.295	0.435	0.047
HIS7_YEAST_Pokusaeva_2019	0.394	0.343	0.431	0.545	0.487	0.540	0.472	0.277	0.279	0.244	0.382
PHOT_CHLRE_Chen_2023	0.807	0.845	0.845	0.780	0.721	0.801	0.839	0.844	0.830	0.380	0.712
F7YBW8_MESOW_Ding_2023	0.594	0.564	0.577	0.629	0.594	0.561	0.521	0.523	0.548	0.517	0.347
Q6WV12_9MAXI_Somermeyer_2022	0.614	0.505	0.425	0.422	0.094	0.422	0.505	0.497	0.494	0.252	0.010
Tsuboyama - 2B88	0.361	0.592	0.679	0.283	0.386	0.458	0.638	0.560	0.669	0.162	-0.304
Tsuboyama - 2HBB	0.652	0.605	0.473	0.731	0.608	0.533	0.441	0.377	0.375	-0.099	-0.371
Moulana CB6	0.917	0.915	0.904	0.913	0.850	0.913	0.886	0.914	0.913	-0.269	0.229
Moulana Cov555	0.826	0.804	0.831	0.812	0.349	0.812	0.826	0.810	0.812	-0.367	0.137
Moulana Regn10987	0.916	0.842	0.928	0.904	0.535	0.904	0.785	0.903	0.871	-0.315	-0.093

Table 17. Spearman R metric for each landscape for 128 training data points in the extrapolation regime. The best metric for each landscape is marked in bold. See Sec. 5.3 for discussion and Sec. A.16.1 for details on each model.

Flexible Kernels for Protein Property Prediction

Landscape	LOCK-GP	Tanimoto-GP	MLP-ESM2-LastLayer	Kermut-GP	KermutSeq-GP	KermutStruc-GP	Ridge-ESM2	Ridge-OH	SigGLM-OH	BLOSUM50-ZeroShot	ESM2-650M-ZeroShot
CAPSD_AAV2S_Sinai_2021	0.711	0.527	0.297	0.358	0.588	0.310	0.495	0.175	0.176	0.455	0.363
Q8WTC7_9CNID_Somermeyer_2022	0.495	0.341	0.342	0.409	0.111	0.415	0.390	0.405	0.404	0.210	-0.052
CH65 - G189E	0.821	0.802	0.816	0.751	0.636	0.750	0.779	0.741	0.744	0.537	-0.384
CH65 - MA90	0.887	0.867	0.885	0.799	0.726	0.794	0.884	0.793	0.787	0.521	-0.431
CH65 - SI06	0.846	0.849	0.786	0.825	0.770	0.825	0.852	0.824	0.828	0.461	-0.343
CR6261 - H1	0.941	0.925	0.922	0.936	0.920	0.890	0.909	0.871	0.903	0.613	-0.714
CR6261 - H9	0.873	0.855	0.837	0.875	0.874	0.840	0.852	0.846	0.831	0.550	-0.579
CR9114 - H1	0.697	0.620	0.526	0.610	0.238	0.610	0.599	0.534	0.530	0.425	-0.333
CR9114 - H3	0.838	0.683	0.614	0.527	0.472	0.527	0.623	0.530	0.772	0.370	-0.163
D7PM05_CLYGR_Somermeyer_2022	0.502	0.324	0.232	0.460	0.052	0.460	0.266	0.285	0.288	0.365	0.052
GCN4_YEAST_Staller_2018	0.175	0.230	0.200	0.424	0.424	0.162	0.292	0.183	0.182	0.068	0.124
GFP_AEQVI_Sarkisyan_2016	0.557	0.349	0.345	0.333	0.159	0.507	0.425	0.335	0.335	0.465	0.063
HIS7_YEAST_Pokusaeva_2019	0.449	0.399	0.405	0.552	0.487	0.564	0.526	0.352	0.356	0.224	0.397
PHOT_CHLRE_Chen_2023	0.805	0.848	0.830	0.786	0.730	0.807	0.841	0.847	0.835	0.390	0.722
F7YBW8_MESOW_Ding_2023	0.708	0.679	0.658	0.758	0.635	0.707	0.652	0.664	0.666	0.409	0.280
Q6WV12_9MAXI_Somermeyer_2022	0.624	0.520	0.464	0.119	0.470	0.539	0.522	0.521	0.521	0.266	0.011
Tsuboyama - 2B88	0.598	0.667	0.714	0.482	0.489	0.639	0.690	0.603	0.605	0.080	-0.259
Tsuboyama - 2HBB	0.630	0.598	0.499	0.704	0.569	0.521	0.479	0.400	0.398	-0.106	-0.353
Moulana CB6	0.989	0.982	0.982	0.978	0.856	0.978	0.962	0.978	0.982	-0.249	0.223
Moulana Cov555	0.902	0.848	0.819	0.851	0.366	0.851	0.847	0.850	0.873	-0.342	0.172
Moulana Regn10987	0.893	0.824	0.821	0.842	0.384	0.842	0.818	0.838	0.788	-0.326	-0.113

Table 18. Pearson R metric for each landscape for 128 training data points in the extrapolation regime. The best metric for each landscape is marked in bold. See Sec. 5.3 for discussion and Sec. A.16.1 for details on each model.

Landscape	LOCK-GP	Tanimoto-GP	MLP-ESM2-LastLayer	Kermut-GP	KermutSeq-GP	KermutStruc-GP	Ridge-ESM2	Ridge-OH	SigGLM-OH
CAPSD_AAV2S_Sinai_2021	0.582	0.927	1.338	1.112	0.680	1.248	0.970	1.488	1.541
Q8WTC7_9CNID_Somermeyer_2022	0.813	0.893	0.829	0.817	0.894	0.836	0.822	0.812	0.814
CH65 - G189E	0.627	0.734	0.738	0.700	0.763	0.701	0.704	0.722	0.776
CH65 - MA90	0.689	0.748	0.671	0.594	0.724	0.683	0.537	0.707	0.753
CH65 - SI06	0.559	0.489	0.724	0.535	0.500	0.535	0.454	0.516	0.448
CR6261 - H1	0.242	0.364	0.319	0.330	0.382	0.593	0.330	0.627	0.395
CR6261 - H9	0.364	0.431	0.429	0.369	0.387	0.509	0.414	0.538	0.447
CR9114 - H1	0.461	0.567	0.576	0.598	0.691	0.598	0.525	0.669	0.664
CR9114 - H3	0.506	1.893	1.853	3.369	1.606	3.369	2.996	4.045	1.464
D7PM05_CLYGR_Somermeyer_2022	0.624	1.283	1.208	0.775	1.281	0.775	1.132	1.165	1.159
GCN4_YEAST_Staller_2018	0.726	0.720	0.894	0.638	0.638	0.719	0.744	0.715	0.714
GFP_AEQVI_Sarkisyan_2016	0.761	1.243	1.110	1.071	1.152	0.914	0.991	1.155	1.153
HIS7_YEAST_Pokusaeva_2019	0.724	0.790	0.819	0.684	0.719	0.691	0.779	0.886	0.889
PHOT_CHLRE_Chen_2023	0.788	0.426	0.444	1.423	0.926	0.885	0.479	0.451	0.460
F7YBW8_MESOW_Ding_2023	0.811	0.905	0.900	0.600	1.042	0.825	0.994	0.970	0.911
Q6WV12_9MAXI_Somermeyer_2022	0.730	1.039	0.874	0.890	1.041	0.890	0.830	0.865	0.865
Tsuboyama - 2B88	1.046	0.528	0.548	1.262	0.668	0.889	0.679	0.630	0.605
Tsuboyama - 2HBB	0.632	0.733	0.724	0.647	0.719	0.807	0.859	1.444	1.471
Moulana CB6	0.091	0.211	0.233	0.199	0.581	0.199	0.214	0.195	0.156
Moulana Cov555	0.349	0.513	0.492	0.446	1.328	0.446	0.457	0.446	0.403
Moulana Regn10987	0.303	0.573	0.460	0.679	0.833	0.679	0.487	0.682	0.716

Table 19. MAE metric for each landscape for 128 training data points in the extrapolation regime. The best metric for each landscape is marked in bold. See Sec. 5.3 for discussion and Sec. A.16.1 for details on each model.

Landscape	Cross-validation (192 data points)					Unseen mutations (96 data points)					Extrapolation (128 data points)				
	LOCK-GP	Tanimoto-GP	Kermut-GP	KermutSeq-GP	KermutStruc-GP	LOCK-GP	Tanimoto-GP	Kermut-GP	KermutSeq-GP	KermutStruc-GP	LOCK-GP	Tanimoto-GP	Kermut-GP	KermutSeq-GP	KermutStruc-GP
CAPSD_AAV2S_Sinai_2021	1.05	1.12	1.11	1.20	1.19	1.19	1.24	1.23	1.29	1.37	1.19	1.52	1.82	1.27	2.24
Q8WTC7_9CNID_Somermeyer_2022	1.14	1.36	1.23	1.41	1.17	1.35	1.46	1.42	924	1.42	1.42	1.70	1.52	1.73	1.54
CH65 - G189E	0.298	0.369	0.666	0.699	0.814	3.13	2.50	3.29	3.39	2.88	7.83	10.1	29.4	17.0	29.5
CH65 - MA90	0.011	0.059	0.287	0.322	0.758	1.39	1.54	1.41	2.08	2.07	3.50	4.29	2.95	5.45	8.75
CH65 - SI06	0.265	0.446	0.676	0.688	0.770	2.93	3.11	6.02	4.58	6.01	1.52	1.05	3.70	1.04	3.70
CR6261 - H1	-0.123	0.163	-0.054	0.001	0.736	3.41	1.74	1.28	1.42	1.58	0.440	0.682	0.850	0.756	1.73
CR6261 - H9	0.145	0.431	0.193	0.231	0.659	4.36	4.09	3.16	2.88	2.70	0.860	1.16	1.10	1.26	1.50
CR9114 - H1	0.259	0.534	0.828	0.771	0.938	19.4	20.2	20.7	20.7	24.0	6.43	3.71	9.69	9.42	9.69
CR9114 - H3	0.724	1.10	1.29	1.24	1.32	1.29	1.33	1.37	1.39	1.40	1.39	2.16	6.96	2.07	6.96
D7PM05_CLYGR_Somermeyer_2022	1.13	1.37	1.16	1.35	1.21	1.18	1.42	1.26	1.43	1.26	1.17	1.99	1.39	3.3 × 10³	1.39
GCN4_YEAST_Staller_2018	1.42	1.41	1.36	1.33	1.44	1.47	1.46	1.43	1.43	1.50	1.40	1.48	1.31	1.31	1.41
GFP_AEQVI_Sarkisyan_2016	1.03	1.19	1.05	1.24	1.14	1.11	1.41	1.19	1.28	1.18	1.59	2.67	1.94	2.42	1.64
HIS7_YEAST_Pokusaeva_2019	1.02	1.10	1.05	1.07	1.02	1.35	1.44	1.27	1.29	1.29	1.89	2.14	1.37	1.53	1.67
PHOT_CHLRE_Chen_2023	0.354	0.439	0.410	0.445	0.508	2.22	2.27	2.01	2.09	1.82	1.36	0.982	3.34	1.52	3.09
F7YBW8_MESOW_Ding_2023	0.923	0.935	0.971	0.990	1.01	1.09	1.07	1.12	1.15	1.13	1.42	1.56	1.18	1.70	1.51
Q6WV12_9MAXI_Somermeyer_2022	0.885	1.17	1.08	1.35	1.07	1.12	1.41	1.29	1.45	1.23	1.40	1.88	1.66	1.81	1.66
Tsuboyama - 2B88	0.935	0.982	0.876	1.16	1.04	1.38	1.47	1.40	1.48	1.70	1.21	1.91	1.34	1.54	1.54
Tsuboyama - 2HBB	0.934	0.987	0.874	0.979	1.10	1.28	1.38	1.20	1.21	1.40	1.28	1.40	1.31	1.34	1.53
Moulana CB6	-0.204	-0.347	-0.192	0.154	-0.172	0.078	0.222	0.669	1.16	0.613	-0.413	0.069	2.33	1.17	2.33
Moulana Cov555	0.062	0.466	0.746	0.890	0.766	2.23	2.54	3.18	2.81	3.23	0.691	1.15	1.50	2.83	1.50
Moulana Regn10987	0.232	0.487	0.883	0.805	0.658	0.828	0.973	2.37	1.15	2.86	2.97	1.68	4.23	3.58	4.23

Table 20. We report the negative log likelihood (NLL; lower is better) for each GP variant on 21 landscapes in three different evaluation regimes. The best metric for each landscape is marked in bold, while NLLs larger than 10 are marked red.

	Cross-validation (48 data points)			Cross-validation (1536 data points)			Unseen mutations (96 data points)			Extrapolation (128 data points)			Extrapolation (512 data points)		
	Spearman	Pearson	MAE	Spearman	Pearson	MAE	Spearman	Pearson	MAE	Spearman	Pearson	MAE	Spearman	Pearson	MAE
LOCK-GP	0.655	0.682	0.496	0.867	0.914	0.210	0.610	0.622	0.591	0.669	0.711	0.592	0.759	0.807	0.439
Tanimoto-GP	0.520	0.517	0.588	0.846	0.888	0.272	0.555	0.560	0.675	0.632	0.654	0.762	0.739	0.769	0.574
Kermut-GP	0.638	0.639	0.547	0.850	0.888	0.285	0.629	0.632	0.617	0.639	0.654	0.845	0.750	0.767	0.670
KermutSeq-GP	0.541	0.539	0.615	0.794	0.838	0.338	0.532	0.537	0.681	0.518	0.505	0.836	0.605	0.614	0.714
KermutStruc-GP	0.622	0.625	0.564	0.809	0.805	0.417	0.614	0.613	0.634	0.624	0.641	0.847	0.706	0.713	0.847
ConFit	0.628	0.615	0.574	0.863	0.867	0.344	0.548	0.545	0.726	0.632	0.622	0.882	0.733	0.723	0.828
MLP-ESM2-LastLayer	0.607	0.627	0.529	0.851	0.900	0.243	0.558	0.573	0.626	0.628	0.619	0.771	0.738	0.742	0.618
MLP-ESM2-8M	0.597	0.618	0.538	0.856	0.898	0.256	0.530	0.551	0.678	0.621	0.648	0.711	0.739	0.763	0.562
MLP-ESM2-650M	0.552	0.563	0.652	0.828	0.873	0.288	0.496	0.511	0.760	0.568	0.576	0.767	0.677	0.702	0.640
MLP-ESM2-8M-MeanPool	0.561	0.579	0.594	0.797	0.843	0.362	0.501	0.510	0.725	0.536	0.553	0.814	0.658	0.673	0.679
MLP-OH	0.553	0.580	0.565	0.764	0.808	0.459	0.506	0.527	0.668	0.577	0.596	0.863	0.653	0.680	0.810
MLP-OH-ESM2-650M-Aug	0.581	0.605	0.548	0.771	0.818	0.442	0.567	0.589	0.620	0.619	0.632	0.789	0.681	0.702	0.737
MLP-ESM2-8M-RandInit	0.558	0.575	0.572	0.839	0.880	0.295	0.496	0.517	0.705	0.581	0.609	0.747	0.709	0.727	0.606
MLP-SaProt-LastLayer	0.608	0.629	0.536	0.851	0.903	0.238	0.568	0.583	0.633	0.644	0.640	0.789	0.751	0.759	0.609
Ridge-ESM2-8M	0.606	0.617	0.558	0.837	0.877	0.301	0.476	0.490	1.628	0.629	0.653	0.781	0.719	0.740	0.685
Ridge-ESM2-650M	0.625	0.630	0.552	0.836	0.877	0.304	0.506	0.523	1.007	0.623	0.649	0.864	0.710	0.729	0.685
Ridge-OH-ESM2-650M-Aug	0.568	0.570	0.582	0.804	0.805	0.419	0.549	0.552	0.716	0.620	0.624	0.909	0.703	0.705	0.864
Ridge-ESM2-8M-MeanPool	0.552	0.550	0.603	0.786	0.829	0.376	0.361	0.370	1.617	0.552	0.564	0.886	0.654	0.658	0.762
Ridge-OH	0.547	0.553	0.590	0.802	0.800	0.422	0.514	0.519	0.682	0.589	0.599	0.939	0.694	0.698	0.877
SigGLM-OH	0.550	0.569	0.570	0.804	0.849	0.342	0.515	0.535	0.661	0.587	0.610	0.800	0.697	0.724	0.669
BLOSUM50-ZeroShot	0.396	0.386	–	0.396	0.386	–	0.291	0.281	–	0.270	0.256	–	0.270	0.256	–
ESM2-650M-ZeroShot	-0.044	-0.040	–	-0.044	-0.040	–	-0.070	-0.062	–	-0.072	-0.063	–	-0.072	-0.063	–

Table 21. Model performance metrics for three different evaluation regimes, with the number of training data points ranging from 48 to 1536. Metrics are averaged across 21 datasets. For each column the best performing metric is marked in bold. This table is identical to Table 1, except that it contains additional models (marked in purple). We do not include MAE metrics for zero-shot methods, since they are wildly off-scale. Note that Ridge-ESM2-8M is referred to as Ridge-ESM2 in the main text. We find that ESM2-650M-ZeroShot exhibits significant variability depending in the landscape; see Tables 11-19. See Sec. 5.3 for discussion and Sec. A.16.1 for details on each model.

	Cross-validation (48 data points)			Cross-validation (1536 data points)			Unseen mutations (96 data points)			Extrapolation (128 data points)			Extrapolation (512 data points)		
	NLL	ECE	CRPS	NLL	ECE	CRPS	NLL	ECE	CRPS	NLL	ECE	CRPS	NLL	ECE	CRPS
LOCK-GP	0.987	0.060	0.360	0.112	0.108	0.160	2.56	0.113	0.456	1.95	0.120	0.442	1.26	0.097	0.333
Tanimoto-GP	1.13	0.079	0.413	0.281	0.068	0.201	2.59	0.125	0.499	2.12	0.120	0.573	1.55	0.087	0.424
Kermut-GP	19.8	0.048	0.397	0.358	0.066	0.212	2.77	0.097	0.465	3.87	0.159	0.657	2.30	0.108	0.498
KermutSeq-GP	48.1	0.056	0.440	0.476	0.061	0.248	46.7	0.109	0.507	159.8	0.138	0.635	2.98	0.122	0.540
KermutStruc-GP	1.80	0.047	0.409	0.763	0.040	0.300	2.98	0.104	0.478	4.24	0.159	0.665	4.81	0.167	0.670

Table 22. Uncertainty metrics for GP models for three different evaluation regimes. For each column the best performing metric is marked in bold. LOCK exhibits the best NLL (negative log likelihood) and CRPS (continuous ranked probability score; Gneiting & Raftery (2007)) across the board, while the picture for ECE (expected calibration error) is more mixed. In any case most of the GPs exhibit good (average) calibration, with most ECE values $\lesssim 0.1$, except for the extrapolation regime, in which some of the average ECEs for the three Kermut variants are ~ 0.15 . See Sec. 5.3 for discussion and Sec. A.16.1 for details on each model.

	Cross-validation (48 data points)			Cross-validation (1536 data points)			Unseen mutations (96 data points)			Extrapolation (128 data points)			Extrapolation (512 data points)		
	Spearman	Pearson	MAE	Spearman	Pearson	MAE	Spearman	Pearson	MAE	Spearman	Pearson	MAE	Spearman	Pearson	MAE
LOCK-GP	2.08	1.95	1.92	1.71	1.67	1.33	3.24	2.95	2.81	2.86	2.38	2.62	2.62	2.43	2.00
Tanimoto-GP	5.89	5.75	5.81	3.76	3.67	3.48	4.90	4.71	5.90	4.48	3.86	5.48	4.38	3.71	3.43
Kermut-GP	4.05	4.19	4.33	4.00	3.90	4.10	3.29	3.43	3.48	4.33	4.29	4.24	4.29	3.57	4.43
KermutSeq-GP	6.65	6.27	6.06	6.00	5.86	6.05	6.05	6.00	5.38	6.81	7.29	6.19	7.24	6.81	6.43
KermutStruc-GP	4.73	5.06	5.67	6.86	7.52	7.76	3.57	4.05	4.38	5.19	4.86	5.19	5.43	6.19	5.76
MLP-ESM2-LastLayer	3.87	4.24	3.49	3.24	2.71	2.81	4.48	4.52	4.05	4.05	5.62	5.19	3.48	4.43	4.76
Ridge-ESM2	5.16	5.03	5.22	5.05	4.90	4.90	5.90	6.05	6.57	5.19	4.67	4.57	5.10	5.00	5.33
Ridge-OH	6.54	6.90	6.90	7.48	8.33	8.19	7.10	7.38	6.71	5.95	6.14	6.00	6.33	7.00	7.00
SigGLM-OH	6.03	5.60	5.59	6.90	6.43	6.38	6.48	5.90	5.71	6.14	5.90	5.52	6.14	5.86	5.86

Table 23. We report average performance ranks w.r.t. three metrics for three different evaluation regimes and averaged across 21 landscapes. Ranks range from 1 (for the top performing model) to 9 (for the worst performing model). So for example a rank of 1.5 would suggest that a model typically ranks first or second w.r.t. the corresponding metric. Similarly a rank of 8.9 would mean that the model almost always ranks last among the 9 models w.r.t. the corresponding metric. See Sec. 5.3 for discussion and Sec. A.16.1 for details on each model.

Table 24. This table gives quantitative performance metrics for the multi-task experiment in Sec. 5.4. As such it complements Fig. 4 and Fig. 15. We show results for a few model classes that are not included in the main text. This includes CNN-ZS-OH which concatenates CNN-based zero shot predictions (i.e. these are scalar features) with one-hot sequences and passes the concatenated features to a ridge regression head (as opposed to CNN-EMB-OH, which uses neural features extracted from the CNN together with one-hot features). We also include two local models, Ridge-EMB and Ridge-EMB-OH, which combine a ridge regression head with mean-pooled sequence-and-structure features from Chroma (in the latter case the mean-pooled features are concatenated with one-hot features). In addition, we include 4 GP models (marked by ‘-LIN’) where the kernel is *linear* (i.e. $k_{\text{lin}}^{\text{LOCK}}$), in contrast to the 4 GPs in the main text, which use non-linear kernels (i.e. $k_{\text{nl}}^{\text{LOCK}}$). CLOCK-GP+CNN-ZS performs best across the board w.r.t. all training sizes and all four metrics.

Model	50 training points				150 training points				700 training points			
	Spear.	MAE	Pear.	RMSE	Spear.	MAE	Pear.	RMSE	Spear.	MAE	Pear.	RMSE
Ridge-OH	0.457	0.673	0.474	0.884	0.621	0.567	0.629	0.774	0.748	0.446	0.754	0.638
Ridge-EMB	0.520	0.657	0.550	0.847	0.643	0.567	0.672	0.740	0.746	0.478	0.773	0.626
Ridge-EMB-OH	0.576	0.611	0.604	0.800	0.717	0.499	0.740	0.671	0.829	0.375	0.849	0.517
CNN-ZS	0.724	1.688	0.754	1.820	0.724	1.679	0.754	1.811	0.720	1.688	0.750	1.820
CNN-EMB-OH	0.796	0.433	0.824	0.571	0.838	0.374	0.863	0.502	0.884	0.300	0.904	0.414
CNN-ZS-OH	0.791	0.439	0.819	0.578	0.826	0.388	0.853	0.520	0.863	0.326	0.887	0.449
LOCK-GP	0.498	0.643	0.530	0.852	0.689	0.505	0.712	0.704	0.848	0.332	0.862	0.496
Global CLOCK-GP	0.529	0.611	0.571	0.829	0.720	0.462	0.751	0.657	0.877	0.296	0.893	0.436
CLOCK-GP	0.755	0.446	0.795	0.610	0.826	0.362	0.857	0.508	0.907	0.254	0.924	0.367
CLOCK-GP+CNN-ZS	0.800	0.407	0.832	0.553	0.852	0.338	0.880	0.469	0.911	0.249	0.927	0.358
LOCK-GP-LIN	0.478	0.641	0.514	0.860	0.670	0.513	0.695	0.720	0.833	0.350	0.849	0.518
Global CLOCK-GP-LIN	0.484	0.618	0.534	0.862	0.698	0.480	0.722	0.698	0.859	0.322	0.874	0.476
CLOCK-GP-LIN	0.509	0.581	0.582	0.853	0.743	0.424	0.771	0.641	0.890	0.279	0.904	0.412
CLOCK-GP-LIN+CNN-ZS	0.778	0.435	0.805	0.595	0.837	0.360	0.862	0.502	0.899	0.269	0.916	0.387

A.18. Additional Figures and Tables for the Multi-task Learning Experiment

For additional figures for the multi-task experiment in Sec. 5.4 see Fig. 15-29. In particular see Fig. 15 for Pearson R and RMSE metrics. See Fig. 16-19 for visualizations of correlation matrices. See Fig. 20-27 for additional figures pertaining to the landscape subsampling experiment. For additional quantitative performance metrics see Table 24.

A.19. Code Snippets

See Fig. 30 and Fig. 31.

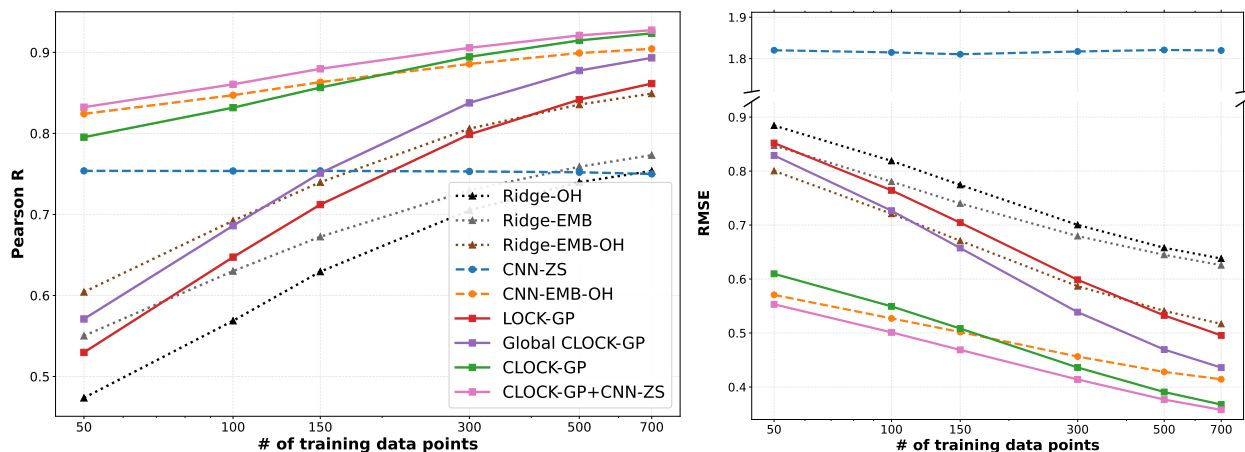


Figure 15. [Companion figure to Fig. 4] Predictive performance as a function of number of training data on the multi-landscape thermostability data described in Sec. 5.4. We depict both Pearson R (left) and root mean squared error (right); metrics are averaged across 100 train/test splits in 50 held-out landscapes. We also include Ridge-EMB and Ridge-EMB-OH models not depicted in the main text; see Sec. A.16.2 for details. The structure-conditioned CLOCK-GP augmented with an additional CNN-based kernel performs best across the board. Note the axis break for the RMSE figure: the absolute scale of the zero-shot predictor is very poorly calibrated. See Table 24 for more quantitative results.

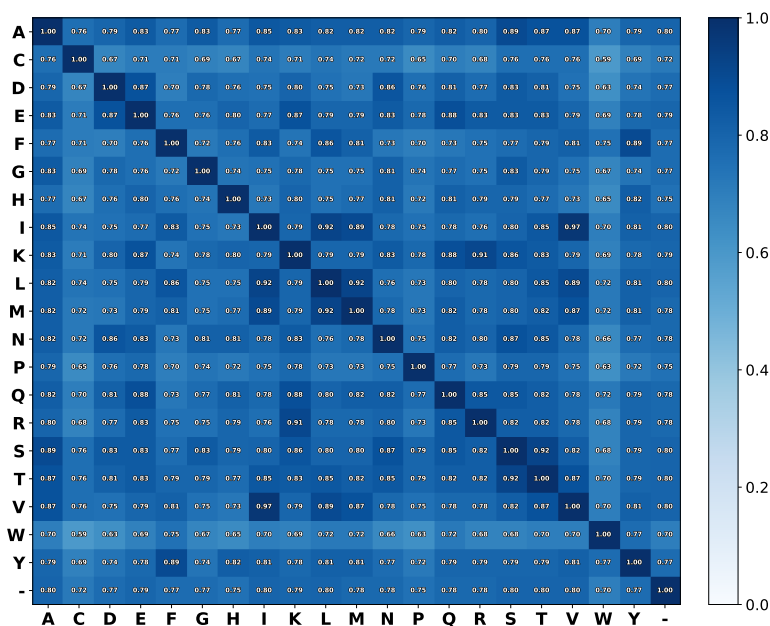


Figure 16. The BLOSUM50 substitution matrix normalized as a 21×21 correlation matrix. See Sec. A.3.1 for details on the normalization scheme used. We include the gap token '-'. The correlation ranges from 0.59 to 1.0.

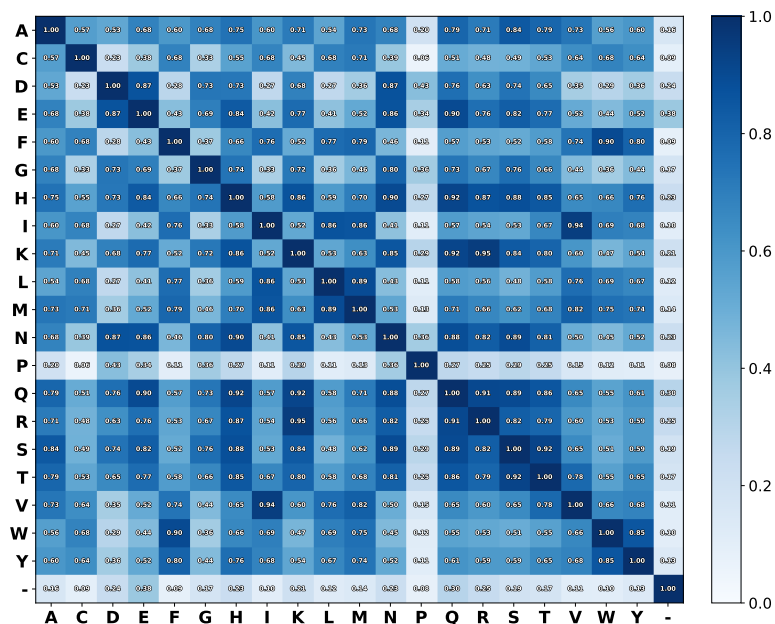


Figure 17. The global correlation matrix fit to the thermostability data described in Sec. 5.4. We include the gap token ‘-’. The correlation ranges from 0.06 to 1.0.

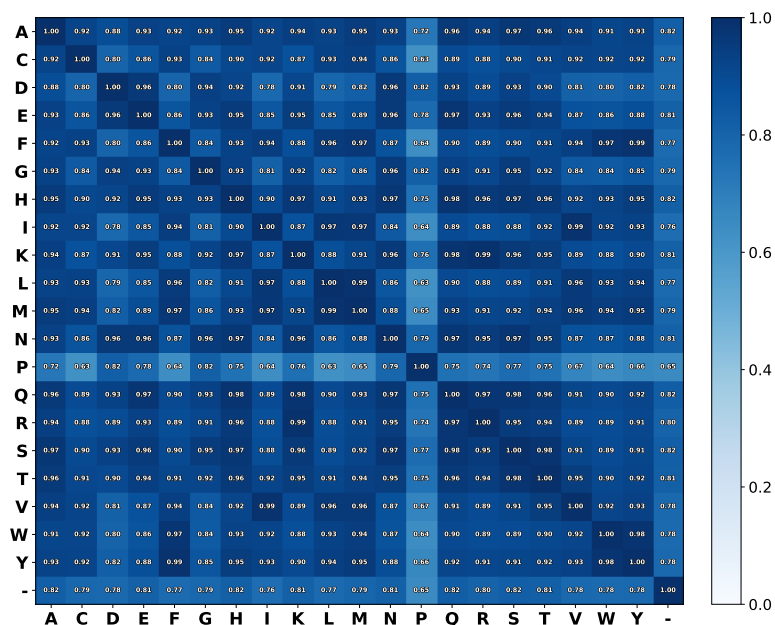


Figure 18. A visual representation of the CLOCK-GP correlation matrix fit to the thermostability data described in Sec. 5.4. Since the correlation matrix varies from position to position, we depict the *average* correlation matrix across all positions in the 50 held-out test landscapes. We include the gap token ‘-’. The correlation ranges from 0.63 to 1.0.

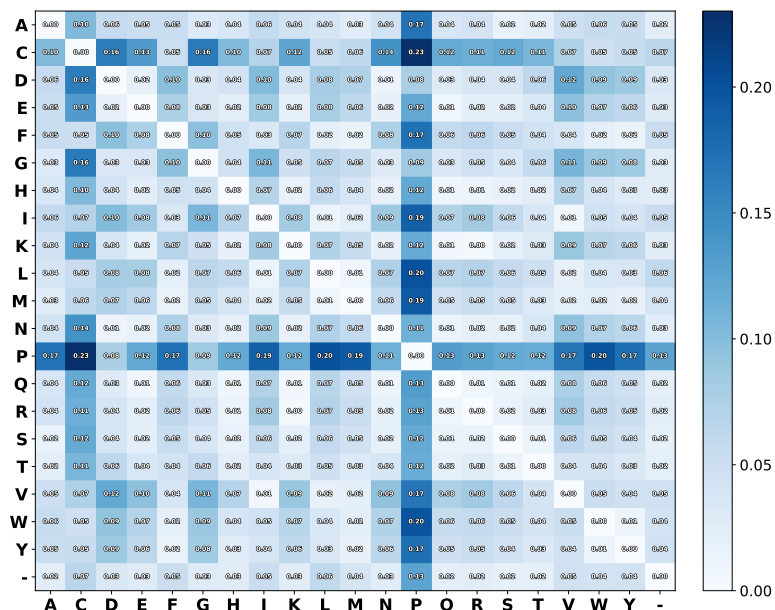


Figure 19. [Companion figure to Fig. 18] We depict the *variability* of the CLOCK-GP correlation matrix fit to the thermostability data described in Sec. 5.4. In particular we depict the quantile difference defined by $\delta = q_{95} - q_5$ where q_{95} and q_5 are the 0.95 and 0.05 quantiles, respectively. This quantile difference is computed using all positions in the 50 held-out test landscapes and ranges from 0.0 (there is no variability at all in the diagonal) to 0.23 (for proline-cysteine).

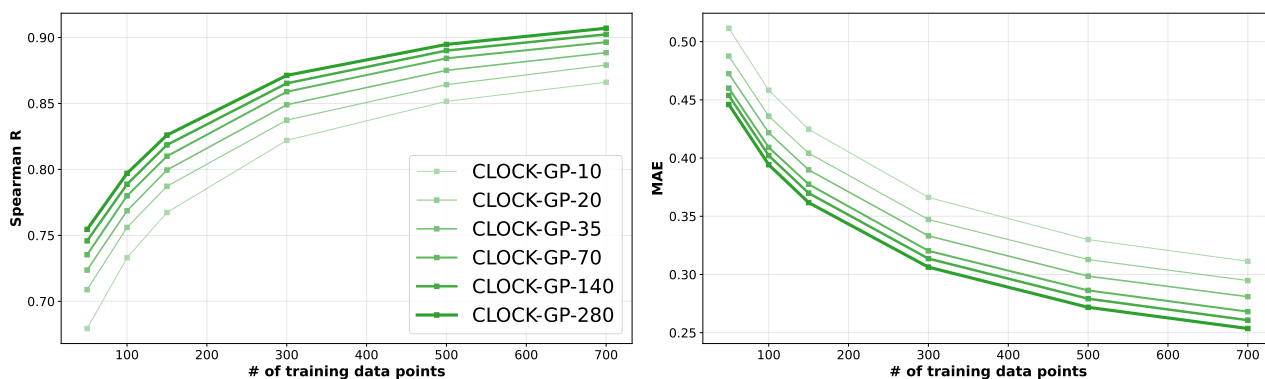


Figure 20. [Companion figure to Fig. 5] We depict how CLOCK-GP performance changes as we vary the number of training landscapes from 10 to 280. Spearman R (left) and MAE (right) are averaged across 100 train/test splits in 50 held-out landscapes. For models trained on fewer than 280 landscapes, we train two models on distinct random subsets of the 280 training landscapes, in which case metrics are averaged across both models. See Sec. 5.4 for additional details on the setup.

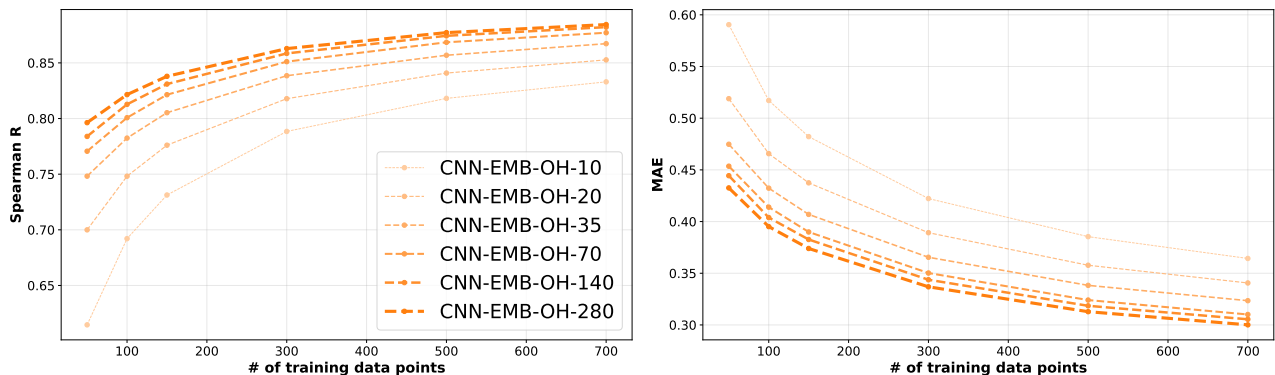


Figure 21. [Companion figure to Fig. 5] We depict how CNN-EMB-OH performance changes as we vary the number of training landscapes from 10 to 280. Spearman R (left) and MAE (right) are averaged across 100 train/test splits in 50 held-out landscapes. For models trained on fewer than 280 landscapes, we train two models on distinct random subsets of the 280 training landscapes, in which case metrics are averaged across both models. See Sec. 5.4 for additional details on the setup.

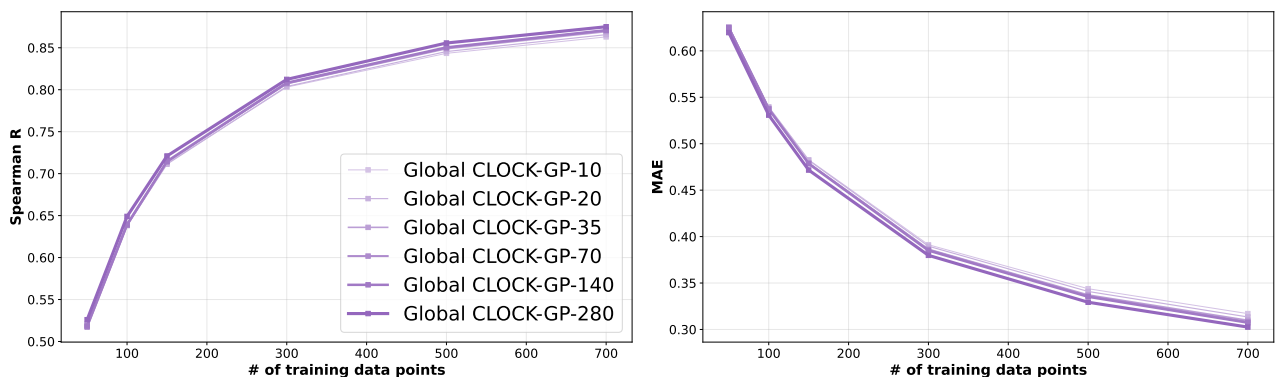


Figure 22. [Companion figure to Fig. 5] We depict how Global CLOCK-GP performance changes as we vary the number of training landscapes from 10 to 280. Spearman R (left) and MAE (right) are averaged across 100 train/test splits in 50 held-out landscapes. For models trained on fewer than 280 landscapes, we train two models on distinct random subsets of the 280 training landscapes, in which case metrics are averaged across both models. See Sec. 5.4 for additional details on the setup.

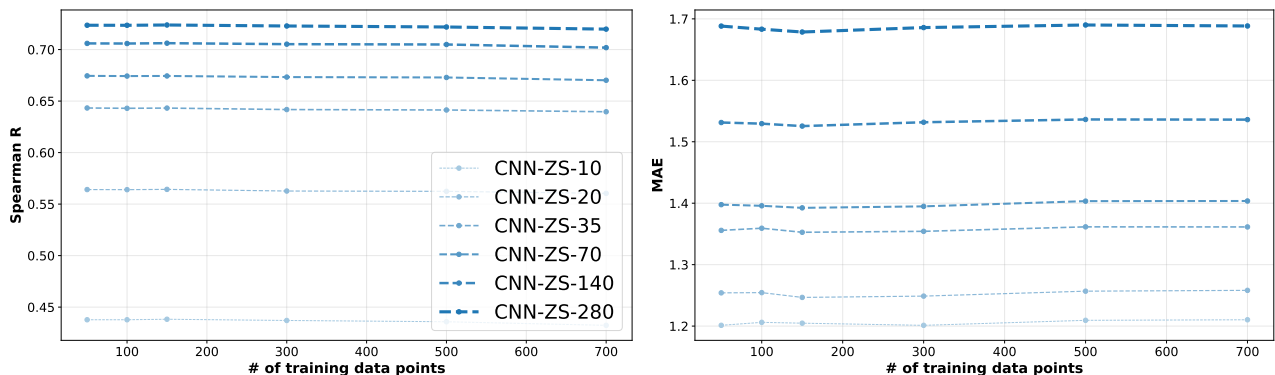


Figure 23. [Companion figure to Fig. 5] We depict how CNN-ZS performance changes as we vary the number of training landscapes from 10 to 280. Spearman R (left) and MAE (right) are averaged across 100 train/test splits in 50 held-out landscapes. Note that even though CNN-ZS is zero-shot, the performance varies slightly from left to right because the held-out test data varies as the size of the “training set” increases and thus the test set changes. For models trained on fewer than 280 landscapes, we train two models on distinct random subsets of the 280 training landscapes, in which case metrics are averaged across both models. See Sec. 5.4 for additional details on the setup.

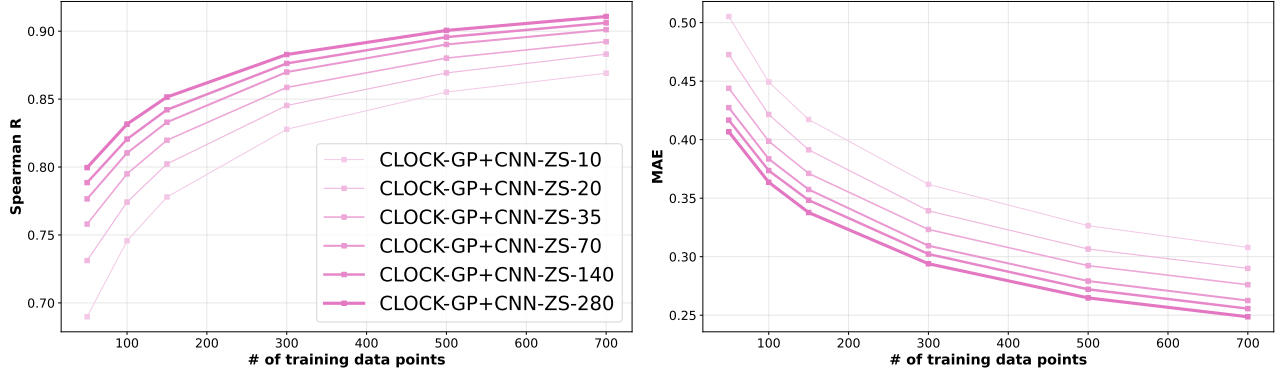


Figure 24. [Companion figure to Fig. 5] We depict how CLOCK-GP+CNN-ZS performance changes as we vary the number of training landscapes from 10 to 280. Spearman R (left) and MAE (right) are averaged across 100 train/test splits in 50 held-out landscapes. For models trained on fewer than 280 landscapes, we train two models on distinct random subsets of the 280 training landscapes, in which case metrics are averaged across both models. See Sec. 5.4 for additional details on the setup.

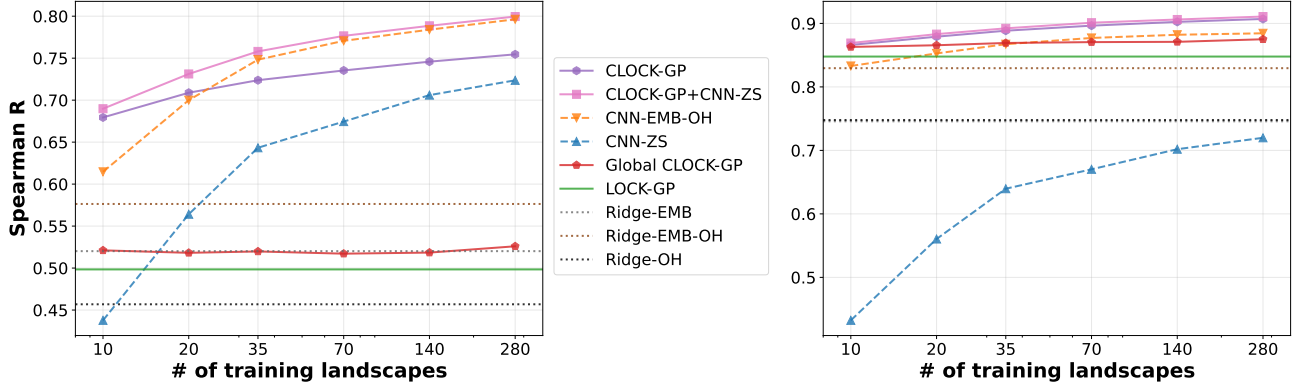


Figure 25. [Companion figure to Fig. 5] We depict how various multi-task models’ performance changes as we vary the number of training landscapes from 10 to 280 for a fixed number of training points: (left) $N_{\text{train}} = 50$; (right) $N_{\text{train}} = 700$. Spearman R is averaged across 100 train/test splits in 50 held-out landscapes. Note that LOCK-GP, Ridge-EMB, Ridge-EMB-OH and Ridge-OH are landscape-local models and so their performance does not vary with the number of training landscapes. For models trained on fewer than 280 landscapes, we train two models on distinct random subsets of the 280 training landscapes, in which case metrics are averaged across both models. See Sec. 5.4 for additional details on the setup.

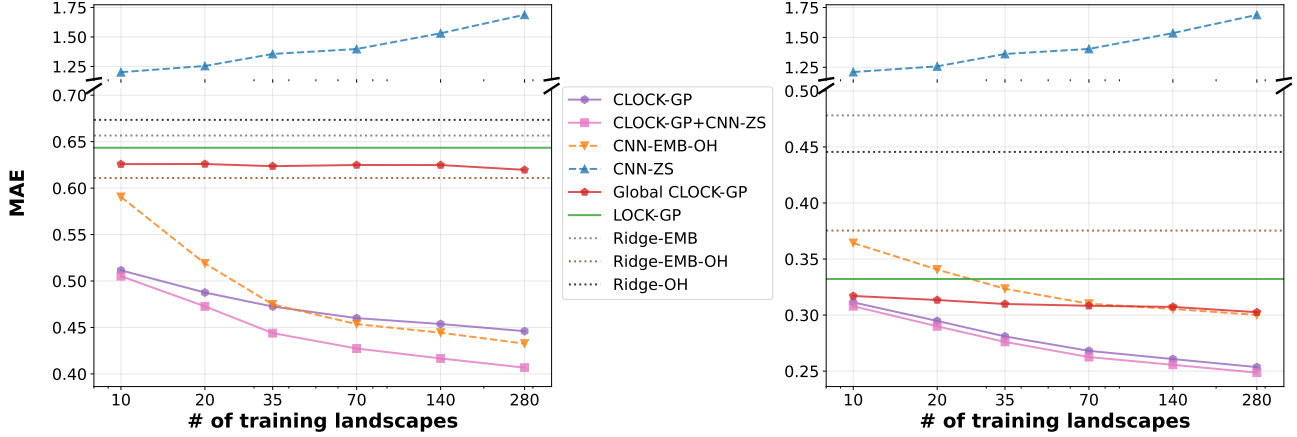


Figure 26. [Companion figure to Fig. 5] We depict how various multi-task models’ performance changes as we vary the number of training landscapes from 10 to 280 for a fixed number of training points: (left) $N_{\text{train}} = 50$; (right) $N_{\text{train}} = 700$. MAE is averaged across 100 train/test splits in 50 held-out landscapes. Note that LOCK-GP, Ridge-OH, Ridge-EMB and Ridge-EMB-OH are landscape-local models and so their performance does not vary with the number of training landscapes. For models trained on fewer than 280 landscapes, we train two models on distinct random subsets of the 280 training landscapes, in which case metrics are averaged across both models. See Sec. 5.4 for additional details on the setup.

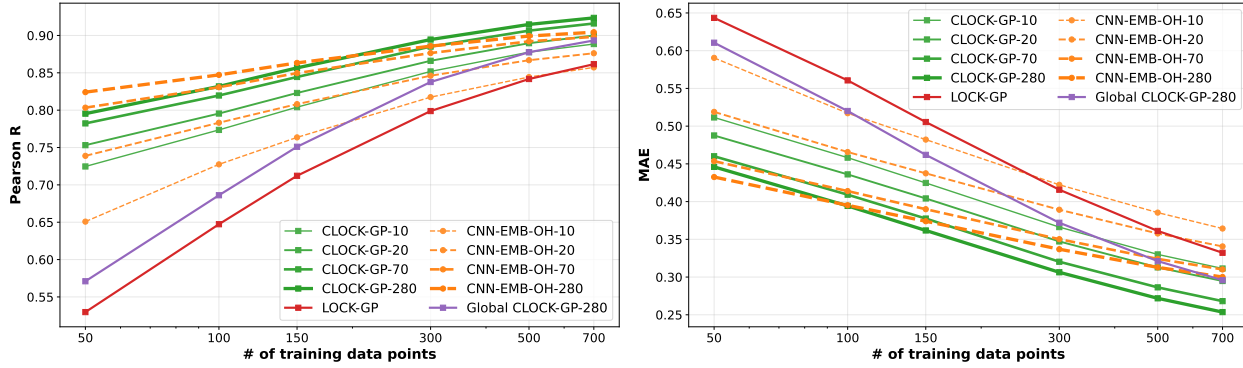


Figure 27. [Companion figure to Fig. 5] We depict how multi-task model performance changes as we vary the number of training landscapes from 10 to 280. Pearson R (left) and MAE (right) are averaged across 100 train/test splits in 50 held-out landscapes. The number of (landscape-local) training data points is plotted on the horizontal axis. CLOCK-GP performs best in the low landscape regime: e.g. CLOCK-GP-10 approximately matches the performance of CNN-EMB-OH-20.

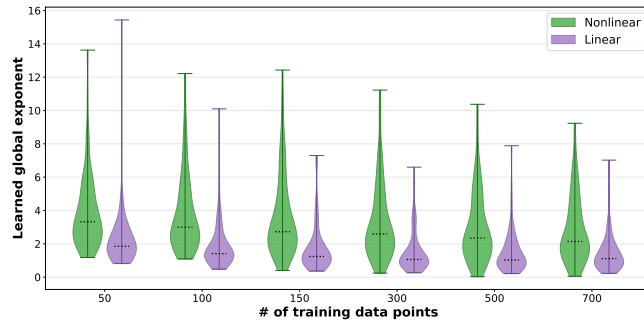


Figure 28. [Companion figure to Fig. 5] We depict the learned exponent α in the CLOCK kernel across 100 train/test splits in 50 held-out landscapes. Notably, the learned exponent is larger for the non-linear kernel $k_{\text{nl}}^{\text{LOCK}}$ than for the linear kernel $k_{\text{lin}}^{\text{LOCK}}$.

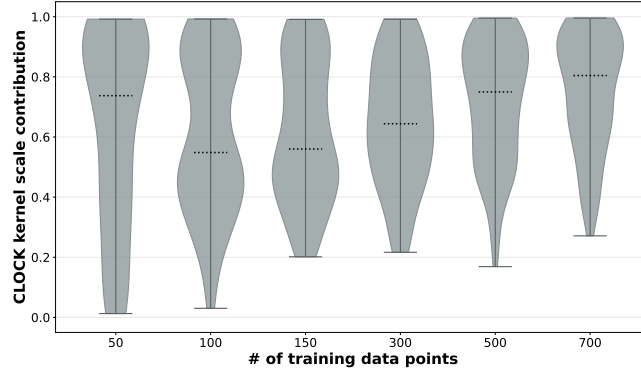


Figure 29. [Companion figure to Fig. 5] We depict the *relative* contribution of the CLOCK kernel scale to CLOCK-GP+CNN-ZS across 100 train/test splits in 50 held-out landscapes. That is the full kernel is of the form $\sigma_0^2 k_{\text{nl}}^{\text{LOCK}} + \sigma_{\text{rbf}}^2 k_{\text{rbf}}$ and we depict distributions of $\frac{\sigma_0^2}{\sigma_0^2 + \sigma_{\text{rbf}}^2}$. As we would expect—given its good performance for larger numbers of training data points—the CLOCK kernel plays an increasingly larger role as the number of training data points increases. We note that across all training sizes the median relative contribution is greater than one half.

```
class CorrelationMachine(nn.Module):
    """Takes structure embeddings and computes residue-wise correlation matrices."""

    def __init__(self, in_dim=128, alphabet_size=21):
        super(CorrelationMachine, self).__init__()
        self.in_dim = in_dim
        self.alphabet_size = alphabet_size
        self.embedding_dim = alphabet_size - 1
        self.layer_norm = nn.LayerNorm(in_dim)
        self.projection = nn.Linear(in_dim, alphabet_size * self.embedding_dim, bias=False)

    def forward(self, embeddings):
        """
        Args:
            embeddings: Structure embeddings (num_landscapes, max_seq_len, in_dim)

        Returns:
            correlation_matrices: (num_landscapes, max_seq_len, alphabet_size, alphabet_size)
        """
        assert embeddings.ndim == 3
        num_landscapes, max_seq_len, _ = embeddings.shape

        zs = self.projection(self.layer_norm(embeddings))
        zs = zs.reshape(num_landscapes, max_seq_len, self.alphabet_size, self.embedding_dim)
        zs = zs / math.sqrt(self.embedding_dim)
        correlation_matrices = torch.exp(-torch.cdist(zs, zs, p=2).pow(2))

        expected_shape = (num_landscapes, max_seq_len, self.alphabet_size, self.alphabet_size)
        assert correlation_matrices.shape == expected_shape
        return correlation_matrices
```

Figure 30. Pytorch implementation of the machinery described in Sec. 3.6, which takes positional structure embeddings $\mathbf{h}_{1:L}(S)$ as input and returns correlation matrices $\mathbf{C}_{1:L}$.

```

def loss(
    seqs: torch.FloatTensor,
    targets: torch.FloatTensor,
    correlation_matrices: torch.FloatTensor,
    signal_to_noise_ratio: torch.FloatTensor,
    mask: torch.FloatTensor = None,
):
    """
    Compute the loss for a given set of sequences, targets, and correlation matrices.

    Args:
        seqs: One-hot encoded sequences of shape (num_landscapes, batch_size, L, A)
        targets: Target values of shape (num_landscapes, batch_size)
        correlation_matrices: Correlation matrices of shape (num_landscapes, L, A, A)
        signal_to_noise_ratio: Signal-to-noise ratio of shape ()
        mask: Optional mask of shape (num_landscapes, batch_size)

    Returns:
        loss: loss tensor of shape (num_landscapes,)
    """
    num_landscapes, batch_size, L, A = seqs.shape

    assert targets.shape == (num_landscapes, batch_size)
    assert correlation_matrices.shape == (num_landscapes, L, A, A)
    assert seqs.dtype == targets.dtype and seqs.device == targets.device
    assert correlation_matrices.dtype == seqs.dtype
    assert correlation_matrices.device == seqs.device
    assert signal_to_noise_ratio.shape == ()

    if mask is not None:
        assert mask.shape == (num_landscapes, batch_size)
        assert mask.dtype == seqs.dtype and mask.device == seqs.device
        seqs = seqs * mask[..., None, None]

    kernels = torch.einsum("nbla,nBlA,nlaA->nbB", seqs, seqs, correlation_matrices)
    kernels = signal_to_noise_ratio * kernels + \
        torch.eye(batch_size, device=kernels.device, dtype=kernels.dtype)

    Ls = torch.linalg.cholesky(kernels, upper=False)
    L_inv_y = torch.linalg.solve_triangular(Ls, targets[..., None], upper=False).squeeze(-1)
    assert L_inv_y.shape == (num_landscapes, batch_size)

    if mask is None:
        num_seqs_per_landscape = batch_size
        sigma_f_hat_sq = torch.einsum("nb,nb->n", L_inv_y, L_inv_y) / batch_size
    else:
        num_seqs_per_landscape = mask.sum(1)
        dot_products = torch.einsum("nb,nb,nb->n", L_inv_y, L_inv_y, mask)
        sigma_f_hat_sq = dot_products / num_seqs_per_landscape
    assert sigma_f_hat_sq.shape == (num_landscapes,)

    loss = 0.5 * sigma_f_hat_sq.log() + \
        torch.diagonal(Ls, dim1=-1, dim2=-2).log().sum(-1) / num_seqs_per_landscape
    assert loss.shape == (num_landscapes,)
    return loss

```